

Advanced Python Programming

TTPS4805-GKJ

John Strickler

Version 2.2.3, February 2026

Table of Contents

About Advanced Python Programming: TTPS4805-GKJ	1
Course Outline	2
Student files	3
Examples	4
Appendices	5
Classroom etiquette	6
Chapter 1: Intermediate Classes	7
What is a class?	8
Defining Classes	9
Object Instances	10
Instance attributes	11
Instance Methods	12
Constructors	13
Properties	14
Class Data	17
Class Methods	19
Inheritance	21
Using super()	22
Multiple Inheritance	27
Abstract base classes	30
Special Methods	32
Static Methods	38
Chapter 2: Unit Testing with pytest	40
What is a unit test?	41
The pytest module	42
Creating tests	44
Running tests (basics)	46
Special assertions	47
Fixtures	50
User-defined fixtures	51
Builtin fixtures	54
Configuring fixtures	58
Parametrizing tests	61
Marking tests	64
Running tests (advanced)	66
Making test files executable	68

Skipping and failing	69
Mocking data	72
Mocking in pytest	73
Mock where the object is used	74
Pytest plugins	81
Chapter 3: Serializing Data	83
Reading CSV data	84
Customizing CSV readers and writers	85
Using csv.DictReader	88
Writing CSV Data	90
About JSON	93
Reading JSON	94
Writing JSON	97
Customizing JSON	99
Reading and writing YAML	102
Which XML module to use?	107
Getting Started With ElementTree	108
How ElementTree Works	109
Elements	110
Parsing An XML Document	113
Navigating the XML Document	114
Using XPath	118
Creating a New XML Document	121
Pickle	124
Chapter 4: Metaprogramming	129
Metaprogramming	130
globals() and locals()	131
Using <code>dir()</code>	134
Working with attributes	136
Adding instance methods	138
The inspect module	140
Callable classes	144
Decorators	147
Applying decorators	149
Trivial Decorator	151
Simple decorator functions	152
Decorator Classes	155
Decorators with arguments	158

Assigning decorators to imported functions	160
Creating classes at runtime	161
Monkey Patching	163
Chapter 5: Iterables and Generators	167
Iterables	168
What <i>exactly</i> is an iterable?	169
Iterators	170
How a for loop really works	172
Iterator classes	173
Generator Expressions	175
Generator functions	178
Iterable unpacking	181
Unpacking function arguments	186
Chapter 6: Consuming RESTful Data	190
The REST API	191
When is REST not REST?	193
Consuming REST APIs	194
Printing JSON	199
Requests sessions	200
Posting data to a RESTful server	203
Other operations	205
Using Thunder Client	208
Chapter 7: Database Access	210
The DB API	211
Connecting to a Server	213
connect() examples	214
Creating a Cursor	216
Querying data	217
Fetch methods	220
Non-query statements	223
SQL Injection	226
Parameterized Statements	228
Metadata	240
Dictionary Cursors	243
Generic alternate cursors	247
Transactions	251
Autocommit	253
Object-relational Mappers	254

NoSQL	255
Chapter 8: An Overview of Flask	258
What is Flask?	259
Origins and purpose	260
Microframework vs Framework	261
Views, controllers, but no models	262
Extensions	263
Who's using Flask?	264
Flask is server-side software	265
Chapter 9: Jinja Templates	266
Using Jinja templates	267
Jinja variables and filters	273
Jinja control structures	277
Jinja inheritance	280
Adding Bootstrap to templates	283
Macros	287
Extra topics	290
Chapter 10: Flask REST Overview	291
What is REST?	292
Base REST constraints	293
Designing a REST API	294
Using jsonify	295
Setting up RESTful routes	296
Choosing the return type	299
Handling invalid requests	301
Receiving data from POST	302
Handling a DELETE request	303
Helper modules	304
Chapter 11: Flask RESTful	306
About Flask extensions	307
Using Flask-RESTX	309
Creating resources	310
Adding resources	312
Trying out the API	314
Serializing data	317
About SQLAlchemy	321
Using SQLAlchemy in resources	323
Filtering queries	328

Chapter 12: Container Classes	330
Container classes	331
Builtin containers	332
Containers in the standard library	335
Emulating builtin types	344
Creating list-like containers	348
Using collections.abc	351
Creating dict-like containers	354
Free-form containers	357
Chapter 13: Beautiful Soup	359
Using requests	360
Using Beautiful Soup	365
Getting Started	366
Retrieving information	367
Search Criteria	368
Modifying the Document	372
Chapter 14: Advanced Data Handling	377
Deep vs shallow copying	378
Default dictionary values	380
Counting with Counter	382
Named Tuples	384
Alternatives to collections.namedtuple	387
Printing data structures	388
Zipped archives	392
Making archives with shutil	395
Serializing Data with pickle	396
Chapter 15: Concurrency	400
Concurrency	401
Threads	402
The Python Thread Manager	403
The threading Module	404
Threads for the impatient	405
Subclassing Thread	407
Variable sharing	410
Thread coordination	413
Using queues	417
Debugging threaded Programs	420
The multiprocessing module	421

Using pools	424
Alternatives to <code>P00L.map()</code>	429
Alternatives to threading and multiprocessing	430
Appendix A: Field Guide to Python Expressions	432
Appendix B: String Formatting	433
Overview	433
Parameter Selectors	434
f-strings	436
Data types	437
Field Widths	440
Alignment	443
Fill characters	446
Signed numbers	448
Parameter Attributes	451
Formatting Dates	453
Run-time formatting	457
Miscellaneous tips and tricks	459
Appendix C: Python Bibliography	461
Index	464

About Advanced Python Programming: TTPS4805-GKJ

Course Outline

Half-Day 1

Chapter 1 [Intermediate Classes](#)

Chapter 2 [Unit Testing with pytest](#)

Half-Day 2

Chapter 3 [Serializing Data](#)

Chapter 4 [Metaprogramming](#)

Chapter 5 [Iterables and Generators](#)

Half-Day 3

Chapter 6 [Consuming RESTful Data](#)

Chapter 7 [Database Access](#)

Half-Day 4

Chapter 8 [An Overview of Flask](#)

Chapter 9 [Jinja Templates](#)

Extra topics

Chapter 10 [Flask REST Overview](#)

Chapter 11 [Flask RESTful](#)

Chapter 12 [Container Classes](#)

Chapter 13 [Beautiful Soup](#)

Chapter 14 [Advanced Data Handling](#)

Chapter 15 [Concurrency](#)



The actual schedule varies with circumstances. The last day may include *ad hoc* topics requested by students

Student files

You will need to load some student files onto your computer. The files are in a compressed archive. When you extract them onto your computer, they will all be extracted into a directory named **Advanced_Python**. See the setup guides for details.

What's in the files?

Advanced_Python contains all files necessary for the class

Advanced_Python/EXAMPLES/ contains the examples from the course manuals.

Advanced_Python/ANSWERS/ contains sample answers to the labs.

Advanced_Python/DATA/ contains data used in examples and answers

Advanced_Python/SETUP/ contains any needed setup scripts (may be empty)

Advanced_Python/TEMP/ initially empty; used by some examples for output files

The following folders *may* be present:

Advanced_Python/BIG_DATA/ contains large data files used in examples and answers

Advanced_Python/NOTEBOOKS/ Jupyter notebooks for use in class

Advanced_Python/LOGS/ initially empty; used by some examples to write log files



The student files do not contain Python itself. It will need to be installed separately. This may already have been done.

Examples

Most of the examples from the course manual are provided in EXAMPLES subdirectory.

It will look like this:

Example

cmd_line_args.py

```
import sys    # Import the sys module

print(sys.argv) # Print all parameters, including script itself

name = sys.argv[1] # Get the first actual parameter
print("name is", name)
```

cmd_line_args.py apple mango 123

```
['/Users/jstrick/curr/courses/python/common/examples/cmd_line_args.py', 'apple', 'mango', '123']
name is apple
```

Appendices

Appendix A [Field Guide to Python Expressions](#)

Appendix B [String Formatting](#)

Appendix C [Python Bibliography](#)

Classroom etiquette

Remote learning

- Mic off when you're not speaking. If multiple mics are on, it makes it difficult to hear
- The instructor doesn't know you need help unless you let them know via voice or chat.
- It's ok to ask for help a lot.
 - Ask questions. Ask questions. Ask questions.
 - **INTERACT** with the instructor and other students.
- Log off the remote S/W at the end of the day

In-person learning

- Noisemakers off
- No phone conversations
- Come and go quietly during class.

Please turn off cell phone ringers and other noisemakers.

If you need to have a phone conversation, please leave the classroom.

We're all adults here; feel free to leave the classroom if you need to use the restroom, make a phone call, etc. You don't have to wait for a lab or break, but please try not to disturb others.



Please do not bring any exploding penguins to class. They might maim, dismember, or otherwise disturb your fellow students.

Chapter 1: Intermediate Classes

Objectives

- Defining a class and its constructor
- Creating object methods
- Adding properties to a class
- Working with class data and methods
- Leveraging inheritance for code reuse
- Implementing special methods
- Knowing when NOT to use classes

Random historic note: if I hadn't chosen the `__dunder__` naming scheme for Python language internals long ago, dunders would have been an obscure feature of the C preprocessor.

— Guido van Rossum, Twitter, April 2020

What is a class?

- Represents a *thing*
- Encapsulates functions and variables
- Creator of object *instances*
- Basic unit of object-oriented programming

A class is definition that represents a *thing*. The thing could be a file, a process, a database record, a strategy, a string, a person, or a truck.

The class describes both data, which represents one instance of the thing, and methods, which are functions that act upon the data. There can be both class data, which is shared by all instances, and instance data, which is only accessible from the instance.



Classes are a very powerful tool to organize code. However, there are some circumstances in Python where classes are not needed. If you just need some functions, and they don't need to share or remember data, just put the functions in a module. If you just need some data, but you don't need functions to process it, just used a nested data structure built out of dictionaries, lists, and tuples, as needed.

Defining Classes

- Syntax

```
class ClassName(base_class,...):  
    # class body → methods and data
```

- Specify base classes
- Use StudlyCaps for name

The **class** statement defines a class and assigns it to a name.

The simplest form of class definition looks like this:

```
class ClassName():  
    pass
```

Normally, the contents of a class definition will be method definitions and shared data.

A class definition creates a new local namespace. All variable assignments go into this new namespace. All methods are called via the instance or the class name.

A list of base classes may be specified in parentheses after the class name.

Object Instances

- Call class name as a function
- ***self*** contains attributes
- Syntax

```
obj = ClassName(args...)
```

An object instance is an object created from a class. Each object instance has its own private attributes, which are usually created in the `__init__()` method.



For an entertaining explanation of how classes work in Python, see Raymond Hettinger's talk ***Python's Class Development Toolkit***: <https://www.youtube.com/watch?v=HTLu2DFOdTg>

Instance attributes

- Methods and data
- Accessed using dot notation
- Privacy by convention (`_name`)

An instance of a class (AKA object) normally contains methods and data. To access these attributes, use "dot notation": `object.attribute`.

Instance attributes are dynamic; they can be accessed directly from the object. You can create, update, and delete attributes in this way.

Attributes cannot be made private, but names that begin with an underscore are understood by convention to be for internal use only. Users of your class will not consider methods that begin with an underscore to be part of your class's API.

Example

```
class Spam():
    def eggs(self):
        pass

    def _beverage(self):  # private!
        pass

s = Spam()
s.eggs()
s.toast = 'buttered'
print(s.toast)

s._beverage()  # legal, but wrong!
```

Note that you can just create an attribute named *toast* without defining it anywhere. However, in most cases, it is better to use properties (described later) to access data attributes.

Instance Methods

- Called from objects
- Object is implicit parameter

An instance method is a function defined in a class. When a method is called from an object, the object is passed in as the implicit first parameter, named **self** by strong convention.

Example

rabbit.py

```
class Rabbit:

    def __init__(self, size, danger): # constructor, passed self
        self._size = size
        self._danger = danger
        self._victims = []

    def threaten(self): # instance method, passed self
        print(f"I am a {self._size} bunny with {self._danger}!")

r1 = Rabbit('large', "sharp, pointy teeth") # pass parameters to constructor
r1.threaten() # instance method has access to variables via self

r2 = Rabbit('small', 'fluffy fur')
r2.threaten()
```

rabbit.py

```
I am a large bunny with sharp, pointy teeth!
I am a small bunny with fluffy fur!
```

Constructors

- Named `__init__()`
- Implicitly called when object is created
- **self** is object *itself*

If a class defines a method named `__init__()`, it will be automatically called when an object instance is created. This is the *constructor*.

The object being created is implicitly passed as the first parameter to `__init__()`. This parameter is named **self** by very strong convention. Data attributes can be assigned to **self**. These attributes can then be accessed by other methods.

Example

```
class Rabbit:

    def __init__(self, size, danger):
        self._size = size
        self._danger = danger
        self._victims = []
```



In C++, Java, and C#, **self** is called **this**.

Properties

- Accessed like variables
- Invoke implicit getters and setters
- Can be read-only

While object attributes can be accessed directly, in many cases the class needs to exercise some control over the attributes.

A more elegant approach is to use properties. A property is a kind of managed attribute. Properties are accessed directly, like normal attributes (variables), but getter, setter, and deleter functions are implicitly called, so that the class can control what values are stored or retrieved from the attributes.

You can create getter, setter, and deleter properties.

To create the getter property (which must be created first), apply the **@property** decorator to a method with the name you want. It receives no parameters other than **self**.

To create the setter property, create another function with the property name (yes, there will be two function definitions with the same name). Decorate this with the property name plus ".setter". In other words, if the property is named "spam", the decorator will be "@spam.setter". The setter method will take one parameter (other than self), which is the value assigned to the property.

It is common for a setter property to raise an error if the value being assigned is invalid.

While you seldom need a deleter property, creating it is the same as for a setter property, but use "@propertyname.deleter".



You can also write getter and setter methods, but properties are preferred for readability.

Example

knight.py

```
class Knight():
    def __init__(self, name, title, color):
        self._name = name
        self._title = title
        self._color = color

    @property # getter property decorator
    def name(self): # property implemented by name() method
        return self._name

    @property
    def color(self):
        return self._color

    @color.setter # setter property decorator
    def color(self, color):
        self._color = color

    @property
    def title(self):
        return self._title

if __name__ == '__main__':
    k = Knight("Lancelot", "Sir", 'blue')

    # Bridgekeeper's question
    print(f'Sir {k.name}, what is your...favorite color?') # use property

    # Knight's answer
    print(f"red, no -- {k.color}!")

    k.color = 'red' # set property

    print(f"color is now: {k.color}")
```

knight.py

```
Sir Lancelot, what is your...favorite color?  
red, no -- blue!  
color is now: red
```

Class Data

- Attached to class, not instance
- Shared by all instances

Data can be attached to the class itself, and shared among all instances. Class data can be accessed via the class name from inside or outside of the class.

Any class attribute not overwritten by an instance attribute is also available through the instance.

Example

class_data.py

```
class Rabbit:
    LOCATION = "the Cave of Caerbannog" # class data

    def __init__(self, weapon):
        self.weapon = weapon

    def display(self):
        print("This rabbit guarding {} uses {} as a weapon".
              format(self.LOCATION, self.weapon)) # look up class data via instance

r1 = Rabbit("a nice cup of tea")
r1.display() # instance method uses class data

r1 = Rabbit("big pointy teeth")
r1.display() # instance method uses class data
```

class_data.py

```
This rabbit guarding the Cave of Caerbannog uses a nice cup of tea as a weapon
This rabbit guarding the Cave of Caerbannog uses big pointy teeth as a weapon
```

Class Methods

- Called from class or instance
- Use @classmethod to define
- First (implicit) parameter named "cls" by convention

If a method only needs class attributes, it can be made a class method via the @classmethod decorator. This alters the method so that it gets a copy of the class object rather than the instance object. This is true whether the method is called from the class or from an instance.

The parameter to a class method is named **cls** by strong convention.

Example

class_methods_and_data.py

```
class Rabbit:
    LOCATION = "the Cave of Caerbannog" # class data (not duplicated in instances)

    def __init__(self, weapon):
        self.weapon = weapon

    def display(self):
        print("This rabbit guarding {} uses {} as a weapon".
              format(self.LOCATION, self.weapon)) # instance method

    @classmethod # the @classmethod decorator makes a function receive the class object,
    not the instance object
    def get_location(cls): # get_location() is a _class_ method
        return cls.LOCATION # class methods can access class data via cls

r = Rabbit("a nice cup of tea")
print(Rabbit.get_location()) # call class method from class
print(r.get_location()) # call class method from instance
```

class_methods_and_data.py

```
the Cave of Caerbannog
the Cave of Caerbannog
```

Inheritance

- Specify base class in class definition
- Call base class constructor explicitly

Any language that supports classes supports *inheritance*. One or more base classes may be specified as part of the class definition. All of the previous examples in this chapter have used the default base class, `object`.

The base class must already be imported, if necessary. If a requested attribute is not found in the class, the search looks in the base class. This rule is applied recursively if the base class itself is derived from some other class. For instance, all classes inherit the implementation from **`object`**, unless a class explicitly implements it.

Classes may override methods of their base classes. (For Java and C++ programmers: all methods in Python are effectively virtual.)

To extend rather than simply replace a base class method, call the base class method directly: `BaseClassName.methodname(self, arguments)`.

Using super()

- Follows MRO (method resolution order) to find function
- Great for single inheritance tree
- Use explicit base class names for multiple inheritance
- Syntax:

```
super().method()
```

The **super()** function can be used in a class to invoke methods in base classes. It searches the base classes and their bases, recursively, from left to right until the method is found.

The advantage of super() is that you don't have to specify the base class explicitly, so if you change the base class, it automatically does the right thing.

For classes that have a single inheritance tree, this works great. For classes that have a diamond-shaped tree, super() may not do what you expect. In this case, using the explicit base class name is best.

```
class Foo(Bar):  
    def __init__(self):  
        super().__init__()    # same as Bar.__init__(self)
```

Example

animal.py

```
class Animal:
    _count: int = 0 # class data

    def __init__(self, species, name, sound):
        self._species = species
        self._name = name
        self._sound = sound
        Animal._count += 1

    @property
    def species(self):
        return self._species

    def die(cls):
        Animal._count -= 1

    @classmethod
    def kill(cls, obj):
        obj.die()

    @property
    def name(self):
        return self._name

    def make_sound(self):
        print(self._sound)

    @classmethod
    def count(cls): # zoo_size gets class object when called from instance or class
        return cls._count

if __name__ == "__main__":
    leo = Animal("African lion", "Leo", "Roarrrrrrrr")
    garfield = Animal("cat", "Garfield", "Meowwww")
    felix = Animal("cat", "Felix", "Meowwww")

    for animal in leo, garfield, felix:
        print(f"{animal.name} is species {animal.species} --", end=' ')
        animal.make_sound()

    print(animal.count())
    animal.kill(leo)
    print(animal.count())
```

```
garfield.die()
print(animal.count())
```

insect.py

```
from animal import Animal

class Insect(Animal):
    """
    An animal with 2 sets of wings and 3 pairs of legs
    """

    def __init__(self, species, name, sound, can_fly=True): # constructor (AKA
initializer)
        super().__init__(species, name, sound) # call base class constructor
        self._can_fly: int = can_fly

    @property
    def can_fly(self): # "getter" property
        return self._can_fly

if __name__ == '__main__':
    mon = Insect('monarch butterfly', 'Mary', None) # defaults to can_fly being True
    scar = Insect('scarab beetle', 'Rupert', 'Bzzz', False)

    for insect in mon, scar:
        flying_status = 'can' if insect.can_fly else "can't"
        print(f"Hi! I am {insect.name} the {insect.species} and I {flying_status} fly!")
        insect.make_sound() # .make_sound inherited from Animal
        print()
```

insect.py

```
Hi! I am Mary the monarch butterfly and I can fly!
None

Hi! I am Rupert the scarab beetle and I can't fly!
Bzzz
```

Example**mammal.py**

```
from animal import Animal

class Mammal(Animal): # inherit from Animal
    def __init__(self, species, name, sound, gestation):
        super().__init__(species, name, sound)
        self._gestation: int = gestation

    @property
    def gestation(self): # add property to existing Animal properties
        """Length of gestation period in days"""
        return self._gestation

if __name__ == "__main__":
    mammal1 = Mammal("African lion", "Bob", "Roarrrr", 120)
    print(mammal1.name, "is a", mammal1.species, "--", end=' ')
    mammal1.make_sound()

    print("Number of animals", mammal1.count())

    mammal2 = Mammal("Fruit bat", "Freddie", "Squeak!!", 180)
    print(mammal2.name, "is a", mammal2.species, "--", end=' ')
    mammal2.make_sound()

    print("Number of animals mammal2.count()", mammal2.count())
    print("Number of animals Mammal.count()", Mammal.count())
    print("Number of animals Animal.count()", Animal.count())

    mammal1.die()
    print("Number of animals", Mammal.count())

    print(f"Gestation period of the {mammal1.species} is {mammal1.gestation} days")
```



```
print(f"Gestation period of the {mammal2.species} is {mammal2.gestation} days")

print(Animal.count())
Animal.kill(mammal1)
print(Animal.count())
```

mammal.py

```
Bob is a African lion -- Roarrrr
Number of animals 1
Freddie is a Fruit bat -- Squeak!!
Number of animals mammal2.count() 2
Number of animals Mammal.count() 2
Number of animals Animal.count() 2
Number of animals 1
Gestation period of the African lion is 120 days
Gestation period of the Fruit bat is 180 days
1
0
```

Multiple Inheritance

- More than one base class
- All data and methods are inherited
- Methods resolved left-to-right, depth-first

Python classes can inherit from more than one base class. This is called "multiple inheritance".

Classes designed to be added to a base class are sometimes called "mixin classes", or just "mixins".

Methods are searched for in the first base class, then its parents, then the second base class and parents, and so forth.

Put the "extra" classes before the main base class, so any methods in those classes will override methods with the same name in the base class.



To find the exact method resolution order (MRO) for a class, call the class's **mro()** method.

Example

multiple_inheritance.py

```
class AnimalBase(): # create primary base class
    def __init__(self, name):
        self._name = name

    def get_id(self):
        print(self._name)

class CanBark(): # create additional (mixin) base class
    def bark(self):
        print("woof-woof")

class CanFly(): # create additional (mixin) base class
    def fly(self):
        print("I'm flying")

class Dog(CanBark, AnimalBase): # inherit from primary base class plus mixin
    pass

class Sparrow(CanFly, AnimalBase): # inherit from primary base class plus mixin
    pass

d = Dog('Dennis')
d.get_id() # all animals have id()
d.bark() # dogs can bark() (from mixin)
print()

s = Sparrow('Steve')
s.get_id()
s.fly() # sparrows can fly() (from mixin)
print()

print("Sparrow mro:", Sparrow.mro())
```

multiple_inheritance.py

```
Dennis  
woof-woof
```

```
Steve  
I'm flying
```

```
Sparrow mro: [<class '__main__.Sparrow'>, <class '__main__.CanFly'>, <class  
'__main__.AnimalBase'>, <class 'object'>]
```

Abstract base classes

- Designed for inheritance
- Abstract methods *must* be implemented
- Non-abstract methods *may* be overwritten

The **abc** module provides abstract base classes. When a method in an abstract class is designated **abstract**, it must be implemented in any derived class. If a method is not marked abstract, it may be overwritten or extended.

To create an abstract class, import ABCMeta and abstractmethod. Create the base (abstract) class normally, but assign ABCMeta to the class option **metaclass**. Then decorated any desired abstract methods with `*@abstractmethod`.

Now, any classes that inherit from the base class must implement any abstract methods. Non-abstract methods do not have to be implemented, but of course will be inherited.



abc also provides decorators for abstract properties and abstract class methods.

Example

abstract_base_classes.py

```
from abc import ABCMeta, abstractmethod

class AnimalBase(metaclass=ABCMeta): # metaclasses control how classes are created;
    ABCMeta adds restrictions to classes that inherit from Animal

    @abstractmethod # when decorated with @abstractmethod, speak() becomes an abstract
    method
    def speak(self):
        pass

class Dog(AnimalBase): # Inherit from abstract base class Animal
    def speak(self): # speak() must be implemented
        print("woof! woof!")

class Cat(AnimalBase): # Inherit from abstract base class Animal
    def speak(self): # speak() must be implemented
        print("Meow meow meow")

class Duck(AnimalBase): # Inherit from abstract base class Animal
    pass # Duck does not implement speak()

d = Dog()
d.speak()

c = Cat()
c.speak()

try:
    d = Duck() # Duck throws a TypeError if instantiated
except TypeError as err:
    print(err)
else:
    d.speak()
```

abstract_base_classes.py

```
woof! woof!
Meow meow meow
Can't instantiate abstract class Duck with abstract method speak
```

Special Methods

- User-defined classes emulate standard types
- Define behavior for builtin functions
- Override operators

Python has a set of special methods that can be used to make user-defined classes emulate the behavior of builtin classes. These methods can be used to define the behavior for builtin functions such as `str()`, `len()` and `repr()`; they can also be used to override many Python operators, such as `+`, `*`, and `==`.

These methods expect the `self` parameter, like all instance methods. They frequently take one or more additional methods. `self` is the object being called from the builtin function, or the left operand of a binary operator such as `==`.

For instance, if your object represented a database connection, you could have `str()` return the hostname, port, and maybe the connection string. The default for `str()` is to call `repr()`, which returns something like `<main.DBConn object at 0xb7828c6c>`, which is not nearly so user-friendly.



See <http://docs.python.org/reference/datamodel.html#special-method-names> for detailed documentation on the special methods.

Table 1. Special Methods and Variables

Method or Variables	Description
<code>__new__(cls,...)</code>	Returns new object instance; Called before <code>__init__()</code>
<code>__init__(self,...)</code>	Object initializer (constructor)
<code>__del__(self)</code>	Called when object is about to be destroyed
<code>__repr__(self)</code>	Called by <code>repr()</code> builtin
<code>__str__(self)</code>	Called by <code>str()</code> builtin
<code>__eq__(self, other)</code> <code>__ne__(self, other)</code> <code>__gt__(self, other)</code> <code>__lt__(self, other)</code> <code>__ge__(self, other)</code> <code>__le__(self, other)</code>	Implement comparison operators <code>==</code> , <code>!=</code> , <code>></code> , <code><</code> , <code>>=</code> , and <code><=</code> . <code>self</code> is object on the left.
<code>__cmp__(self, other)</code>	Called by comparison operators if <code>__eq__</code> , etc., are not defined
<code>__hash__(self)</code>	Called by <code>hash()</code> builtin, also used by <code>dict</code> , <code>set</code> , and <code>frozenset</code> operations
<code>__bool__(self)</code>	Called by <code>bool()</code> builtin. Implements truth value (boolean) testing. If not present, <code>bool()</code> uses <code>len()</code>
<code>__unicode__(self)</code>	Called by <code>unicode()</code> builtin
<code>__getattr__(self, name)</code> <code>__setattr__(self, name, value)</code> <code>__delattr__(self, name)</code>	Override normal fetch, store, and deleter
<code>__getattribute__(self, name)</code>	Implement attribute access for new-style classes
<code>__get__(self, instance)</code>	<code>__set__(self, instance, value)</code>
<code>__del__(self, instance)</code>	Implement descriptors
<code>__slots__ = variable-list</code>	Allocate space for a fixed number of attributes.
<code>__metaclass__ = callable</code>	Called instead of <code>type()</code> when class is created.
<code>__instancecheck__(self, instance)</code>	Return true if instance is an instance of class
<code>__subclasscheck__(self, instance)</code>	Return true if instance is a subclass of class
<code>__call__(self, ...)</code>	Called when instance is called as a function.
<code>__len__(self)</code>	Called by <code>len()</code> builtin
<code>__getitem__(self, key)</code>	Implements <code>self[key]</code>
<code>__setitem__(self, key, value)</code>	Implements <code>self[key] = value</code>
<code>__delitem__(self, key)</code>	Implements <code>del self[key]</code>

Method or Variables	Description
<code>__iter__(self)</code>	Called when iterator is applied to container
<code>__reversed__(self)</code>	Called by reversed() builtin
<code>__contains__(self, object)</code>	Implements in operator
<code>__add__(self, other)</code> <code>__sub__(self, other)</code> <code>__mul__(self, other)</code> <code>__floordiv__(self, other)</code> <code>__mod__(self, other)</code> <code>__divmod__(self, other)</code> <code>__pow__(self, other[, modulo])</code> <code>__lshift__(self, other)</code> <code>__rshift__(self, other)</code> <code>__and__(self, other)</code> <code>__xor__(self, other)</code> <code>__or__(self, other)</code>	Implement binary arithmetic operators +, -, *, //, %, **, <<, >>, &, ^, and . Self is object on left side of expression.
<code>__div__(self, other)</code> <code>__truediv__(self, other)</code>	Implement binary division operator /. <code>__truediv__</code> is called if <code>__future__.division</code> is in effect.
<code>__radd__(self, other)</code> <code>__rsub__(self, other)</code> <code>__rmul__(self, other)</code> <code>__rdiv__(self, other)</code> <code>__rtruediv__(self, other)</code> <code>__rfloordiv__(self, other)</code> <code>__rmod__(self, other)</code> <code>__rdivmod__(self, other)</code> <code>__rpow__(self, other)</code> <code>__rlshift__(self, other)</code> <code>__rrshift__(self, other)</code> <code>__rand__(self, other)</code> <code>__rxor__(self, other)</code> <code>__ror__(self, other)</code>	Implement binary arithmetic operators with swapped operands. (Used if left operand does not support the corresponding operation)
<code>__iadd__(self, other)</code> <code>__isub__(self, other)</code> <code>__imul__(self, other)</code> <code>__idiv__(self, other)</code> <code>__itruediv__(self, other)</code> <code>__ifloordiv__(self, other)</code> <code>__imod__(self, other)</code> <code>__ipow__(self, other[, modulo])</code> <code>__ilshift__(self, other)</code> <code>__irshift__(self, other)</code> <code>__iand__(self, other)</code> <code>__ixor__(self, other)</code> <code>__ior__(self, other)</code>	Implement augmented (+=, -=, etc.) arithmetic operators

Method or Variables	Description
<code>__neg__(self)</code> <code>__pos__(self)</code> <code>__abs__(self)</code> <code>__invert__(self)</code>	Implement unary arithmetic operators -, +, abs(), and ~
<code>__oct__(self)</code> <code>__hex__(self)</code>	Implement oct() and hex() builtins
<code>__index__(self)</code>	Implement operator.index()
<code>__coerce__(self, other)</code>	Implement "mixed-mode" numeric arithmetic.

specialmethods.py

```

class Special():

    def __init__(self, value):
        self._value = str(value) # all Special instances are strings

    def __add__(self, other): # define what happens when a Special instance is added to
        another Special object
        return self._value + other._value

    def __mul__(self, num): # define what happens when a Special instance is multiplied
        by a value
        return ''.join((self._value for i in range(num)))

    def __str__(self): # define what happens when str() called on a Special instance
        return self._value.upper()

    def __eq__(self, other): # define equality between two Special values
        return self._value == other._value

if __name__ == '__main__':
    s = Special('spam')
    t = Special('eggs')
    u = Special\
        ('spam')
    v = Special(5) # parameter to Special() is converted to a string
    w = Special(22)

    print("s + s", s + s) # add two Special instances
    print("s + t", s + t)
    print("t + t", t + t)
    print("s * 10", s * 10) # multiply a Special instance by an integer
    print("t * 3", t * 3)
    print(f"str(s)={s!s} str(t)={t!s}")
    print(f"id(s)={id(s)} id(t)={id(t)} id(u)={id(u)}")
    print("s == s", s == s)
    print("s == t", s == t)
    print("s == u", s == u)
    print("v + v", v + v)
    print("v + w", v + w)
    print("w + w", w + w)
    print("v * 10", v * 10)
    print("w * 3", w * 3)

```

specialmethods.py

```
s + s spamspam
s + t spameggs
t + t eggseggs
s * 10 spamspamspamspamspamspamspamspam
t * 3 eggseggseggs
str(s)=SPAM str(t)=EGGS
id(s)=4490970576 id(t)=4490970960 id(u)=4490974672
s == s True
s == t False
s == u True
v + v 55
v + w 522
w + w 2222
v * 10 5555555555
w * 3 222222
```

Static Methods

- Related to class, but doesn't need instance or class object
- Use `@staticmethod` decorator

A static method is a utility method that is related to the class, but does not need the instance or class object. Thus, it has no automatic parameter.

One use case for static methods is to factor some kind of logic out of several methods, when the logic doesn't require any of the data in the class.



Static methods are seldom needed.

Chapter 1 Exercises

Exercise 1-1 (president.py, president_main.py)

Create a module that implements a **President** class. This class has a constructor that takes the index number of the president (1-N) and creates an object containing the associated information from the `presidents.txt` file in the `DATA` folder.

Provide the following properties (types indicated after ->):

```
term_number -> int
first_name -> string
last_name -> string
birth_date -> date object
death_date -> date object (or None, if still alive)
birth_place -> string
birth_state -> string
term_start_date -> date object
term_end_date -> date object (or None, if still in office)
party -> string
```

HINT: Only implement the first 3 properties until you get everything working. Then it's easy to add the other properties.

Write a main script to exercise some or all of the properties. It should look something like this:

```
from president import President

p = President(1)    # George Washington
print("George was born at {0}, {1} on {2}".format(
    p.birth_place, p.birth_state, p.birth_date
))
```

Chapter 2: Unit Testing with pytest

Objectives

- Understand the purpose of unit tests
- Design and implement unit tests with pytest
- Run tests in different ways
- Use builtin fixtures
- Create and use custom fixtures
- Mark tests for running in groups
- Learn how to mock data for tests

What is a unit test?

- Tests *unit* of code in isolation
- Ensures repeatable results
- Asserts expected behavior

A *unit test* is a test which asserts that an isolated piece of code (one function, method, class, or module) has some expected behavior. It is a way of making sure that code provides repeatable results.

There are four main components of a unit testing system:

1. Unit tests – individual assertions that an expected condition has been met
2. Test cases – collections of related unit tests
3. Fixtures — provide data to set up tests in order to get repeatable results
4. Test runners – utilities to execute the tests in one or more test cases

Unit tests should each test one aspect of your code, and each test should be independent of all other tests, including the order in which tests are run.

Each test asserts that some condition is true.

Unit tests may be collected into a **test case**, which is a related group of unit tests. With `pytest`, a test case can be either a module or a class.

Fixtures provide repeatable, known input to a test.

The final component is a **Test runner**, which executes one, some, or all tests and reports on the results. There are many different test runners for `pytest`. The builtin runner is very flexible.

The pytest module

- Provides
 - test runner
 - fixtures
 - special assertions
 - extra tools
- Not based on xUnit¹

The **pytest** module provides tools for creating, running, and managing unit tests.

Here's how **pytest** implements the main components of unit testing:

unit test

A normal Python function that uses the **assert** statement to assert some condition is true

test case

A class that contains a group of related unit tests (tests can be also be grouped with *markers*).

fixture

A function that provides test resources (fixtures can be nested).

test runner

A text-based test runner is built in, and there are many third-party test runners



pytest is more flexible than classic **xUnit** implementations. For example, fixtures can be associated with any number of individual tests, or with a test class. Test cases need not be classes. These features help make it popular.

¹ The builtin unit testing module, **unittest**, is based on **xUnit** patterns, as implemented in Java and other languages. The **pytest** test runner will detect **unittest** tests as well. This can be handy for transitioning legacy code to **pytest**.

Installing the modules to test

Before running some tests, `spamlib` and `hamlib` must be locally installed on the computer where you're taking the class; otherwise the tests won't be able to import them. You can install them in editable mode like this:

From the EXAMPLES folder:

```
cd hamlib
pip install -e .

cd ../spamlib
pip install -e .
```

Creating tests

- Create test functions
- Use builtin `assert`
- Confirm something is true
- Optional message

To create a test, create a function whose name begins with "test". These should normally be in a separate script, whose name begins with "test_" or ends with "_test". For the simplest cases, tests do not even need to import `pytest`.

Each test function will use the builtin `assert` statement one or more times to confirm that that some condition is true. If all of the assertions are True, then the test passes. If any assertion fails, the test fails.

`pytest` will print an appropriate message by introspecting the expression, or you can add your own failure message after the expression, separated by a comma.

```
assert result == 100
assert 2 == 2, "Two is not equal to two!"
```

It is a good idea to make test names verbose. This will help when running tests in verbose mode, so you can see what tests are passing (or failing).

Real-life unit tests

`requests` is one of the most commonly used Python modules outside of the standard library. It provides an HTTP client with many helpful options. Here are some of the unit tests for `requests`:

https://github.com/psf/requests/blob/main/tests/test_requests.py

Example

tests/test_simple.py

```
# tests should begin with "test" for automatic discovery
def test_two_plus_two_equals_four():
    assert 2 + 2 == 4    # if assert statement succeeds, the test passes

def test_three_is_greater_than_two():
    assert 3 > 2
```

pytest -v tests/test_simple.py

```
===== test session starts =====
platform darwin -- Python 3.11.6, pytest-9.0.2, pluggy-1.5.0 -- /usr/local/bin/python
cachedir: .pytest_cache
rootdir: /Users/jstrick/curr/courses/python/common/examples/tests
configfile: pytest.ini
plugins: cov-6.0.0, anyio-4.0.0, Faker-37.1.0, mock-3.12.0, django-4.12.0
collecting ... collected 2 items

tests/test_simple.py::test_two_plus_two_equals_four PASSED          [ 50%]
tests/test_simple.py::test_three_is_greater_than_two PASSED         [100%]

===== 2 passed in 0.18s =====
```

Running tests (basics)

- Needs a test runner
- `pytest` provides `pytest` script

To actually run tests, you need a *test runner*. A test runner is software that runs one or more tests and reports the results.

`pytest` provides a script (also named `pytest`) to run tests from the command line (if using Anaconda, do this from an Anaconda prompt).

You can run a single test, a test case, a module, or all tests in a folder and all its subfolders.

```
pytest test_...py
```

to run the tests in a particular module, and

```
pytest -v test_...py
```

to add verbose output.

By default, `pytest` captures (and does not display) anything written to `stdout/stderr`. If you want to see the output of `print()` statements in your tests, add the `-s` option, which turns off output capture.

```
pytest -s ...
```



If `pytest` does not run from the command line, use `python -m pytest`.



In older versions of `pytest`, the test runner script was named `py.test`. While newer versions support that name, the developers recommend only using `pytest`.

Special assertions

- Special cases
 - `pytest.raises()`
 - `pytest.approx()`

There are two special cases not easily handled by `assert`.

`pytest.raises`

For testing whether an exception is raised, use `pytest.raises()`. This should be used with the **with** statement:

```
with pytest.raises(ValueError):  
    w = Wombat('blah')
```

The assertion will succeed if the code inside the **with** block raises the specified error.

`pytest.approx`

For testing whether two floating point numbers are *close enough* to each other, use `pytest.approx()`:

```
assert result == pytest.approx(1.55)
```

The default tolerance is 1e-6 (one part in a million). You can specify the relative or absolute tolerance to any degree. Infinity and NaN are special cases. NaN is normally not equal to anything, even itself, but you can specify `nanok=True` as an argument to `approx()`.



See <https://docs.pytest.org/en/latest/reference.html#pytest-approx> for more information on `pytest.approx()`

Example

tests/test_special_assertions.py

```
import pytest
import math

FILE_NAME = 'IDONOTEXIST.txt'

#####
# subject under test                                     #
#####
def read_file_data(file_name):
    with open(file_name) as file_in:
        data = file_in.read().splitlines()
        return data
#####

def test_missing_filename():
    """
    Assert FileNotFoundError is raised
    """
    with pytest.raises(FileNotFoundError):
        read_file_data(FILE_NAME) # will pass test if file is NOT found

def test_floats_approximately_match():
    # fail unless values are within 0.000001 of each other
    # (actual result is 0.30000000000000004)
    assert (.1 + .2) == pytest.approx(.3)

def test_22_div_7_is_approximately_pi():
    # Default tolerance is 0.000001
    # smaller (or larger) tolerance can be specified
    assert 22 / 7 == pytest.approx(math.pi, .001)
```

pytest -v tests/test_special_assertions.py

```
===== test session starts =====
platform darwin -- Python 3.11.6, pytest-9.0.2, pluggy-1.5.0 -- /usr/local/bin/python
cachedir: .pytest_cache
rootdir: /Users/jstrick/curr/courses/python/common/examples/tests
configfile: pytest.ini
plugins: cov-6.0.0, anyio-4.0.0, Faker-37.1.0, mock-3.12.0, django-4.12.0
collecting ... collected 3 items

tests/test_special_assertions.py::test_missing_filename PASSED [ 33%]
tests/test_special_assertions.py::test_floats_approximately_match PASSED [ 66%]
tests/test_special_assertions.py::test_22_div_7_is_approximately_pi PASSED [100%]

===== 3 passed in 0.18s =====
```


Fixtures

- Provide resources for tests
- Implement as functions
- Scope
 - Per test
 - Per class
 - Per module
- Source of fixtures
 - Builtin
 - User-defined

When writing tests for a particular object, many tests might require an instance of the object. This instance might be created with a particular set of arguments.

What happens if twenty different tests instantiate a particular object, and the object's API changes? Now you have to make changes in twenty different places.

To avoid duplicating code across many tests, pytest supports *fixtures*, which are functions that provide information to tests. The same fixture can be used by many tests, which lets you keep the fixture creation in a single place.

A fixture provides items needed by a test, such as data, functions, or class instances. A fixtures can be either builtin or custom.

What fixtures provide

Consistency

test uses the same, repeatable data

Readability

keeps test itself short and simple

Auto-use

Reduces number of imports

Teardown

Provides cleanup capabilities



Use `pytest --fixtures` to list all available builtin and user-defined fixtures.

User-defined fixtures

- Decorate with **pytest.fixture**
- Return value to be used in test
- Fixtures may be nested

To create a fixture, decorate a function with **pytest.fixture**. Whatever the function returns is the value of the fixture.

To use the fixture, pass it to the test function as a parameter. The return value of the fixture will be available as a local variable in the test.

Fixtures can take other fixtures as parameters as well, so they can be nested to any level.

It is convenient to put fixtures into a separate module so they can be shared across multiple test scripts.



Add docstrings to your fixtures and the docstrings will be displayed via **pytest --fixtures**

Example

tests/test_simple_fixture.py

```
import os
import pytest

from .silly import Silly

@pytest.fixture
def silly_object():
    return Silly() # fixture returns instance of Silly

def test_silly_triples_value(silly_object): # pass fixture as test parameter
    assert silly_object.triple(5) == 15

def test_silly_normalizes_string(silly_object):
    assert silly_object.normalize("  Spam  ") == "spam"
```

pytest -v tests/test_simple_fixture.py

```
===== test session starts =====
platform darwin -- Python 3.11.6, pytest-9.0.2, pluggy-1.5.0 -- /usr/local/bin/python
cachedir: .pytest_cache
rootdir: /Users/jstrick/curr/courses/python/common/examples/tests
configfile: pytest.ini
plugins: cov-6.0.0, anyio-4.0.0, Faker-37.1.0, mock-3.12.0, django-4.12.0
collecting ... collected 2 items

tests/test_simple_fixture.py::test_silly_triples_value PASSED          [ 50%]
tests/test_simple_fixture.py::test_silly_normalizes_string PASSED     [100%]

===== 2 passed in 0.18s =====
```

Builtin fixtures

- Variety of common fixtures
- Provide
 - Temp files and dirs
 - Logging
 - STDOUT/STDERR capture
 - Monkeypatching tools

Pytest provides a large number of builtin fixtures for common testing requirements.

Using a builtin fixture is like using user-defined fixtures. Just specify the fixture name as a parameter to the test. No imports are needed for this.

See <https://docs.pytest.org/en/latest/reference.html#fixtures> for details on builtin fixtures.

Example

tests/test_builtin_fixtures.py

```
import pytest

COUNTER_KEY = 'test_cache/counter'

def test_cache(cache): # cache persists values between test runs
    value = cache.get(COUNTER_KEY, 0)
    print("\nCounter before:", value)
    cache.set(COUNTER_KEY, value + 1) # cache fixture is similar to dictionary, but with
    .set() and .get() methods
    value = cache.get(COUNTER_KEY, 0) # cache fixture is similar to dictionary, but with
    .set() and .get() methods
    print("\nCounter after:", value)
    assert True # Make test successful

def hello():
    print("Hello, pytesting world")

def test_capsys(capsys):
    hello() # Call function that writes text to STDOUT
    out, err = capsys.readouterr() # Get captured output
    assert out == "Hello, pytesting world\n"

@pytest.mark.parametrize("num", range(3))
def test_temp_dir(tmpdir, num):
    print("TEMP DIR:", tmpdir, "\n") # tmpdir fixture provides unique temporary folder
    name
```

pytest -v tests/test_builtin_fixtures.py

```
===== test session starts =====
platform darwin -- Python 3.11.6, pytest-9.0.2, pluggy-1.5.0 -- /usr/local/bin/python
cachedir: .pytest_cache
rootdir: /Users/jstrick/curr/courses/python/common/examples/tests
configfile: pytest.ini
plugins: cov-6.0.0, anyio-4.0.0, Faker-37.1.0, mock-3.12.0, django-4.12.0
collecting ... collected 5 items

tests/test_builtin_fixtures.py::test_cache PASSED [ 20%]
tests/test_builtin_fixtures.py::test_capsys PASSED [ 40%]
tests/test_builtin_fixtures.py::test_temp_dir[0] PASSED [ 60%]
tests/test_builtin_fixtures.py::test_temp_dir[1] PASSED [ 80%]
tests/test_builtin_fixtures.py::test_temp_dir[2] PASSED [100%]
```

```
===== 5 passed in 0.19s =====
```

Table 2. Pytest Builtin Fixtures

Fixture	Brief Description
cache	Return cache object to persist state between testing sessions.
capsys	Enable capturing of writes (text mode) to <code>sys.stdout</code> and <code>sys.stderr</code>
capsysbinary	Enable capturing of writes (binary mode) to <code>sys.stdout</code> and <code>sys.stderr</code>
capfd	Enable capturing of writes (text mode) to file descriptors 1 and 2
capfdbinary	Enable capturing of writes (binary mode) to file descriptors 1 and 2
doctest_namespace	Return <code>dict</code> that will be injected into namespace of doctests
pytestconfig	Session-scoped fixture that returns <code>_pytest.config.Config</code> object.
record_property	Add extra properties to the calling test.
record_xml_attribute	Add extra xml attributes to the tag for the calling test.
caplog	Access and control log capturing.
monkeypatch	Return <code>monkeypatch</code> fixture providing monkeypatching tools
recwarn	Return <code>WarningsRecorder</code> instance that records all warnings emitted by test functions.
tmp_path	Return <code>pathlib.Path</code> instance with unique temp directory
tmp_path_factory	Return a <code>_pytest.tmpdir.TempPathFactory</code> instance for the test session.
tmpdir	Return <code>py.path.local</code> instance unique to each test
tmpdir_factory	Return <code>TempdirFactory</code> instance for the test session.

Configuring fixtures

- Create **conftest.py**
- Automatically included
- Provides
 - Fixtures
 - Hooks
 - Plugins
- Directory scope

The **conftest.py** file can be used to contain user-defined fixtures, as well as hooks and plugins. Subfolders can have their own **conftest.py**, which will only apply to tests in that folder.

In a test folder, define one or more fixtures in **conftest.py**, and they will be available to all tests in that folder, as well as any subfolders.

Hooks

Hooks are predefined functions that will automatically be called at various points in testing. All hooks start with **pytest_**. A **pytest.Function** object, which contains the actual test function, is passed into the hook.

For instance, **pytest_runtest_setup()** will be called before each test.



A complete list of hooks can be found here: <https://docs.pytest.org/en/latest/reference.html#hooks>

Plugins

There are many pytest plugins to provide helpers for testing code that uses common libraries, such as **Django** or **redis**.

You can register plugins in **conftest.py** like so:

```
pytest_plugins = "plugin1", "plugin2",
```

This will load the plugins.

Example

tests/conftest.py

```
#!/usr/bin/env python
from pytest import fixture
from .silly import Silly

@fixture
def silly_object():
    return Silly() # fixture returns instance of Silly

# predefined hook (all hooks start with 'pytest_')
def pytest_runtest_setup(item): # item is test function
    if "test_config" in str(item):
        print(f"Hello from setup, {item}", end=" ")
```

Example

tests/test_config.py

```
def test_one(silly_object): # unit test that uses fixture from conftest.py
    assert silly_object.triple(5) == 15
    assert silly_object.normalize("  Spam  ") == "spam"
```

pytest -v tests/test_config.py

```
===== test session starts =====
platform darwin -- Python 3.11.6, pytest-9.0.2, pluggy-1.5.0 -- /usr/local/bin/python
cachedir: .pytest_cache
rootdir: /Users/jstrick/curr/courses/python/common/examples/tests
configfile: pytest.ini
plugins: cov-6.0.0, anyio-4.0.0, Faker-37.1.0, mock-3.12.0, django-4.12.0
collecting ... collected 1 item

tests/test_config.py::test_one PASSED [100%]

===== 1 passed in 0.17s =====
```

Parametrizing tests

- Run same test on multiple values
- Add parameters to fixture decorator
- Test run once for each parameter
- Use `pytest.mark.parametrize()`

Many tests require testing a method or function against many values. Rather than writing a loop in the test, you can automatically repeat the test for a set of inputs via **parametrizing**.

Apply the `@pytest.mark.parametrize` decorator to the test. The first argument is a string with the comma-separated names of the parameters; the second argument is the list of parameters. The test will be called once for each item in the parameter list. If a parameter list item is a tuple or other multi-value object, the items will be passed to the test based on the names in the first argument.



For more advanced needs, when you need some extra work to be done before the test, you can do indirect parametrizing, which uses a parametrized fixture. See `test_parametrize_indirect.py` for an example.



The authors of pytest deliberately spelled it "parametrizing", not "parameterizing".

Example

tests/test_parametrization.py

```
import pytest
from .silly import Silly

@pytest.fixture
def silly_object():
    return Silly() # fixture returns instance of Silly

# List of values for testing containing input and expected result
test_data = [
    (5, 15), ('a', 'aaa'), (False, False), (True, 3), (0, 0), ([True], [True, True, True])
]

# Parametrize the test with the test data; the first argument is a string mapping
# parameters to the test data
@pytest.mark.parametrize("input,expected", test_data)
def test_triple(silly_object, input, expected):
    assert silly_object.triple(input) == expected # Test the function with the
parameters
```

pytest -v tests/test_parametrization.py

```
===== test session starts =====
platform darwin -- Python 3.11.6, pytest-9.0.2, pluggy-1.5.0 -- /usr/local/bin/python
cachedir: .pytest_cache
rootdir: /Users/jstrick/curr/courses/python/common/examples/tests
configfile: pytest.ini
plugins: cov-6.0.0, anyio-4.0.0, Faker-37.1.0, mock-3.12.0, django-4.12.0
collecting ... collected 6 items

tests/test_parametrization.py::test_triple[5-15] PASSED           [ 16%]
tests/test_parametrization.py::test_triple[a-aaa] PASSED          [ 33%]
tests/test_parametrization.py::test_triple[False-False] PASSED   [ 50%]
tests/test_parametrization.py::test_triple[True-3] PASSED         [ 66%]
tests/test_parametrization.py::test_triple[0-0] PASSED            [ 83%]
tests/test_parametrization.py::test_triple[input5-expected5] PASSED [100%]

===== 6 passed in 0.19s =====
```

Marking tests

- Create groups of tests ("test cases")
- Can create multiple groups
- Use `@pytest.mark.somemark`

You can mark tests with labels so that they can be run as a group. Use `@pytest.mark.marker`, where *marker* is the marker (label), which can be any alphanumeric string.

Then you can select tests which contain or match the marker.

```
pytest -m "alpha"  
pytest -m "not alpha"  
pytest -m "alpha or beta"  
pytest -m "alpha and not beta"
```

Registering markers

You can register markers in the **[pytest]** section of `pytest.ini`, so they will be listed, with a description, with `pytest --markers`:

```
[pytest]  
markers =  
    internet: test requires internet connection  
    slow: tests that take more time (omit with '-m "not slow")
```

Example

tests/test_mark.py

```
import pytest

@pytest.mark.alpha # Mark with label alpha
def test_one():
    assert 1

@pytest.mark.beta
@pytest.mark.alpha # Mark with label alpha
def test_two():
    assert 1

@pytest.mark.gamma
@pytest.mark.beta # Mark with label beta
def test_three():
    assert 1
```

pytest -v tests/test_mark.py

```
===== test session starts =====
platform darwin -- Python 3.11.6, pytest-9.0.2, pluggy-1.5.0 -- /usr/local/bin/python
cachedir: .pytest_cache
rootdir: /Users/jstrick/curr/courses/python/common/examples/tests
configfile: pytest.ini
plugins: cov-6.0.0, anyio-4.0.0, Faker-37.1.0, mock-3.12.0, django-4.12.0
collecting ... collected 3 items

tests/test_mark.py::test_one PASSED [ 33%]
tests/test_mark.py::test_two PASSED [ 66%]
tests/test_mark.py::test_three PASSED [100%]

===== 3 passed in 0.17s =====
```


Running tests (advanced)

- Run all tests
- Run by
 - file name
 - class
 - function
 - name match
 - group (mark)

`pytest` provides many ways to select which tests to run.

Running all tests

By default, `pytest` will find and run all tests in the current and any descendent directories.

Use `-s` to disable capturing, so anything written to STDOUT is displayed. Use `-v` for verbose output.

```
pytest
pytest -v
pytest -s
pytest -vs
```

Running by component

Use the node ID to select by component, such as file, class, method, or function name:

```
file::class
file::class::test
file::test
```

```
pytest test_president.py::test_dates
pytest test_president.py::test_dates::test_birth_date
```



Note that the *file* name is specified, not the *module* name.

Running by name match

Use **-k** to run all tests where the file name, test name, or marker includes a specified string.

```
pytest -k date run all tests whose name includes 'date'
```

Making test files executable

While you should normally use `pytest` or your IDE to run tests, you can also make a test script run the tests when you execute the script normally with `python`. To do this, put the following code at the bottom of the test file:

```
if __name__ == '__main__':  
    pytest.main([__file__, '-v']) # Start the test runner
```

`__file__` is the name of the current file. You can add the `-s` option as another element in the list of arguments to `pytest.main`. You can also omit the `-v` option if you don't want verbose output.



Most of the time you should just use the test runner.

Skipping and failing

- Conditionally skip tests
- Completely ignore tests
- Decorate with
 - `@pytest.mark.xfail`
 - `@pytest.mark.skip`

To skip tests conditionally (or unconditionally), use `@pytest.mark.skip()`. This is useful if some tests rely on components that haven't been developed yet, or for tests that are platform-specific.

To fail on purpose, use `@pytest.mark.xfail()`. This reports the test as "XPASS" or "xfail", but does not provide traceback. Tests marked with xfail will not fail the test suite. This is useful for writing tests for not-yet-implemented features, or for testing objects with known bugs that will be resolved later.

Example

tests/test_skip_xfail.py

```
import sys
import pytest

def test_one(): # Normal test -- no skipping or failing
    assert True

# Uncondition skip
@pytest.mark.skip(reason="can not currently test")
def test_two():
    assert True

# Skip this test if current platform is not Windows
@pytest.mark.skipif(
    sys.platform != 'win32',
    reason="only implemented on Windows"
)
def test_three():
    assert True

@pytest.mark.xfail
def test_four():
    assert False

@pytest.mark.xfail
def test_five():
    assert True
```

pytest -v tests/test_skip_xfail.py

```
===== test session starts =====
platform darwin -- Python 3.11.6, pytest-9.0.2, pluggy-1.5.0 -- /usr/local/bin/python
cachedir: .pytest_cache
rootdir: /Users/jstrick/curr/courses/python/common/examples/tests
configfile: pytest.ini
plugins: cov-6.0.0, anyio-4.0.0, Faker-37.1.0, mock-3.12.0, django-4.12.0
collecting ... collected 5 items

tests/test_skip_xfail.py::test_one PASSED [ 20%]
tests/test_skip_xfail.py::test_two SKIPPED (can not currently test) [ 40%]
tests/test_skip_xfail.py::test_three SKIPPED (only implemented on Wi...) [ 60%]
tests/test_skip_xfail.py::test_four XFAIL [ 80%]
tests/test_skip_xfail.py::test_five XPASS [100%]

===== 1 passed, 2 skipped, 1 xfailed, 1 xpassed in 0.23s =====
```

Mocking data

- Simulate behavior of actual objects
- Replace expensive dependencies (time/resources)
- Use **pytest-mock**

Some objects have dependencies which can make unit testing difficult. These dependencies may be expensive in terms of time or resources, or can be slow to access.

One solution is to create fake dependencies, called *stubs*. A stub is a function or class that acts like the actual dependency, but has hard-coded data and doesn't necessarily implement all features of the dependency.

A problem with stubs is that you end up with a lot of single-purpose stubs, which can be hard to maintain.

The better solution is to use **mock** objects, which pretend to be the real object. A mock object behaves like the original object, but is restricted and controlled in its behavior.

For instance, a class may have a dependency on a database query. A mock object may accept the query, but always returns a hard-coded set of results.

A mock object can record the calls made to it, and assert that the calls were made with correct parameters.

A mock object can be preloaded with a return value, or a function that provides dynamic (or random) return values.

Unlike a stub, a mock object is more elaborate, with record/playback capability, assertions, and many other test-related features.

Mocking in pytest

- Use `pytest-mock` plugin
 - Act as a stub
 - Patch existing code
- Can also use `unittest.mock.Mock`

The standard library provides `unittest.mock`. The `pytest-mock` plugin for `pytest` provides a wrapper around `unittest.mock`.

Once `pytest-mock` is installed, it provides a fixture named `mock`, from which you can create mock objects.

In either case, there are two primary ways of mocking dependencies. One is to provide a replacement class, function, or data object that mimics the real thing.

The second is to patch a module, which temporarily (just during the test) replaces a component with a mock version. The `mock.patch()` function replaces a component with a mock object. Any calls to the component are now recorded, and return values may be specified.

Mock where the object is used

- Don't patch original object
- Patch via subject under test

When the subject under test imports a module, it has a name that corresponds to that module. If you patch the original module, it only patches the original module, not the object pointed to by the subject under test.

For instance, if you are testing module `spam`, and it imports module `eggs` as follows, you might need to test that `spam` calls `eggs.toast`.

`spam.py`

```
from eggs import toast

toast()
```

Rather than patching `eggs.toast`, you will need to patch `spam.toast`. Always patch where the object is *used*, not where it's defined.



Mocking the correct name can be tricky. See https://nedbatchelder.com/blog/201908/why_your_mock_doesnt_work.html for more information.

Example

tests/test_mock.py

```
import pytest
import spamlib
from spamlib.spam import Spam

@pytest.fixture
def ham_value():
    return 42

@pytest.fixture
def ham_result(ham_value): # use ham_value fixture
    return ham_value * 10

def test_spam_calls_ham(mocked, ham_value, ham_result):
    # need to patch spamlib.spam.ham, not hamlib.ham
    mocked.patch("spamlib.spam.ham", return_value=ham_result)
    s = Spam(ham_value) # Create instance of Spam, which calls ham()
    assert s.value == ham_result
    assert spamlib.spam.ham.calledoncewith(ham_value)
```

pytest -v tests/test_mock.py

```
===== test session starts =====
platform darwin -- Python 3.11.6, pytest-9.0.2, pluggy-1.5.0 -- /usr/local/bin/python
cachedir: .pytest_cache
rootdir: /Users/jstrick/curr/courses/python/common/examples/tests
configfile: pytest.ini
plugins: cov-6.0.0, anyio-4.0.0, Faker-37.1.0, mock-3.12.0, django-4.12.0
collecting ... collected 0 items / 1 error

===== ERRORS =====
_____ ERROR collecting test_mock.py _____
ImportError while importing test module
'/Users/jstrick/curr/courses/python/common/examples/tests/test_mock.py'.
Hint: make sure your test modules/packages have valid Python names.
Traceback:
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/importlib/__init__.py:1
26: in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
    ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
tests/test_mock.py:3: in <module>
    from spamlib.spam import Spam
E   ModuleNotFoundError: No module named 'spamlib.spam'
```

```
===== short test summary info =====
ERROR tests/test_mock.py
!!!!!!!!!!!!!!!!!!!! Interrupted: 1 error during collection !!!!!!!!!!!!!!!!!!!!!
===== 1 error in 0.15s =====
```

Example

tests/test_mock_pymock.py

```
from spamlib import spam
PATTERN = 'bug'
STRING_TO_SEARCH = 'lightning bug'

def test_spam_search_calls_re_search(mock): # Unit test
    # Patch re.search (i.e., replace re.search with a Mock object that
    # records calls to it)
    mock.patch('spamlib.spam.re.search')

    s = spam.SpamSearch(PATTERN, STRING_TO_SEARCH) # Create instance of SpamSearch
    s.findit() # Call the method under test

    # Check that method was called just once with the expected parameters
    spam.re.search.assert_called_once_with(PATTERN, STRING_TO_SEARCH)
```

pytest -v tests/test_mock_pymock.py

```
===== test session starts =====
platform darwin -- Python 3.11.6, pytest-9.0.2, pluggy-1.5.0 -- /usr/local/bin/python
cachedir: .pytest_cache
rootdir: /Users/jstrick/curr/courses/python/common/examples/tests
configfile: pytest.ini
plugins: cov-6.0.0, anyio-4.0.0, Faker-37.1.0, mock-3.12.0, django-4.12.0
collecting ... collected 0 items / 1 error

===== ERRORS =====
_____ ERROR collecting test_mock_pymock.py _____
ImportError while importing test module
'/Users/jstrick/curr/courses/python/common/examples/tests/test_mock_pymock.py'.
Hint: make sure your test modules/packages have valid Python names.
Traceback:
/Library/Frameworks/Python.framework/Versions/3.11/lib/python3.11/importlib/__init__.py:1
26: in import_module
      return _bootstrap._gcd_import(name[level:], package, level)
      ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
tests/test_mock_pymock.py:1: in <module>
      from spamlib import spam
E   ImportError: cannot import name 'spam' from 'spamlib' (unknown location)
===== short test summary info =====
ERROR tests/test_mock_pymock.py
!!!!!!!!!!!!!!!!!!!! Interrupted: 1 error during collection !!!!!!!!!!!!!!!!!!!!!
```

```
===== 1 error in 0.14s =====
```

Example

tests/test_mock_play.py

```
import pytest
from unittest.mock import Mock

@pytest.fixture
def small_list(): # Create fixture that provides a small list
    return [1, 2, 3]

def test_m1_returns_correct_list(small_list):
    m1 = Mock(return_value=small_list) # Create mock object that "returns" a small list
    mock_result = m1('a', 'b') # Call mock object with arbitrary parameters
    assert mock_result == small_list # Check the mocked result

m2 = Mock() # Create generic mock object

m2.spam('a', 'b') # Call fake methods on mock object
m2.ham('wombat') # Call fake methods on mock object
m2.eggs(1, 2, 3) # Call fake methods on mock object

print("mock calls:", m2.mock_calls) # Mock object remembers all calls

m2.spam.assert_called_with('a', 'b') # Assert that spam() was called with parameters 'a'
and 'b'
```

pytest -v tests/test_mock_play.py

```
===== test session starts =====
platform darwin -- Python 3.11.6, pytest-9.0.2, pluggy-1.5.0 -- /usr/local/bin/python
cachedir: .pytest_cache
rootdir: /Users/jstrick/curr/courses/python/common/examples/tests
configfile: pytest.ini
plugins: cov-6.0.0, anyio-4.0.0, Faker-37.1.0, mock-3.12.0, django-4.12.0
collecting ... collected 1 item

tests/test_mock_play.py::test_m1_returns_correct_list PASSED [100%]

===== 1 passed in 0.18s =====
```

Pytest plugins

- Common plugins
 - `pytest-qt`
 - `pytest-django`

There are some plugins for `pytest` that that integrate various frameworks which would otherwise be difficult to test directly.

The **`pytest-qt`** plugin provides a `qtbott` fixture that can attach widgets and invoke events. This makes it simpler to test your custom widgets.

The **`pytest-django`** plugin allows you to run Django with `pytest`-style tests rather than the default **`unittest`** style.

See https://docs.pytest.org/en/latest/reference/plugin_list.html for a complete list of plugins. There are currently 880 plugins!

Chapter 2 Exercises

Exercise 2-1 (test_president.py)

Using `pytest`, Create some unit tests for the President class you created earlier.¹

Suggestions for tests:

- Create President objects for all current term numbers (1-*n*)
- What happens when an out-of-range term number is given?
- President 1's first name is "George"
- All presidential terms match the correct last name (use list of last names and **parametrize**)
- Confirm date fields return an object of type **datetime.date**

¹ If there was not an exercise where you created a President class, you can use **president.py** in the top-level folder of the student guide.

Chapter 3: Serializing Data

Objectives

- Read and write CSV data
- Load JSON data from strings or files
- Write JSON data to strings or files
- Read and write YAML data
- Know which modules are available to process XML
- Use lxml ElementTree to create a new XML file
- Parse an existing XML file with ElementTree
- Using XPath for searching XML nodes

Reading CSV data

- Use `csv` module
- Create `reader` object with file object or any iterable
- Iterate through reader to get rows as lists of columns

To read CSV data, first open the CSV file with the `open()` function to get a file object. Then create an instance of the `reader` class from the `csv` module, passing in the file object.

The `reader` object is an iterator over the rows (lines) in the file. It does not actually contain the data. Each row is returned as a list of the fields.



Any iterable with data in CSV format can be used in place of a file object.

Example

`csv_read.py`

```
import csv

with open('../DATA/knights.csv') as knights_in:
    rdr = csv.reader(knights_in) # create CSV reader
    for name, title, color, quest, comment, number in rdr: # Read and unpack records one
        at a time; each record is a list
        print(f'{title:4s} {name:9s} {quest}')
```

`csv_read.py`

```
King Arthur    The Grail
Sir Galahad    The Grail
Sir Robin      Not Sure
Sir Bedevere   The Grail
Sir Gawain     The Grail
Sir Hector     The Grrrrrail
Sir Launcelot  The Grail
```

...

Customizing CSV readers and writers

- Variations in how CSV data is written
- Most common alternate is for Excel
- Add parameters to reader/writer

You can customize how the CSV parser and generator work by passing extra parameters to `csv.reader()` or `csv.writer()`. You can change the field and row delimiters, the escape character, and for output, what level of quoting. This can be used for any text file, not just CSV formats.

Dialects

You can also specify a *dialect*, which is a custom set of CSV parameters. TO create a custom dialect, use `csv.register_dialect()` with the name of the dialect, then specify "`dialect=dialect`" as a parameter to `csv.reader()` or `csv.writer()`.



You could put a dialect specification in a separate module, and import it as needed.

Table 3. CSV reader/writer Parameters

Parameter	Meaning
<code>quotechar</code>	One-character string to use as quoting character (default: <code>'</code>)
<code>delimiter</code>	One-character string to use as field separator (default: <code>,</code>)
<code>skipinitialspace</code>	If True, skip white space after field separator (default: <code>False</code>)
<code>lineterminator</code>	The character sequence which terminates rows (default: depends on OS)
<code>quoting</code>	When should quotes be generated when writing CSV <code>csv.QUOTE_MINIMAL</code> – only when needed (default) <code>csv.QUOTE_ALL</code> – quote all fields <code>csv.QUOTE_NONNUMERIC</code> – quote all fields that are not numbers <code>csv.QUOTE_NONE</code> – never put quotes around fields
<code>escapechar</code>	One-character string to escape delimiter when quoting is set to <code>csv.QUOTE_NONE</code>
<code>doublequote</code>	Control quote handling inside fields. When <code>True</code> , two consecutive quotes are read as one, and one quote is written as two. (default: <code>True</code>)
<code>dialect</code>	string representing registered dialect name, such as "excel"

Example

csv_nonstandard.py

```
import csv

with open('../DATA/computer_people.txt') as computer_people_in:
    rdr = csv.reader(computer_people_in, delimiter=';') # specify alternate field
    delimiter

    # iterate over rows of data -- csv reader is an iterator

    for first_name, last_name, known_for, birth_date in rdr:
        print(f'{last_name}: {known_for}')
```

csv_nonstandard.py

```
Gates: Gates Foundation
Jobs: Apple
Wall: Perl
Allen: Microsoft
Ellison: Oracle
van Rossum: Python
Kurtz: BASIC
Hopper: COBOL
Gates: Microsoft
Zuckerberg: Facebook
Brin: Google
van Rossum: Python
Lovelace:
Page: Google
Torvalds: Linux
```

Example

csv_dialects.py

```
import csv

csv.register_dialect('colon-sep', delimiter=":")

with open('../DATA/knights.txt') as knights_in:
    reader = csv.reader(knights_in, dialect="colon-sep")
    for row in reader:
        print(row)
print()

with open('../DATA/primeministers.txt') as pm_in:
    reader = csv.reader(pm_in, dialect="colon-sep")
    for row in reader:
        print(row)
```

csv_dialects.py

```
['Arthur', 'King', 'blue', 'The Grail', 'King of the Britons']
['Galahad', 'Sir', 'red', 'The Grail', '"I could handle some more peril"']
['Lancelot', 'Sir', 'blue', 'The Grail', '"It's too perilous!"]']
['Robin', 'Sir', 'yellow', 'Not Sure', 'He boldly ran away']
['Bedevere', 'Sir', 'red, no blue!', 'The Grail', 'AARRRRRRRRGGGGHH']
['Gawain', 'Sir', 'blue', 'The Grail', 'none']

['1', 'Sir John A.', 'Macdonald', '1867-7-1', '1873-11-5', 'Glasgow, Scotland', '1867-07-01', '1873-11-05', 'Liberal-Conservative']
['2', 'Alexander', 'Mackenzie', '1873-11-7', '1878-10-8', 'Logierait, Scotland', '1873-11-07', '1878-10-08', 'Liberal']
['3', 'Sir John A.', 'Macdonald', '1878-10-17', '1891-6-6', 'Glasgow, Scotland', '1878-10-17', '1891-06-06', 'Liberal-Conservative']
```

Using csv.DictReader

- Returns each row as dictionary
- Keys are field names
- Use header or specify

Instead of the normal reader, you can create a dictionary-based reader by using the `DictReader` class.

By default, it will get field names from the first line of the file. Otherwise, you can specify a list of field names with the `fieldnames` parameter. For each row, you can then look up a field by name, rather than position.

Example

`csv_dictreader.py`

```
import csv

field_names = ['term', 'firstname', 'lastname', 'birthplace', 'state', 'party'] # field
names, which will become dictionary keys on each row

with open('../DATA/presidents.csv') as presidents_in:
    rdr = csv.DictReader(presidents_in, fieldnames=field_names) # create reader, passing
in field names (if not specified, uses first row as field names)
    for row in rdr: # iterate over rows in file
        print(f"{row['firstname']:25} {row['lastname']:12} {row['party']}")
```

csv_dictreader.py

George	Washington	no party
John	Adams	Federalist
Thomas	Jefferson	Democratic - Republican
James	Madison	Democratic - Republican
James	Monroe	Democratic - Republican
John Quincy	Adams	Democratic - Republican
Andrew	Jackson	Democratic
Martin	Van Buren	Democratic
William Henry	Harrison	Whig
John	Tyler	Whig
James Knox	Polk	Democratic
Zachary	Taylor	Whig
Millard	Fillmore	Whig
Franklin	Pierce	Democratic
James	Buchanan	Democratic
Abraham	Lincoln	Republican
Andrew	Johnson	Republican
Ulysses Simpson	Grant	Republican
Rutherford Birchard	Hayes	Republican
James Abram	Garfield	Republican

...

Writing CSV Data

- Use `csv.writer()`
- Parameter is file-like object (must implement `write()` method)
- Can specify parameters to writer constructor
- Use `writerow()` or `writerows()` to output CSV data

To output data in CSV format, first open the output file for writing using `open()` with the `w` mode argument. Then create a writer by passing the file object to `csv.writer()`.

For each row to write, call the `writerow()` method of the writer, passing in an iterable with the values for that row. If you have your data in an iterable of iterables (a two-dimensional data structure) you can use `writerows()`.



On Windows, to prevent double-spaced output, add `lineterminator='\n'` when creating a CSV writer.



Any file-like object that implements the `write()` method can be used.

Example

csv_write.py

```
import sys
import csv

chicago_data = [
    ['Name', 'Position Title', 'Department', 'Employee Annual Salary'],
    ['BONADUCE, MICHAEL J', 'POLICE OFFICER', 'POLICE', '$80724.00'],
    ['MELLON, MATTHEW J "Matt"', 'POLICE OFFICER', 'POLICE', '$75372.00'],
    ['FIERI, JOHN J', 'FIREFIGHTER-EMT', 'FIRE', '$75342.00'],
    ['GALAHAD, MERLE S', 'CLERK III', 'BUSINESS AFFAIRS', '$45828.00'],
    ['ORCATTI, JENNIFER L', 'FIRE COMMUNICATIONS OPERATOR I', 'OEMC', '$63121.68'],
    ['ASHE, JOHN W', 'FOREMAN OF MACHINISTS', 'AVIATION', '$96553.60'],
    ['SADINSKY BLAKE, MICHAEL G', 'POLICE OFFICER', 'POLICE', '$78012.00'],
    ['GRANT, CRAIG A', 'SANITATION LABORER', 'STREETS & SAN', '$69576.00'],
    ['MILLER, JONATHAN D', 'POLICE OFFICER', 'POLICE', '$75372.00'],
    ['FRANK, ARTHUR R', 'POLICE OFFICER/EXPLSV DETECT, K9 HNDLR', 'POLICE', '$87918.00'],
    ['POVOTTI, JAMES S "Jimmy P"', 'TRAFFIC CONTROL AIDE-HOURLY', 'OEMC', '$19167.20'],
    ['TRAWLER, DANIEL J', 'POLICE OFFICER', 'POLICE', '$75372.00'],
    ['SCUBA, ANDREW G', 'POLICE OFFICER', 'POLICE', '$75372.00'],
    ['SWINE, MATTHEW W', 'SERGEANT', 'POLICE', '$99756.00'],
    ['"RYDER, MYRTA T "Lil Myrt"', 'POLICE OFFICER', 'POLICE', '$83706.00'],
    ['KORSHAK, ROMAN', 'PARAMEDIC', 'FIRE', '$75372.00']
]

with open('../TEMP/chi_data.csv', 'w') as chi_out:
    # On Windows, output line terminator must be set to '\n'.
    # While it's not needed on Linux/Mac, it doesn't cause any problems,
    # so this keeps the code portable.
    wtr = csv.writer(chi_out, lineterminator='\n') # create CSV writer from file object
    that is opened
    for data_row in chicago_data: # iterate over records from file
        data_row[0] = data_row[0].title() # make first field title case rather than all
        uppercase
        data_row[-1] = data_row[-1].lstrip('$') # strip leading $ from last field
        wtr.writerow(data_row) # write one row (of iterables) to output file
```

Example

csv_writerows.py

```
import sys
import csv

chicago_data = [
    ['Name', 'Position Title', 'Department', 'Employee Annual Salary'],
    ['BONADUCE, MICHAEL J', 'POLICE OFFICER', 'POLICE', '$80724.00'],
    ['MELLON, MATTHEW J "Matt"', 'POLICE OFFICER', 'POLICE', '$75372.00'],
    ['FIERI, JOHN J', 'FIREFIGHTER-EMT', 'FIRE', '$75342.00'],
    ['GALAHAD, MERLE S', 'CLERK III', 'BUSINESS AFFAIRS', '$45828.00'],
    ['ORCATTI, JENNIFER L', 'FIRE COMMUNICATIONS OPERATOR I', 'OEMC', '$63121.68'],
    ['ASHE, JOHN W', 'FOREMAN OF MACHINISTS', 'AVIATION', '$96553.60'],
    ['SADINSKY BLAKE, MICHAEL G', 'POLICE OFFICER', 'POLICE', '$78012.00'],
    ['GRANT, CRAIG A', 'SANITATION LABORER', 'STREETS & SAN', '$69576.00'],
    ['MILLER, JONATHAN D', 'POLICE OFFICER', 'POLICE', '$75372.00'],
    ['FRANK, ARTHUR R', 'POLICE OFFICER/EXPLSV DETECT, K9 HNDLR', 'POLICE', '$87918.00'],
    ['POVOTTI, JAMES S "Jimmy P"', 'TRAFFIC CONTROL AIDE-HOURLY', 'OEMC', '$19167.20'],
    ['TRAWLER, DANIEL J', 'POLICE OFFICER', 'POLICE', '$75372.00'],
    ['SCUBA, ANDREW G', 'POLICE OFFICER', 'POLICE', '$75372.00'],
    ['SWINE, MATTHEW W', 'SERGEANT', 'POLICE', '$99756.00'],
    ['''RYDER, MYRTA T "Lil'Myrt"', 'POLICE OFFICER', 'POLICE', '$83706.00'],
    ['KORSHAK, ROMAN', 'PARAMEDIC', 'FIRE', '$75372.00']
]

with open('../TEMP/chi_data2.csv', 'w') as chi_out:
    wtr = csv.writer(chi_out, lineterminator='\n')
    wtr.writerows(chicago_data) # write all rows to output file
```

About JSON

- Lightweight, human-friendly format for data
- Contains dictionaries and lists
- Stands for JavaScript Object Notation
- Looks like Python
- Basic types: Number, String, Boolean, Array, Object
- White space is ignored
- Stricter rules than Python

JSON is a lightweight and human-friendly format for sharing or storing data. It was developed and popularized by Douglas Crockford starting in 2001 .

A JSON file contains objects and arrays, which correspond exactly to Python dictionaries and lists.

White space is ignored, so JSON may be formatted for readability.

Data types are Number, String, and Boolean. Strings are enclosed in double quotes (only); numbers look like integers or floats; Booleans are represented by true or false; null (None in Python) is represented by null.

Reading JSON

- `json` module in standard library
- Parsing
 - `json.load()` from file object or file-like object
 - `json.loads()` parse from string
- Both methods return Python `dict` or `list`

To read a JSON file, import the `json` module. Use `json.loads()` to parse a string containing valid JSON. Use `json.load()` to read JSON from a file object (or a file-like object).

Both methods return a Python data structure (`dict` or `list`) containing all the data from the JSON file. The type of data structure returned depends on the outermost JSON data type in the file.



It is possible for a JSON document to be a single variable, but this is very uncommon. If this occurs, a single value will be returned, rather than a `dict` or `list`.

Example

json_read.py

```
from pprint import pprint
import json

# json.loads(String)      load from string
# json.load(FILE_OBJECT) load from file-like object

with open('../DATA/solar.json') as solar_in: # open JSON file for reading
    solar = json.load(solar_in) # load from file object and convert to Python data
    structure

# uncomment to see raw Python data
# print('-' * 60)
# pprint(solar)
# print('-' * 60)
# print('\n\n')

print(solar['innerplanets']) # solar is just a Python dictionary
print('*' * 60)
print(solar['innerplanets'][0]['name'])
print('*' * 60)
for planet in solar['innerplanets'] + solar['outerplanets']:
    print(planet['name'])

print('*' * 60)
for group in solar:
    if group.endswith('planets'):
        for planet in solar[group]:
            print(planet['name'])
```

json_read.py

```
[{'name': 'Mercury', 'moons': None}, {'name': 'Venus', 'moons': None}, {'name': 'Earth',  
'moons': ['Moon']}, {'name': 'Mars', 'moons': ['Deimos', 'Phobos']}]
```

```
*****
```

```
Mercury
```

```
*****
```

```
Mercury
```

```
Venus
```

```
Earth
```

```
Mars
```

```
Jupiter
```

```
Saturn
```

```
Uranus
```

```
Neptune
```

```
*****
```

```
Mercury
```

```
Venus
```

```
Earth
```

```
Mars
```

```
Jupiter
```

```
Saturn
```

```
Uranus
```

```
Neptune
```

```
Pluto
```

Writing JSON

- Use `json.dumps()` or `json.dump()`

To create a JSON file from a Python data structure, open a text file for writing to get a file object. Then pass the data structure and the file object to `json.dump()`.

To create a JSON string, pass the data structure to `json.dumps()`.

Example

`json_write.py`

```
import json

george = [
    {
        'num': 1,
        'lname': 'Washington',
        'fname': 'George',
        'dstart': [1789, 4, 30],
        'dend': [1797, 3, 4],
        'birthplace': 'Westmoreland County',
        'birthstate': 'Virginia',
        'dbirth': [1732, 2, 22],
        'ddeath': [1799, 12, 14],
        'assassinated': False,
        'party': None,
    },
    {
        'spam': 'ham',
        'eggs': [1.2, 2.3, 3.4],
        'toast': {'a': 5, 'm': 9, 'c': 4},
    }
] # Python data structure

js = json.dumps(george, indent=4) # dump structure to JSON string
print(js)

with open('george.json', 'w') as george_out: # open file for writing
    json.dump(george, george_out, indent=4) # dump structure to JSON file using open
    file object
```


json_write.py

```
[
    {
        "num": 1,
        "lname": "Washington",
        "fname": "George",
        "dstart": [
            1789,
            4,
            30
        ],
        "dend": [
            1797,
            3,
            4
        ],
        "birthplace": "Westmoreland County",
        "birthstate": "Virginia",
        "dbirth": [
            1732,
            2,
            22
        ],
        "ddeath": [
            1799,
            12,
            14
        ],
        "assassinated": false,
        "party": null
    },
    {
        "spam": "ham",
        "eggs": [
            1.2,
            2.3,
            3.4
        ],
        "toast": {
            "a": 5,
            "m": 9,
            "c": 4
        }
    }
]
```

Customizing JSON

- JSON data types limited
- simple cases — dump dict
- create custom encoders

The JSON spec only supports a limited number of datatypes. If you try to dump a data structure contains dates, user-defined classes, or many other types, the json encoder will not be able to handle it.

You can a custom encoder for various data types. To do this, write a function that expects one Python object, and returns some object that JSON can parse, such as a string or dictionary. The function can be called anything. Specify the function with the `default` parameter ("default" is the actual name of the parameter) to `json.dump()` or `json.dumps()`.

The function should check the type of the object. If it is a type that needs special handling, return a JSON-friendly version, otherwise just return the original object.

Table 4. Python types that JSON can encode

Python	JSON
<code>dict</code>	object
<code>list</code> <code>tuple</code>	array
<code>str</code>	string
<code>int</code> <code>float</code>	number
<code>True</code>	true
<code>False</code>	false
<code>None</code>	null



see the file `json_custom singledispatch.py` in EXAMPLES for how to use the `singledispatch` decorator (in the `functools` module) for another way to handle other data types.

Example

json_custom_encoding.py

```
import json
from datetime import date

class Parrot(): # sample user-defined class (not JSON-serializable)
    def __init__(self, name, color):
        self._name = name
        self._color = color

    @property
    def name(self): # JSON does not understand arbitrary properties
        return self._name

    @property
    def color(self):
        return self._color

parrots = [ # list of Parrot objects
    Parrot('Polly', 'green'), #
    Parrot('Peggy', 'blue'),
    Parrot('Roger', 'red'),
]

data = { # dictionary of arbitrary data
    'spam': [1, 2, 3],
    'ham': ('a', 'b', 'c'),
    'toast': date(2014, 8, 1),
    'parrots': parrots,
}

# try to encode the Python data
try:
    print(json.dumps(data, indent=4))
except TypeError as err:
    print("ENCODING FAILED: ", err)
else:
    print("ENCODING SUCCESSFUL")
print('-' * 60)

def custom_encoder(obj): # custom JSON encoder function
    if isinstance(obj, date): # check for date object
        return obj.ctime() # convert date to string
    elif isinstance(obj, Parrot): # check for Parrot object
        return {'name': obj.name, 'color': obj.color} # convert Parrot to dictionary
```

```

        return obj # if not processed, return object for JSON to parse with default parser

# 'default' parameter specifies function for custom encoding;
try:
    print(json.dumps(data, default=custom_encoder, indent=4))
except TypeError as err:
    print("ENCODING FAILED: ", err)
else:
    print("ENCODING SUCCESSFUL")

```

json_custom_encoding.py

```

ENCODING FAILED: Object of type date is not JSON serializable
-----
{
    "spam": [
        1,
        2,
        3
    ],
    "ham": [
        "a",
        "b",
        "c"
    ],
    "toast": "Fri Aug  1 00:00:00 2014",
    "parrots": [
        {
            "name": "Polly",
            "color": "green"
        },
        {
            "name": "Peggy",
            "color": "blue"
        },
        {
            "name": "Roger",
            "color": "red"
        }
    ]
}
ENCODING SUCCESSFUL

```

Reading and writing YAML

- yaml module from PYPI
- syntax like **json** module
- `yaml.load()`, `dump()` parse from/to file-like object
- `yaml.loads()`, `dumps()` parse from/to string

YAML is a structured data format which is a superset of JSON. However, YAML allows for a more compact and readable format.

Reading and writing YAML uses the same syntax as JSON, other than using the **yaml** module, which is NOT in the standard library. To install the **yaml** module:

```
pip install pyyaml
```

To read a YAML file (or string) into a Python data structure, use `yaml.load(file_object)` or `yaml.loads(string)`.

To write a data structure to a YAML file or string, use `yaml.dump(data, file_object)` or `yaml.dumps(data)`.

You can also write custom YAML processors.



YAML parsers will parse JSON data

Example

yaml_load_solar.py

```
import yaml

PLANET_SECTIONS = "inner outer plutoid".split()

with open('../DATA/solar.yaml') as solar_in:
    solar_data = yaml.load(solar_in, Loader=yaml.Loader)

star = solar_data['star']
print(f"Our star is {star}\n")

for section in PLANET_SECTIONS:
    for planet in solar_data[section]:
        print(planet['name'])
        for moon in planet['moons']:
            print(f"    {moon}")
```

yaml_load_solar.py

Our star is Sun

Mercury

None

Venus

None

Earth

Moon

Mars

Deimos

Phobos

Metis

Jupiter

Adrastea

Amalthea

Thebe

Io

Europa

Ganymede

Callisto

Themisto

Himalia

Lysithea

Elara

Saturn

Rhea

Hyperion

Titan

Iapetus

Mimas

...

Example

yaml_dump.py

```
import sys
from datetime import date
import yaml

potus = {
    'presidents': [
        {
            'lastname': 'Washington',
            'firstname': 'George',
            'dob': date(1732, 2, 22),
            'dod': date(1799, 12, 14),
            'birthplace': 'Westmoreland County',
            'birthstate': 'Virginia',
            'term': [ date(1789, 4, 30), date(1797, 3, 4) ],
            'assassinated': False,
            'party': None,
        },
        {
            'lastname': 'Adams',
            'firstname': 'John',
            'dob': date(1735, 10, 30),
            'dod': date(1826, 7, 4),
            'birthplace': 'Braintree, Norfolk',
            'birthstate': 'Massachusetts',
            'term': [date(1797, 3, 4), date(1801, 3, 4)],
            'assassinated': False,
            'party': 'Federalist',
        }
    ]
}

with open('potus.yaml', 'w') as potus_out:
    yaml.dump(potus, potus_out)

yaml.dump(potus, sys.stdout)
```


yaml_dump.py

```
presidents:
- assassinated: false
  birthplace: Westmoreland County
  birthstate: Virginia
  dob: 1732-02-22
  dod: 1799-12-14
  firstname: George
  lastname: Washington
  party: null
  term:
  - 1789-04-30
  - 1797-03-04
- assassinated: false
  birthplace: Braintree, Norfolk
  birthstate: Massachusetts
  dob: 1735-10-30
  dod: 1826-07-04
  firstname: John
  lastname: Adams
  party: Federalist
  term:
  - 1797-03-04
  - 1801-03-04
```

Which XML module to use?

- Bewildering array of XML modules
- Some are SAX, some are DOM
- Use `xml.etree.ElementTree`

When you are ready to process Python with XML, you turn to the standard library, only to find a number of different modules with confusing names.

To cut to the chase, use **lxml.etree**, which is based on **ElementTree** with some nice extra features, such as pretty-printing. While not part of the core Python library, it is provided by the Anaconda bundle.

If **lxml.etree** is not available, you can use **xml.etree.ElementTree** from the core library.

Getting Started With ElementTree

- Import `xml.etree.ElementTree` (or `lxml.etree`) as `ET` for convenience
- Parse XML or create empty `ElementTree`

`ElementTree` is part of the Python standard library; `lxml` is included with the Anaconda distribution.

Since putting `"xml.etree.ElementTree"` in front of its methods requires a lot of extra typing, it is typical to alias `xml.etree.ElementTree` to just `ET` when importing it: `import xml.etree.ElementTree as ET`

How ElementTree Works

- ElementTree contains root Element
- Document is tree of Elements

In ElementTree, an XML document consists of a nested tree of Element objects. Each Element corresponds to an XML tag.

An ElementTree object serves as a wrapper for reading or writing the XML text.

If you are parsing existing XML, use `ElementTree.parse()`; this creates the ElementTree wrapper and the tree of Elements. You can then navigate to, or search for, Elements within the tree. You can also insert and delete new elements.

If you are creating a new document from scratch, create a top-level (AKA "root") element, then create child elements as needed.

```
element = root.find('sometag')
for subelement in element:
    print(subelement.tag)
print(element.get('someattribute'))
```

Elements

- Element has
 - Tag name
 - Attributes (implemented as a dictionary)
 - Text
 - Tail
 - Child elements (implemented as a list) (if any)
- SubElement creates child of Element

When creating a new Element, you can initialize it with the tag name and any attributes. Once created, you can add the text that will be contained within the element's tags, or add other attributes.

When you are ready to save the XML into a file, initialize an ElementTree with the root element.

The **Element** class is a hybrid of list and dictionary. You access child elements by treating it as a list. You access attributes by treating it as a dictionary. (But you can't use subscripts for the attributes – you must use the `get()` method).

The Element object also has several useful properties: **tag** is the element's tag; **text** is the text contained inside the element; **tail** is any text following the element, before the next element.

The **SubElement** class is a convenient way to add children to an existing Element.



Only the tag property of an Element is required; other properties are optional.

Table 5. Element methods and properties

Method/Property	Description
<code>append(element)</code>	Add a subelement element to end of subelements
<code>attrib</code>	Dictionary of element's attributes
<code>clear()</code>	Remove all subelements
<code>find(path)</code>	Find first subelement matching path
<code>findall(path)</code>	Find all subelements matching path
<code>findtext(path)</code>	Shortcut for <code>find(path).text</code>
<code>get(attr)</code>	Get an attribute; Shortcut for <code>attrib.get()</code>
<code>getiterator()</code>	Returns an iterator over all descendants
<code>getiterator(path)</code>	Returns an iterator over all descendants matching path
<code>insert(pos,element)</code>	Insert subelement element at position pos
<code>items()</code>	Get all attribute values; Shortcut for <code>attrib.items()</code>
<code>keys()</code>	Get all attribute names; Shortcut for <code>attrib.keys()</code>
<code>remove(element)</code>	Remove subelement element
<code>set(attrib,value)</code>	Set an attribute value; shortcut for <code>attr[attrib] = value</code>
<code>tag</code>	The element's tag
<code>tail</code>	Text following the element
<code>text</code>	Text contained within the element

Table 6. ElementTree methods and properties

Property	Description
<code>find(path)</code>	Finds the first toplevel element with given tag; shortcut for <code>getroot().find(path)</code> .
<code>findall(path)</code>	Finds all toplevel elements with the given tag; shortcut for <code>getroot().findall(path)</code> .
<code>findtext(path)</code>	Finds element text for first toplevel element with given tag; shortcut for <code>getroot().findtext(path)</code> .
<code>getiterator(path)</code>	Returns an iterator over all descendants of root node matching path. (All nodes if path not specified)
<code>getroot()</code>	Return the root node of the document
<code>parse(filename)</code> <code>parse(fileobj)</code>	Parse an XML source (filename or file-like object)
<code>write(filename,encoding)</code>	Writes XML document to filename, using encoding (Default us-ascii).

Parsing An XML Document

- Use `ElementTree.parse()`
- returns an `ElementTree` object
- Use `get*` or `find*` methods to select an element

Use the `parse()` method to parse an existing XML document. It returns an `ElementTree` object, from which you can find the root, or any other element within the document.

To get the root element, use the `getroot()` method.

Example

```
import xml.etree.ElementTree as ET

doc = ET.parse('solar.xml')

root = doc.getroot()
```


Navigating the XML Document

- Use `find()` or `findall()`
- Element is iterable of its children
- `findtext()` retrieves text from element

To find the first child element with a given tag, use `find('tag')`. This will return the first matching element. The `findtext('tag')` method is the same, but returns the text within the tag.

To get all child elements with a given tag, use the `findall('tag')` method, which returns a list of elements.

to see whether a node was found, say

```
if node is None:
```

but to check for existence of child elements, say

```
if len(node) > 0:
```

A node with no children tests as false because it is an empty list, but it is not None.



The `ElementTree` object also supports the `find()` and `findall()` methods of the `Element` object, searching from the root object.

Example

xml_planets_nav.py

```
'''Use etree navigation to extract planets from solar.xml'''
import lxml.etree as ET

def main():
    '''Program entry point'''
    doc = ET.parse('../DATA/solar.xml')

    solar_system = doc.getroot()

    print(solar_system)
    print()

    inner = solar_system.find('innerplanets')
    print('Inner:')

    for planet in inner:
        if planet.tag == 'planet':
            print('\t', planet.get("planetname", "NO NAME"))

    outer = solar_system.find('outerplanets')
    print('Outer:')

    for planet in outer:
        print('\t', planet.get("planetname"))

    plutoids = solar_system.find('dwarfplanets')
    print('Dwarf:')

    for planet in plutoids:
        print('\t', planet.get("planetname"))

if __name__ == '__main__':
    main()
```

xml_planets_nav.py

```
<Element solarsystem at 0x10be30200>
```

```
Inner:
```

```
    Mercury
```

```
    Venus
```

```
    Earth
```

```
    Mars
```

```
Outer:
```

```
    Jupiter
```

```
    Saturn
```

```
    Uranus
```

```
    Neptune
```

```
Dwarf:
```

```
    Pluto
```

Example

xml_read_movies.py

```
# import xml.etree.ElementTree as ET
import lxml.etree as ET

movies_doc = ET.parse('../DATA/movies.xml') # read and parse the XML file

movies = movies_doc.getroot() # get the root element (<movies>)

for movie in movies: # loop through children of root element
    movie_name = movie.findtext('title'), # get 'name' attribute of movie element
    movie_director = movie.findtext('director'), # get 'director' attribute of movie
    element
    print(f'{movie_name} by {movie_director}')
```

xml_read_movies.py

```
('Casablanca',) by ('Michael Curtiz',)
('The Princess Bride',) by ('Rob Reiner',)
('Jaws',) by ('Stephen Spielberg',)
('Topaz',) by ('Alfred Hitchcock',)
```

Using XPath

- Use simple XPath patterns Works with find* methods

When a simple tag is specified, the find* methods only search for subelements of the current element. For more flexible searching, the find* methods work with simplified **XPath** patterns. To find all tags named 'spam', for instance, use `./spam`.

```
./movie  
presidents/president/name/last
```

Example

xml_planets_xpath1.py

```
# import xml.etree.ElementTree as ET  
import lxml.etree as ET  
  
doc = ET.parse('../DATA/solar.xml') # parse XML file  
  
inner_nodes = doc.findall('innerplanets/planet') # find all elements (relative to root  
element) with tag "planet" under "innerplanets" element  
  
outer_nodes = doc.findall('outerplanets/planet') # find all elements with tag "planet"  
under "outerplanets" element  
  
print('Inner:')  
for planet in inner_nodes: # loop through search results  
    print('\t', planet.get("planetname")) # print "name" attribute of planet element  
  
print('Outer:')  
for planet in outer_nodes: # loop through search results  
    print('\t', planet.get("planetname")) # print "name" attribute of planet element
```

xml_planets_xpath1.py

```
Inner:
    Mercury
    Venus
    Earth
    Mars
Outer:
    Jupiter
    Saturn
    Uranus
    Neptune
```

Example**xml_planets_xpath2.py**

```
# import xml.etree.ElementTree as ET
import lxml.etree as ET

doc = ET.parse('../DATA/solar.xml')

jupiter = doc.find('.//planet[@planetname="Jupiter"]')

if jupiter is not None:
    for moon in jupiter:
        print(moon.text) # grab attribute
```

xml_planets_xpath2.py

```
Metis
Adrastea
Amalthea
Thebe
Io
Europa
Ganymede
Callisto
Themisto
Himalia
Lysithea
Elara
```

Table 7. ElementTree XPath Summary

Syntax	Meaning
<code>tag</code>	Selects all child elements with the given tag. For example, “spam” selects all child elements named “spam”, “spam/egg” selects all grandchildren named “egg” in all child elements named “spam”. You can use universal names (“{url}local”) as tags.
<code>*</code>	Selects all child elements. For example, “*/egg” selects all grandchildren named “egg”.
<code>.</code>	Select the current node. This is mostly useful at the beginning of a path, to indicate that it’s a relative path.
<code>//</code>	Selects all subelements, on all levels beneath the current element (search the entire subtree). For example, “//egg” selects all “egg” elements in the entire tree.
<code>..</code>	Selects the parent element.
<code>[@attrib]</code>	Selects all elements that have the given attribute. For example, “//a[@href]” selects all “a” elements in the tree that has a “href” attribute.
<code>[@attrib=‘value’]</code>	Selects all elements for which the given attribute has the given value. For example, “//div[@class=‘sidebar’]” selects all “div” elements in the tree that has the class “sidebar”. In the current release, the value cannot contain quotes.
<code>parent_tag[child_tag]</code>	Selects all parent elements that has a child element named <i>child_tag</i> . In the current version, only a single tag can be used (i.e. only immediate children are supported). Parent tag can be <code>*</code> .
<code>[position]</code>	Selects all elements that are located at the given position. The position can be either an integer (1 is the first position), the expression “last()” (for the last position), or a position relative to last() (e.g. “last()-1” for the second to last position). This predicate must be preceded by a tag name.

Creating a New XML Document

- Create root element
- Add descendants via SubElement
- Use keyword arguments for attributes
- Add text after element created
- Create ElementTree for import/export

To create a new XML document, first create the root (top-level) element. This will be a container for all other elements in the tree. If your XML document contains books, for instance, the root document might use the "books" tag. It would contain one or more "book" elements, each of which might contain author, title, and ISBN elements.

Once the root element is created, use SubElement to add elements to the root element, and then nested Elements as needed. SubElement returns the new element, so you can assign the contents of the tag to the **text** attribute.

Once all the elements are in place, you can create an ElementTree object to contain the elements and allow you to write out the XML text. From the ElementTree object, call write.

To output an XML string from your elements, call ET.tostring(), passing the root of the element tree as a parameter. It will return a bytes object (pure ASCII), so use .decode() to convert it to a normal Python string.

For an example of creating an XML document from a data file, see **xml_create_knights.py** in the EXAMPLES folder

Example

xml_create_movies.py

```
# from xml.etree import ElementTree as ET
import lxml.etree as ET

movie_data = [
    ('Jaws', 'Spielberg, Stephen'),
    ('Vertigo', 'Alfred Hitchcock'),
    ('Blazing Saddles', 'Brooks, Mel'),
    ('Princess Bride', 'Reiner, Rob'),
    ('Avatar', 'Cameron, James'),
]

movies = ET.Element('movies')

for title, director in movie_data:
    movie = ET.SubElement(movies, 'movie')
    ET.SubElement(movie, 'title').text = title
    ET.SubElement(movie, 'director').text = director

print(ET.tostring(movies, pretty_print=True).decode())

doc = ET.ElementTree(movies)

doc.write('movies.xml')
```

xml_create_movies.py

```
<movies>
  <movie>
    <title>Jaws</title>
    <director>Spielberg, Stephen</director>
  </movie>
  <movie>
    <title>Vertigo</title>
    <director>Alfred Hitchcock</director>
  </movie>
  <movie>
    <title>Blazing Saddles</title>
    <director>Brooks, Mel</director>
  </movie>
  <movie>
    <title>Princess Bride</title>
    <director>Reiner, Rob</director>
  </movie>
  <movie>
    <title>Avatar</title>
    <director>Cameron, James</director>
  </movie>
</movies>
```

Pickle

- Use the pickle module
- Create a binary stream that can be saved to file
- Can also be transmitted over the network

Python uses the pickle module for data serialization. This format is unique to Python, and a binary format.

The extension for pickle files is ".pkl", or sometimes ".pickle".

To create pickled data, use either `pickle.dump()` or `pickle.dumps()`. Both functions take a data structure as the first argument. `dumps()` returns the pickled data as a string. `dump()` writes the data to a file-like object which has been specified as the second argument. The file-like object must be opened for writing.

To read pickled data, use `pickle.load()`, which takes a file-like object that has been open for writing, or `pickle.loads()` which reads from a string. Both functions return the original data structure that had been pickled.



The syntax of the **json** module is based on the **pickle** module.

Example

pickling.py

```
import pickle
from pprint import pprint

# some data structures
airports = {
    'RDU': 'Raleigh-Durham', 'IAD': 'Dulles', 'MGW': 'Morgantown',
    'EWR': 'Newark', 'LAX': 'Los Angeles', 'ORD': 'Chicago'
}

colors = [
    'red', 'blue', 'green', 'yellow', 'black',
    'white', 'orange', 'brown', 'purple'
]

values = [
    3/7, 1/9, 14.5
]

data = [ # list of data structures
    colors,
    airports,
    values,
]

print("BEFORE:")
pprint(data)
print('-' * 60)

with open('../TEMP/pickled_data.pkl', 'wb') as pkl_out: # open pickle file for writing
    in binary mode
    pickle.dump(data, pkl_out) # serialize data structures to pickle file

with open('../TEMP/pickled_data.pkl', 'rb') as pkl_in: # open pickle file for reading in
    in binary mode
    pickled_data = pickle.load(pkl_in) # de-serialize pickle file back into data
    structures

print("AFTER:")
pprint(pickled_data) # view data structures
```

pickling.py

```
BEFORE:
[['red',
  'blue',
  'green',
  'yellow',
  'black',
  'white',
  'orange',
  'brown',
  'purple'],
{'EWR': 'Newark',
 'IAD': 'Dulles',
 'LAX': 'Los Angeles',
 'MGW': 'Morgantown',
 'ORD': 'Chicago',
 'RDU': 'Raleigh-Durham'}],
[0.42857142857142855, 0.1111111111111111, 14.5]]
```

```
-----
AFTER:
[['red',
  'blue',
  'green',
  'yellow',
  'black',
  'white',
  'orange',
  'brown',
  'purple'],
{'EWR': 'Newark',
 'IAD': 'Dulles',
 'LAX': 'Los Angeles',
 'MGW': 'Morgantown',
 'ORD': 'Chicago',
 'RDU': 'Raleigh-Durham'}],
[0.42857142857142855, 0.1111111111111111, 14.5]]
```

Chapter 3 Exercises

Exercise 3-1 (cpresidents.py)

Parse `presidents.csv`. Loop through and print out each president's first and last names and their political party.

Exercise 3-2 (jpresidents.py)

Parse `presidents.json`. Loop through and print out each president's first and last names and their political party.

Exercise 3-3 (ypresidents.py)

Parse `presidents.yaml`. Loop through and print out each president's first and last names and their political party.

Exercise 3-4 (xwords.py)

Using ElementTree, create a new XML file containing all the words that start with 'x' from `words.txt`. The root tag should be named 'words', and each word should be contained in a 'word' tag. The finished file should look like this:

```
<words>
  <word>xanthan</word>
  <word>xanthans</word>
  ...
</words>
```

Exercise 3-5 (xpresidents.py)

Use ElementTree to parse `presidents.xml`. Loop through and print out each president's first and last names and their state of birth.

Exercise 3-6 (pickle_potus.py)

Write a script which reads the data from `presidents.json` into memory. It will be a list of dictionaries.

Using the `pickle` module, Write the entire dictionary out to a file named `presidents.pkl`.

Exercise 3-7 (unpickle_potus.py)

Write a script to open `presidents.pkl`, and restore the data back into a list.

Then loop through the list and print out each president's first name, last name, and party.

Chapter 4: Metaprogramming

Objectives

- Learn what metaprogramming means
- Access local and global variables by name
- Inspect the details of any object
- Use attribute functions to manipulate an object
- Design decorators for classes and functions
- Define classes with the `type()` function

Metaprogramming

- Writing code that modifies existing code or writes new code
- Can simplify some kinds of programs
- Not as hard as you think! (probably)
- Considered deep magic in other languages

Metaprogramming is writing code that generates or modifies other code. It includes fetching, changing, or deleting attributes, and writing functions that return functions (AKA factories).

Metaprogramming is easier in Python than many other languages. Python provides explicit access to objects, even the parts that are hidden or restricted in other languages.

For instance, you can easily replace one method with another in a Python class, or even in an object instance. In other languages this would be difficult or impossible.

globals() and locals()

- Contain all variables in a namespace
- `globals()` returns all global objects
- `locals()` returns all local variables

The builtin function `globals()` returns a dictionary of all global objects. The keys are the object names, and the values are the objects' values. The dictionary is "live" — changes to the dictionary affect global variables.



The `locals()` builtin returns a dictionary of all objects in local scope. If not called within a function, it just calls `globals()`.

Example

globals_locals.py

```
from pprint import pprint # import prettyprint function

# global variables
count = 42
animal = 'Wombat'

def spam(fruit): # function parameters are local
    knight = 'Lancelot' # local variable

spam('mango')

# globals() returns dict of all globals
pprint(globals())
print()

g = globals()
g['color'] = "blue" # create a new variable
print("color:", color)
```

globals_locals.py

```
{'__annotations__': {},
 '__builtins__': <module 'builtins' (built-in)>,
 '__cached__': None,
 '__doc__': None,
 '__file__': '/Users/jstrick/curr/courses/python/common/examples/globals_locals.py',
 '__loader__': <_frozen_importlib_external.SourceFileLoader object at 0x10ec212d0>,
 '__name__': '__main__',
 '__package__': None,
```

...

```
'__package__': None,
 '__spec__': None,
 'animal': 'Wombat',
 'count': 42,
 'pprint': <function pprint at 0x1094042c0>,
 'spam': <function spam at 0x10928c860>}
```

color: blue

Example

../DATA/sample.cfg

```
animal=wombat  
color=ecru  
detective=Wolfe  
language=Python
```

config_to_variables.py

```
CONFIG_FILE_PATH = "../DATA/sample.cfg"  
  
g = globals() # get access to dictionary of global names  
  
with open(CONFIG_FILE_PATH) as config_in:  
    for raw_line in config_in:  
        line = raw_line.rstrip()  
        name, value = line.split('=')  
        g[name] = value # create a new global variable  
  
print(animal, color, detective, language)
```

config_to_variables.py

```
wombat ecru Wolfe Python
```

Using `dir()`

- With argument
 - Shows attributes of object
- Without argument
 - Shows names in current scope
- Syntax

```
dir(__obj__) # attributes of __obj__  
dir()       # names in current scope
```

The builtin `dir()` function returns a list of all attributes of an object. The list contains the *names* of the attribute, as strings. This includes all "private" names that start with one or two underscores.

This can be useful to find out all the methods and properties available on an object, for instance, if the documentation is not available.

With no argument, `dir()` returns all names in the current scope.



Use a list comprehension to filter out "private" names.

```
+ names = [name for name in dir(OBJ) if not name.startswith('_')]
```



(advanced) An object can decide what attributes to show by implementing the `__dir__()` special method.

Example

using_dir.py

```
class Dog:
    def bark(self):
        print("woof! woof!")

    def wag(self):
        print("wag wag wag wag wag")

d = Dog()
print(f"{dir(d) = }\n")

colors = ["red", "purple", "black"]
attributes = [name for name in dir(colors) if not name.startswith('_')]
print(f"PUBLIC attributes: {attributes}\n")
```

using_dir.py

```
dir(d) = ['__class__', '__delattr__', '__dict__', '__dir__', '__doc__', '__eq__',
'__format__', '__ge__', '__getattr__', '__getstate__', '__gt__', '__hash__',
'__init__', '__init_subclass__', '__le__', '__lt__', '__module__', '__ne__', '__new__',
'__reduce__', '__reduce_ex__', '__repr__', '__setattr__', '__sizeof__', '__str__',
'__subclasshook__', '__weakref__', 'bark', 'wag']

PUBLIC attributes: ['append', 'clear', 'copy', 'count', 'extend', 'index', 'insert',
'pop', 'remove', 'reverse', 'sort']
```

Working with attributes

- Objects are dictionaries of attributes
- Special functions can be used to access attributes
- Attributes specified as strings
- Syntax

```
getattr(object, attribute [,defaultvalue] )  
hasattr(object, attribute)  
setattr(object, attribute, value)  
delattr(object, attribute)
```

All Python objects are essentially dictionaries of attributes. There are four special builtin functions for managing attributes. These may be used to programmatically access attributes specifying the attribute name as a string.

`getattr()` returns the value of a specified attribute, or raises an error if the object does not have that attribute.

`getattr(a, 'spam')` is effectively the same as `a.spam`.

An optional third argument to `getattr()` provides a default value for nonexistent attributes (and does not raise an error).

`hasattr()` returns the value of a specified attribute, or `None` if the object does not have that attribute.

`setattr()` an attribute to a specified value.

`delattr()` deletes an attribute and its corresponding value.

Example

attributes.py

```
class Spam():

    def eggs(self, msg): # create attribute
        print("eggs!", msg)

s = Spam()

s.eggs("fried")

print("hasattr()", hasattr(s, 'eggs')) # check whether attribute exists

e = getattr(s, 'eggs') # retrieve attribute
e("scrambled")

def toast(self, msg):
    print("toast!", msg)

setattr(Spam, 'eggs', toast) # set (or overwrite) attribute

s.eggs("battered!")

delattr(Spam, 'eggs') # remove attribute

try:
    s.eggs("shirred")
except AttributeError as err: # missing attribute raises error
    print(err)
```

attributes.py

```
eggs! fried
hasattr() True
eggs! scrambled
toast! battered!
'Spam' object has no attribute 'eggs'
```


Adding instance methods

- Use `setattr()`
- Add instance method to class
- Add instance method to instance

Using `setattr()`, it is easy to add instance methods to classes. Just add a function object to the class. Because it is part of the class itself, it will automatically be bound to the instance. Remember that an instance method expects `self` as the first parameter. In fact, this is the meaning of a bound instance — it is "bound" to the instance, and therefore when called, it is passed the instance as the first parameter.

Once added, the method may be called from any existing *or new* instance of the class.

To add an instance method to an *instance* takes a little more effort. Because it's not being added to the class, it is not automatically bound. The function needs to know what instance it should be bound to. This can be accomplished with the `types.MethodType()` function.

Pass the function and the instance to `MethodType()`.

Example

adding_instance_methods.py

```
from types import MethodType

class Dog(): # Define Dog type
    pass

d1 = Dog() # Create instance of Dog

def bark(self): # Define (unbound) function
    print("Woof! woof!")

setattr(Dog, "bark", bark) # Add function to class (which binds it as an instance
method)

d2 = Dog() # Define another instance of Dog

d1.bark() # New function can be called from either instance
d2.bark()

def wag(self): # Create another unbound function
    print("Wagging...")

setattr(d1, "wag", MethodType(wag, d1)) # Add function to instance after passing it
through MethodType()

d1.wag() # Call instance method
try:
    d2.wag() # Instance method not available - only bound to d1
except AttributeError as err:
    print(err)
```

adding_instance_methods.py

```
Woof! woof!
Woof! woof!
Wagging...
'Dog' object has no attribute 'wag'
```

The inspect module

- Simplifies access to metadata
- Easily get metadata

The `inspect` module provides user-friendly functions for accessing the metadata of objects.

This can be useful for troubleshooting, as well as run-time type checking.

Checking types

You can use `inspect.is<type>()` to confirm an object is of a specified type. This can be used for extra validation of function/method parameters.

Checking parameters

If you have a function that expects a callback function as a parameter, you can use `inspect.getfullargspec()` to make sure the callback has the correct signature (number of parameters).

Checking frames (call stack)

The `inspect.currentframe()` function will return the current frame object that contains the current code being executed. Passing that frame object to `inspect.getframeinfo()` will return details, similar to the stack trace that happens when a program ends unexpectedly.

Example

inspect_types.py

```
import inspect
import geometry
from carddeck import CardDeck

deck = CardDeck("Leonard")

things = (
    geometry,
    geometry.circle_area,
    CardDeck,
    CardDeck.get_ranks,
    deck,
    deck.shuffle,
)

def get_instance_type_name(obj):
    return f"{type(thing).__name__} instance"

print("Name           Module?  Function?  Class?  Method?")
for thing in things:
    thing_name = getattr(thing, '__name__', get_instance_type_name(thing))
    print("{:18s} {!s:6s}  {!s:6s}    {!s:6s}  {!s:6s}".format(
        thing_name,
        inspect.ismodule(thing), # test for module
        inspect.isfunction(thing), # test for function
        inspect.isclass(thing), # test for class
        inspect.ismethod(thing),
    ))
```

inspect_types.py

Name	Module?	Function?	Class?	Method?
geometry	True	False	False	False
circle_area	False	True	False	False
CardDeck	False	False	True	False
get_ranks	False	False	False	True
CardDeck instance	False	False	False	False
shuffle	False	False	False	True

Example

inspect_functions.py

```
import inspect

def spam(p1, p2='a', *p3, p4, p5='b', **p6): # define a function
    print(p1, p2, p3, p4, p5, p6)

# get argument specifications for a function
print("Function spec for Ham:", inspect.getfullargspec(spam))
print()
```

inspect_functions.py

```
Function spec for Ham: FullArgSpec(args=['p1', 'p2'], varargs='p3', varkw='p6',
defaults=('a',), kwonlyargs=['p4', 'p5'], kwonlydefaults={'p5': 'b'}, annotations={})
```

Example

inspect_frames.py

```
import inspect

def eggs():
    print("EGGS")
    # get frame (function call stack) info
    current_frame = inspect.currentframe()
    print("Current frame:", inspect.getframeinfo(current_frame))

eggs()
```

inspect_frames.py

```
EGGS
Current frame:
Traceback(filename='/Users/jstrick/curr/courses/python/common/examples/inspect_frames.py'
, lineno=7, function='eggs', code_context=['    print("Current frame:",
inspect.getframeinfo(current_frame))\n'], index=0, positions=Positions(lineno=7,
end_lineno=7, col_offset=28, end_col_offset=63))
```

Table 8. inspect module convenience functions

Function(s)	Description
<code>ismodule()</code> , <code>isclass()</code> , <code>ismethod()</code> , <code>isfunction()</code> , <code>isgeneratorfunction()</code> , <code>isgenerator()</code> , <code>istraceback()</code> , <code>isframe()</code> , <code>iscode()</code> , <code>isbuiltin()</code> , <code>isroutine()</code>	check object types
<code>getmembers()</code>	get members of an object that satisfy a given condition
<code>getfile()</code> , <code>getsourcefile()</code> , <code>getsource()</code>	find an object's source code
<code>getdoc()</code> , <code>getcomments()</code>	get documentation on an object
<code>getmodule()</code>	determine the module that an object came from
<code>get_annotations()</code>	return dictionary of object's annotations
<code>getclasstree()</code>	arrange classes so as to represent their hierarchy
<code>getargspec()</code> , <code>getfullargspec()</code> , <code>getargvalues()</code>	get info about function arguments, return values, and annotations
<code>formatargspec()</code> , <code>formatargvalues()</code>	format an argument spec
<code>getouterframes()</code> , <code>getinnerframes()</code>	get info about frames
<code>currentframe()</code>	get the current stack frame
<code>stack()</code> , <code>trace()</code>	get info about frames on the stack or in a traceback

Callable classes

- Convenient for one-method classes
- Really "callable instances"
- Implement `__call__`
- Convenient for one-method classes
- Useful for decorators

Any class instance may be made callable by implementing the special method `call`. This means that rather than saying:

```
sc = SomeClass()  
sc.some_method()
```

you can say

```
sc = SomeClass()  
sc()
```

You can think of a callable class as a function that can also keep some state. As with many object-oriented features, its main purpose is to simplify the user interface.

What's the advantage? In addition to saving state between calls, it saves having to call a method from the instance. Thus, a good use case is for classes that only have one simple method.

Another place to use callable classes is for implementing decorators as classes, rather than functions.

Example

randomword.py

```
import random

class RandomWord():
    def __init__(self):
        with open('../DATA/words.txt') as words_in:
            self._word_list = [w.rstrip() for w in words_in]

    def __call__(self):
        """
        Return one random word when the instance is called

        :return: A random word as a string.
        """
        return random.choice(self._word_list)

if __name__ == '__main__':
    w = RandomWord()
    for _ in range(10):
        print(w())
```

randomword.py

```
indomitability
materializations
bikeways
emblems
photoreduction
visualised
stroker
playtimes
footings
coveys
```


Example

memorychecker.py

```
import os
import psutil

class MemoryChecker():
    """
    Callable class to get current memory use of program.

    Instances of this class may be called, at which time
    they will return current memory use.
    """

    def __init__(self):
        self.process = psutil.Process(os.getpid()) # Get PID of current process

    def __call__(self):
        return self.process.memory_info().rss # Return memory use for PID

if __name__ == '__main__':
    mc = MemoryChecker()
    print(mc()) # can call at any time to get current memory use
    big_list = [0] * 10_000_000
    print(mc())
```

memorychecker.py

```
13447168
93462528
```

Decorators

- Classic design pattern
- Built into Python
- Implemented via functions or classes
- Can decorate functions or classes
- Can take arguments (but not required to)
- `functools.wraps()` preserves function's properties

In Python, many decorators are provided by the standard library, such as `property()` or `classmethod()`

A decorator is a component that modifies some other component. The purpose is typically to add functionality, but there are no real restrictions on what a decorator can do. Many decorators register a component with some other component. For instance, the `@app.route()` decorator in Flask maps a URL to a view function.

As another example, `unittest` provides decorators to skip tests. A very common decorator is `@property`, which converts a class method into a property object.

A decorator can be any *callable*, which means it can be a normal function, a class method, or a class which implements the `__call__()` method (AKA callable class, as discussed earlier).

A simple decorator expects the item being decorated as its parameter, and returns a replacement. Typically, the replacement is a new function, but there is no restriction on what is returned. If the decorator itself needs arguments, then the decorator returns a wrapper function that expects the item being decorated, and then returns the replacement.



In this chapter, "function decorator" means a decorator that modifies a function. "decorator function" means a decorator implemented as a function.

Table 9. Decorators in the standard library

Decorator	Description
<code>@abc.abstractmethod</code>	Indicate abstract method (must be implemented).
<code>@abc.abstractproperty</code>	Indicate abstract property (must be implemented). DEPRECATED
<code>@asyncio.coroutine</code>	Mark generator-based coroutine.
<code>@atexit.register</code>	Register function to be executed when interpreter (script) exits.
<code>@classmethod</code>	Indicate class method (receives class object, not instance object)
<code>@contextlib.contextmanager</code>	Define factory function for <code>with</code> statement context managers (no need to create <code>__enter__()</code> and <code>__exit__()</code> methods)
<code>@dataclass</code>	Mark a class as a dataclass from the <code>dataclass</code> module
<code>@functools.cache</code> <code>@functools.lru_cache</code>	Wrap a function with a memoizing callable
<code>@functools.singledispatch</code>	Transform function into a single-dispatch generic function.
<code>@basefunction.register(function)</code>	Register function for single dispatch
<code>@functools.total_ordering</code>	Supply all other comparison methods if class defines at least one.
<code>@functools.wraps</code>	Invoke <code>update_wrapper()</code> so decorator's replacement function keeps original function's name and other properties.
<code>@property</code>	Indicate a class property.
<code>@staticmethod</code>	Indicate static method (passed neither instance nor class object).
<code>@types.coroutine</code>	Transform generator function into a coroutine function.
<code>@unittest.mock.patch</code>	Patch target with a new object. When the function/with statement exits patch is undone.
<code>@unittest.mock.patch.dict</code>	Patch dictionary (or dictionary-like object), then restore to original state after test.
<code>@unittest.mock.patch.multiple</code>	Perform multiple patches in one call.
<code>@unittest.mock.patch.object</code>	Patch object attribute with mock object.
<code>@unittest.skip()</code>	Skip test unconditionally
<code>@unittest.skipIf()</code>	Skip test if condition is true
<code>@unittest.skipUnless()</code>	Skip test unless condition is true
<code>@unittest.expectedFailure()</code>	Mark Test as expected failure
<code>@unittest.removeHandler()</code>	Remove Control-C handler

Applying decorators

- Use @ symbol
- Applied to *next* item only
- Multiple decorators OK

The @ sign is used to apply a decorator to a function or class. A decorator only applies to the next definition in the script.

For simple decorators, you can also call the decorator, passing in the decorated function, and assigning the result to the original function name.

This leads to the most important thing to remember about the decorators:

```
@spam
def ham():
    pass
```

is exactly the same as

```
ham = spam(ham)
```

and

```
@spam(a, b, c)
def ham():
    pass
```

is exactly the same as

```
ham = spam(a, b, c)(ham)
```

Once you understand this, then creating decorators is just a matter of writing functions or classes and having them return the appropriate thing.

The last example could also be written as

```
wrapper = spam(a, b, c) # pass args to decorator
ham = wrapper(ham) # decorator returns wrapper, which wraps the original function
```

Table 10. Implementing Decorators

Implemented as	Decorates	Takes arguments	How to do it
function	function	N	Decorator function accepts original function and returns replacement function
function	function	Y	Decorator function accept params and returns wrapper; wrapper accepts original function and returns replacement function
function	class	N	Decorator function accepts original class and modifies original class (or returns new class)
function	class	Y	Decorator function accepts params and returns wrapper; wrapper accepts original class and modifies original class or returns new class.
class	function	N	<code>__init__</code> accepts original function; <code>__call__()</code> is replacement function
class	function	Y	<code>__init__()</code> accepts params; <code>__call__()</code> accepts original function and <i>returns</i> replacement function
class	class	N	<code>__new__()</code> accepts <code>new_class</code> and original class, returns replacement class (which is usually same as original class)
class	class	Y	<code>__init__</code> accepts params; <code>__call__()</code> accepts original class, modifies original class, and then returns replacement class (which is normally also the original class)

Trivial Decorator

- Decorator can return anything
- Not very useful, usually

A decorator does not have to be elaborate. It can return anything, though typically decorators return the same type of object they are decorating.

In this example, the decorator returns the integer value 42. This is not particularly useful, but illustrates that the decorator always replaces the object being decorated with *something*.

Example

deco_trivial.py

```
def void(thing_being_decorated):
    return 42 # replace function with 42

name = "Guido"
x = void(name) # decorate 'name', which is now 42, not a string

@void # decorate hello() function
def hello():
    print("Hello, world")

@void # decorate howdy() function
def howdy():
    print("Howdy, world")

print(hello, type(hello)) # hello is now the integer 42, not a function
print(howdy, type(howdy)) # howdy is now the integer 42, not a function
print(x, type(x))
```

deco_trivial.py

```
42 <class 'int'>
42 <class 'int'>
42 <class 'int'>
```

Simple decorator functions

- Provide a wrapper around a function
- Purposes
 - Add functionality
 - Register
 - ??? (open-ended)
- Optional arguments

A decorator function acts as a wrapper around some object (usually function or class). It allows you to add features to a function without changing the function itself. For instance, the `@property`, `@classmethod`, and `@staticmethod` decorators are used in classes.

A simple decorator function expects only one argument – the function to be modified. It should return a new function, which will replace the original. The replacement function typically calls the original function as well as some new code.

The new function should be defined with generic arguments (`*args`, `**kwargs`) so it can handle any combination of arguments for the original function.

The **`wraps`** decorator from the `functools` module in the standard library should be used with the function that returns the replacement function. This makes sure the replacement function keeps the same properties (especially the name) as the original (target) function. Otherwise, the replacement function keeps all of its own attributes.

Example

deco_debug.py

```
from functools import wraps

def debugger(old_func): # decorator function -- expects decorated (original) function as
    a parameter

    @wraps(old_func) # @wraps preserves name of original function after decoration
    def new_func(*args, **kwargs): # replacement function; takes generic parameters
        print("'" * 40) # new functionality added by decorator
        print("** function", old_func.__name__, "**") # new functionality added by
decorator

        if args: # new functionality added by decorator
            print(f"\targs are {args}")
        if kwargs: # new functionality added by decorator
            print(f"\tkwargs are {kwargs}")

        print("'" * 40) # new functionality added by decorator

        return old_func(*args, **kwargs) # call the original function

    return new_func # return the new function object

@debugger # apply the decorator to a function
def hello(greeting, whom='world'):
    print(f"{greeting}, {whom}")

hello('hello', 'world') # call new function
print()

hello('hi', 'Earth')
print()

hello('greetings')
```


deco_debug.py

```
*****
** function hello **
   args are ('hello', 'world')
*****
hello, world

*****
** function hello **
   args are ('hi', 'Earth')
*****
hi, Earth

*****
** function hello **
   args are ('greetings',)
*****
greetings, world
```

Decorator Classes

- Same purpose as decorator functions
- Decorator can keep state

A class can also be used to implement a function decorator. The advantage of using a class for a decorator is that a class can keep state, so that the replacement function can update information stored at the class level.

The decorator class must implement two methods: `__init__()` is passed the original function, and can perform any setup needed. The `__call__` method is the replacement function.

A good use for a decorator class is to log how many times a function has been called, or even keep track of the arguments it is called with (see example for this).

Example

deco_debug_class.py

```
class debugger(): # class implementing decorator

    function_calls = []

    def __init__(self, func): # original function passed into decorator's constructor
        self._func = func

    def __call__(self, *args, **kwargs): # __call__() is replacement function

        result = self._func(*args, **kwargs) # call the original function

        self.function_calls.append( # add function name and arguments to saved list
            (self._func.__name__, args, kwargs, result)
        )

        return result # return result of calling original function

    @classmethod
    def get_calls(cls): # define method to get saved function call information
        return cls.function_calls

@debugger # apply debugger to function
def hello(greeting, whom="world"):
    print(f"{greeting}, {whom}")
    return greeting

@debugger # apply debugger to function
def bark(bark_word, *, repeat=2):
    print(f"{bark_word * repeat}! ")
    return bark_word

hello('hello', 'world') # call replacement function
print()

hello('hi', 'Earth')
print()

hello('greetings')
```

```

bark("woof", repeat=3)
bark("yip", repeat=4)
bark("arf")

hello('hey', 'you')

print('-' * 60)

for i, info in enumerate(debugger.get_calls(), 1): # display function call info from
class
    print(f"{i:2d}. {info[0]:10s} {info[1]!s:20s} {info[2]!s:20s} {info[3]!s}")

```

deco_debug_class.py

```

hello, world

hi, Earth

greetings, world
woofwoofwoof!
yipypipyip!
arfarf!
hey, you
-----
1. hello      ('hello', 'world')  {}          hello
2. hello      ('hi', 'Earth')  {}          hi
3. hello      ('greetings',)   {}          greetings
4. bark       ('woof',)        {'repeat': 3} woof
5. bark       ('yip',)         {'repeat': 4} yip
6. bark       ('arf',)         {}          arf
7. hello      ('hey', 'you')    {}          hey

```

Decorators with arguments

- Decorator functions require two nested functions
- Method `__call__()` returns replacement function in classes

A decorator can be passed arguments. This requires a little extra work.

Decorator functions

For decorators implemented as functions, the decorator itself is passed the arguments; it contains a nested function that is passed the decorated function (the target), and it returns the replacement function.

Decorator classes

For decorators implemented as classes, `init` is passed the arguments, `__call__()` is passed the decorated function (the target), and `__call__` returns the replacement function.



For an example of a decorator with an optional argument, see [deco_params_optional.py](#) in the EXAMPLES folder.



There are many combinations of decorator implementation (8 total, to be exact). This is because decorators can be implemented as either functions or classes, they may take arguments, or not, and they can decorate either functions or classes. For an example of all 8 approaches, see the file [decor_ama.py](#) in the EXAMPLES folder.

Example

deco_params.py

```
#

from functools import wraps # wrapper to preserve properties of original function

def multiply(multiplier): # actual decorator -- receives decorator parameters

    def deco(old_func): # "inner decorator" -- receives function being decorated

        @wraps(old_func) # retain name, etc. of original function
        def new_func(*args, **kwargs): # replacement function -- this is called instead
of original
            result = old_func(*args, **kwargs) # call original function and get return
value
            return result * multiplier # multiple result of original function by
multiplier

        return new_func # deco() returns new_function

    return deco # multiply returns deco


@multiply(4)
def spam():
    return 5


@multiply(10)
def ham():
    return 8


a = spam()
b = ham()
print(a, b)
```

deco_params.py

```
20 80
```

Assigning decorators to imported functions

A decorator can be applied to a function imported from some other module using the assignment method.

Example

deco_post_import.py

```
from functools import cache
from random import randint
from geometry import circle_area

circle_area = cache(circle_area)
for _ in range(10000): # call circle_area() 10000 times
    result = circle_area(randint(1, 50)) # call with argument in range 1-50

print(circle_area.cache_info()) # show cache hits and misses
```

deco_post_import.py

```
CacheInfo(hits=9950, misses=50, maxsize=None, cursize=50)
```

Creating classes at runtime

- Use the **type()** function
- Provide dictionary of attributes

A class can be created programmatically, without the use of the class statement. The syntax is

```
type("name", (base_class, ...), {attributes})
```

The first argument is the name of the class, the second is a tuple of base classes (use object if you are not inheriting from a specific class), and the third is a dictionary of the class's attributes.

Instead of **type**, any other *metaclass* can be used.



It might be simpler to use the `make_dataclass()` function in the `dataclasses` module.

Example

creating_classes.py

```
def function_1(self): # create method (not inside a class -- could be a lambda)
    print("Hello from f1()")

def function_2(self): # create method (not inside a class -- could be a lambda)
    print("Hello from f2()")

NewClass = type("NewClass", (), { # create class using type() -- parameters are class
    name, base classes, dictionary of attributes
    'hello1': function_1,
    'hello2': function_2,
    'color': 'red',
    'state': 'Ohio',
})

n1 = NewClass() # create instance of new class

n1.hello1() # call instance method
n1.hello2()
print(n1.color) # access class data
print()

SubClass = type("SubClass", (NewClass,), {'fruit': 'banana'}) # create subclass of first
class
s1 = SubClass() # create instance of subclass
s1.hello1() # call method on subclass
print(s1.color) # access class data
print(s1.fruit)
```

creating_classes.py

```
Hello from f1()
Hello from f2()
red

Hello from f1()
red
banana
```

Monkey Patching

- Modify existing class or object
- Useful for enabling/disabling behavior
- Can cause problems

"Monkey patching" refers to technique of changing the behavior of an object by adding, replacing, or deleting attributes from outside the object's class definition.

It can be used for:

- Replacing methods, attributes, or functions
- Modifying a third-party object for which you do not have access
- Adding behavior to objects in memory

If you are not careful when creating monkey patches, some hard-to-debug problems can arise

- If the object being patched changes after a software upgrade, the monkey patch can fail in unexpected ways.
- Conflicts may occur if two different modules monkey-patch the same object.
- Users of a monkey-patched object may not realize which behavior is original and which comes from the monkey patch.

Monkey patching defeats object encapsulation, and so should be used sparingly.

Decorators are a convenient way to monkey-patch a class. The decorator can just add a method to the decorated class.

Example

meta_monkey.py

```
class Spam(): # create normal class

    def __init__(self, name):
        self._name = name

    def eggs(self): # add normal method
        print(f"Good morning, {self._name}. Here are your delicious fried eggs.")

s = Spam('Mrs. Higgenbotham') # create instance of class
s.eggs() # call method

def scrambled(self): # define new method outside of class
    print(f"Hello, {self._name}. Enjoy your scrambled eggs")

setattr(Spam, "eggs", scrambled) # monkey patch the class with the new method

s.eggs() # call the monkey-patched method from the instance
```

meta_monkey.py

```
Good morning, Mrs. Higgenbotham. Here are your delicious fried eggs.
Hello, Mrs. Higgenbotham. Enjoy your scrambled eggs
```

Chapter 4 Exercises

🔗 Exercise 4-1 (pres_attr.py)

Instantiate the `President` class. Get the first name, last name, and party attributes using `getattr()`.

🔗🔗 Exercise 4-2 (pres_monkey.py, pres_monkey_amb.py)

Monkey-patch the `President` class to add a method `get_full_name` which returns a single string consisting of the first name and the last name, separated by a space.



Instead of a method, make `full_name` a property.

🔗🔗 Exercise 4-3 (sillystring.py)

Without using the `class` statement, create a class named `SillyString`, which is initialized with any string. Include an instance method called `every_other` which returns every other character of the string.

Instantiate your string and print the result of calling the `every_other()` method. Your test code should look like this:

```
ss = SillyString('this is a test')
print(ss.every_other())
```

It should output

```
ti sats
```

🔗🔗 Exercise 4-4 (doubledeco.py)

Write a decorator to double the return value of any function. If a function returns 5, after decoration it should return 10. If it returns "spam", after decoration it should return "spamsam", etc.

🔗🔗🔗 Exercise 4-5 (word_actions.py)

Write a decorator, implemented as a class, to register functions that will process a list of words. The decorated functions will take one parameter — a string — and return the modified string.

The decorator itself takes two arguments — minimum length and maximum length. The class will store the min/max lengths as the key, and the functions as values, as class data.

The class will also provide a method named `process_words`, which will open `DATA/words.txt` and read it line by line. Each line contains a word.

For every registered function, if the length of the current word is within the min/max lengths, call all the function(s) whose key is that min/max pair.

In other words, if the registry key is (5, 8), and the value is [func1, func2], when the current word is within range, call `func1(w)` and `func2(w)`, where `w` is the current word.

Example of class usage:

```
word_select = WordSelect() # create callable instance

@word_select(16, 18) # register function for length 16-18, inclusive
def make_upper(s):
    return s.upper()

word_select.process_words() # loop over words, call functions if selected
```

Suggested functions to decorate:

- make the word upper-case
- put stars before or around the word
- reverse the word

Remember that all the decorated functions take one argument, which is one of the strings in the word list, and return the modified word.

Chapter 5: Iterables and Generators

Objectives

- Understand iterables vs iterators
- Develop iterator classes
- Create iterators with generators
- Unpack iterables with wildcards
- Unpack function arguments

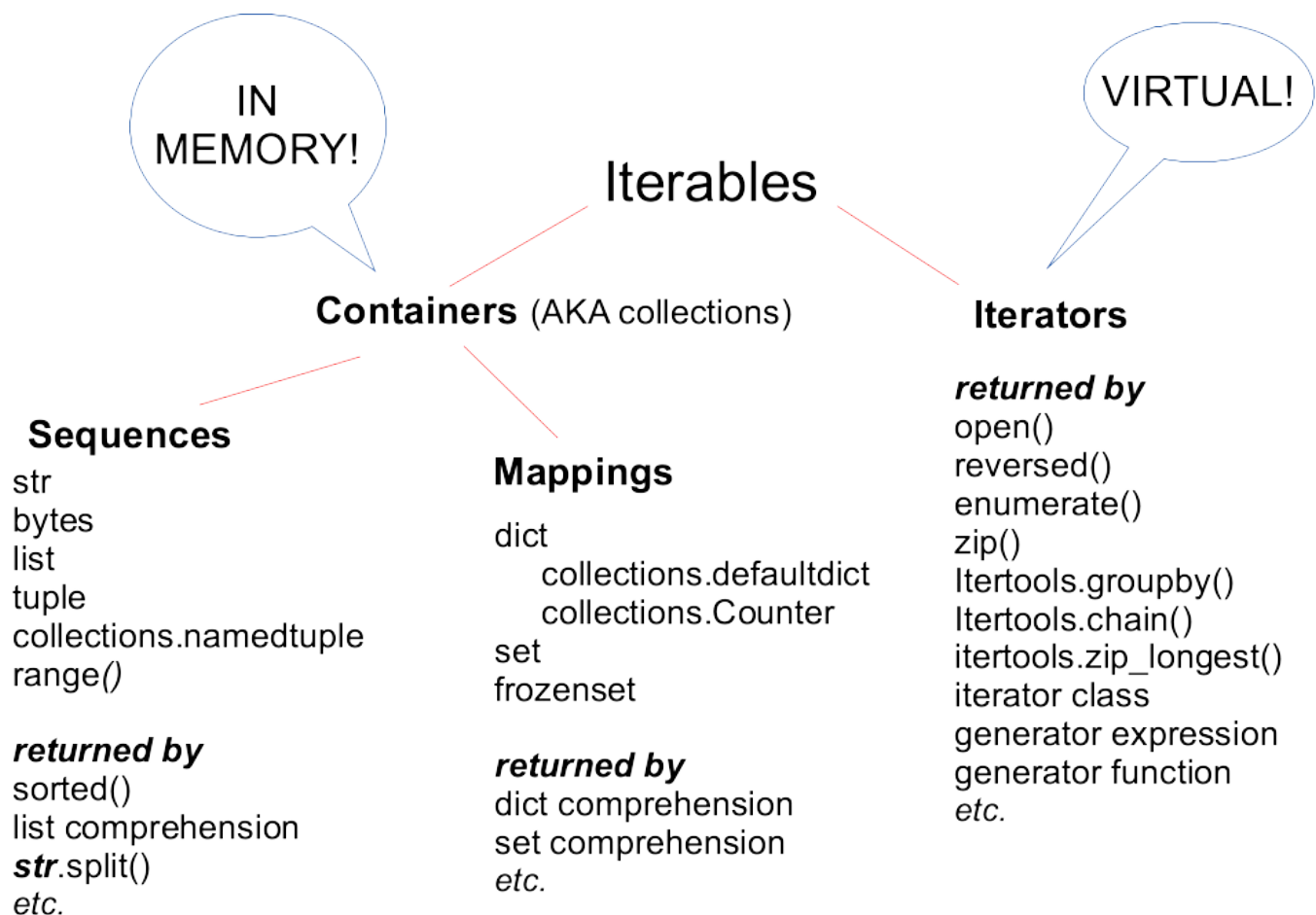
Iterables

- An iterable is an expression that can be looped through with `for`
- Some iterables are collections (`list`, `tuple`, `str`, `bytes`, `dict`, `set`)
- Many iterables are *iterators* (`enumerate()`, `dict.items()`, `open()`, `zip()`, `reversed()`, etc)

`for` is a very useful Python operator. It is used for looping through a sequence of values. The sequence of values is provided by an *iterable*. Python has many builtin iterables – a file object, for instance, which allows iterating through the lines in a file.

All collections (`list`, `tuple`, `str`, `bytes`, etc.) are iterables. They keep all their values in memory.

An *iterator* is an iterable that does not keep all its values in memory – it creates them one at a time as needed, and feeds them to the `for` loop. This saves memory.



What *exactly* is an iterable?

- Object that provides an *iterator*
- Can be **collection**, **generator**, or **iterator**

An *iterable* is an object that provides an *iterator* (via the special method `__iter__()`). An *iterator* is an object that responds to the `next()` builtin function, via the special method `__next__()`.

Any iterable can be looped over with a `for` loop.

All iterators and generators are iterables. Sequence and mapping types are also iterables.

For some iterables (including collections), you can't use `next()` on them directly; first, use the builtin function `iter()` to get the iterator.

A *generator* is an iterator created with a generator expression or a generator function. These are convenient shortcuts to creating an iterator object. You can also define a class which provides an iterator.

```
>>> r = range(1, 4)
>>> it = iter(r)    # range is not an iterator
>>> next(it)
1
>>> next(it)
2
>>> next(it)
3
>>> next(it)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
StopIteration
>>>
```


Iterators

- Lazy iterables
- Do not contain data
- Provide values on demand
- Save memory

An *iterator* is an object that provides values on demand (AKA "lazy"), rather than storing all the values (AKA "eager"). An iterator returns a new value each time `next()` is called on it, until there are no more values.

The advantage of iterators is saving memory. They can act as a *view* over a set of data.

Most iterators may only be used once. After the last value is provided, a new iterator must be created in order to start over.

Iterators are read-only, may not be indexed, and have no length. The only operations possible on an iterator is to loop over it with `for`, or consume the next element with `next()`.

Python has many builtin iterators, and there are three ways to create user-defined iterators in Python:

- iterator class
- generator expression
- generator function



Think of an iterator as a "value factory".



A `for` loop is really a `while` loop in disguise. It repeatedly calls `next()` on an iterator, and stops when `StopIteration` is raised.

Table 11. Iterables

Type	is it an iterable? <i>implements `__iter__()`</i>	Is it an iterator? <i>implements `__next__()`</i>
builtin iterator (file object, <code>enumerate()</code> , <code>reversed()</code> , etc.)	yes	yes
iterator class instance	yes	probably
generator expression generator function	yes	yes
collection (<code>list</code> , <code>dict</code> , <code>set</code> , <code>tuple</code> , etc)	yes	no

How a **for** loop really works

```
values = ["a", "b", "c"]
```

for loop

```
for value in values:  
    print(value)
```

equivalent **while** loop

```
it = iter(values)  
while True:  
    try:  
        value = next(it)  
    except Stopiteration:  
        break  
    print(value)
```

Iterator classes

- Implement `__iter__()` and `__next__()`
- Emit values via **for** or **next()**
- Raise `StopIteration` exception when there are no more values to emit

The most flexible way to create an iterator is by defining an iterator class.

Such a class *must* implement `__iter__()`, and usually also implements `__next__()`:

`__iter__()` must return an iterator — an object that implements `__next__()`. In most cases, this is the same class, so `__iter__()` just returns `self`.

`__next__()` returns the next value from the iterator.

When there are no more values to return, `__next__()` should raise `StopIteration`.

Example

trimmedfile.py

```
class TrimmedFile:
    def __init__(self, file_name): # constructor is passed file name
        self._file_in = open(file_name)

    def __iter__(self):
        # An iterator must implement iter(), which must return an iterator.
        # Typically it returns 'self', as the generator IS the iterator
        return self

    def __next__(self): # next() returns the next value of the iterator
        line = self._file_in.readline()
        if line == '':
            raise StopIteration # Raise StopIteration when there are no more values to
generate
        else:
            return line.rstrip('\n\r') # The actual work of this iterator -- remove the
end-of-line char(s)

if __name__ == '__main__':
    # To use the iterator, create an instance and iterate over it.
    trimmed = TrimmedFile('../DATA/mary.txt')
    for line in trimmed:
        print(line)
```

trimmedfile.py

```
Mary had a little lamb,
Its fleece was white as snow,
And everywhere that Mary went
The lamb was sure to go
```

Generator Expressions

- Like list comprehensions, but create a generator object
- Use parentheses rather than brackets
- Saves memory

A generator expression is similar to a list comprehension, but it provides an iterator instead of a list. That is, while a list comprehension returns a complete list, the generator expression an iterator. The iterator does not contain a copy of the source iterable (it has a *reference* to it), so it saves memory. In many cases generator expressions are also faster than list comprehensions.

The difference in syntax is that a generator expression is surrounded by parentheses, while a list comprehension uses brackets.

```
colors = ['blue', 'red', 'purple']
upper_colors = [c.upper() for c in colors] # list comprehension
upper_colors_gen = (c.upper() for c in colors) # generator expression
```

If a generator expression is passed as the only argument to a function, the surrounding parentheses are not needed:

```
my_func(float(i) for i in values)
```

Example

generator_expressions.py

```

fruits = ['watermelon', 'apple', 'mango', 'kiwi', 'apricot', 'lemon', 'guava']

ufruits = (fruit.upper() for fruit in fruits) # These are all exactly like the list
comprehension example, but return generators rather than lists
afruits = (fruit.title() for fruit in fruits if fruit.startswith('a'))

print("ufruits:", " ".join(ufruits))
print("afruits:", " ".join(afruits))
print()

values = [2, 42, 18, 92, "boom", ['a', 'b', 'c']]
doubles = (v * 2 for v in values)

print("doubles:", end=' ')
for d in doubles:
    print(d, end=' ')
print("\n")

nums = (int(s) for s in values if isinstance(s, int))
for n in nums:
    print(n, end=' ')
print("\n")

dirty_strings = ['   Gronk   ', 'PULABA   ', '   floog']

clean = (d.strip().lower() for d in dirty_strings)
for c in clean:
    print(f">{c}<", end=' ')
print("\n")

powers = ((i, i ** 2, i ** 3) for i in range(1, 11))
for num, square, cube in powers:
    print(f"{num:2d} {square:3d} {cube:4d}")

```

generator_expressions.py

```
ufruits: WATERMELON APPLE MANGO KIWI APRICOT LEMON GUAVA
```

```
afruits: Apple Apricot
```

```
doubles: 4 84 36 184 boomboom ['a', 'b', 'c', 'a', 'b', 'c']
```

```
2 42 18 92
```

```
>gronk< >pulaba< >floog<
```

```
1 1 1
2 4 8
3 9 27
4 16 64
5 25 125
6 36 216
7 49 343
8 64 512
9 81 729
10 100 1000
```


Generator functions

- Mostly like a normal function
- Use `yield` rather than `return`
- Maintains state

A generator function is defined like a normal function, but instead of a return statement, it has a `yield` statement.

The value returned by the generator function is the generator (iterator) itself.

The code in the function does not execute until the generator is passed to `next()` or is used with `for`. When the `for` loop starts, the code in the function executes up to the point of the `yield` statement. The code is paused, and the value of the `yield` statement is copied to the loop variable. On the next iteration of the loop, the function code continues running until it gets to `yield` again.

Each time the `yield` statement is reached, it provides the next value in the sequence. When there are no more values, the function returns. A generator function maintains state between calls, unlike a normal function.



You can think of a generator function as a "generator factory".

Example

sieve_generator.py

```
def next_prime(limit):
    flags = set() # initialize empty set (to be used for "is-prime" flags)

    for i in range(2, limit):
        if i in flags:
            continue
        for j in range(2 * i, limit + 1, i):
            flags.add(j) # add non-prime elements to set
        yield i # execution stops here until next value is requested by for-in loop

np = next_prime(200) # next_prime() returns a generator object
for prime in np: # iterate over yielded primes
    print(prime, end=' ')
```

sieve_generator.py

```
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71 73 79 83 89 97 101 103 107 109
113 127 131 137 139 149 151 157 163 167 173 179 181 191 193 197 199
```

Example

line_trimmer.py

```
def trimmed(file_name):  
    with open(file_name) as file_in:  
        for line in file_in:  
            yield line.rstrip('\n\r') # 'yield' causes this function to return a  
generator object  
  
mary_in = trimmed('../DATA/mary.txt')  
for trimmed_line in mary_in:  
    print(trimmed_line)
```

line_trimmer.py

```
Mary had a little lamb,  
Its fleece was white as snow,  
And everywhere that Mary went  
The lamb was sure to go
```

Iterable unpacking

- Frequently used with `for` loops
- Can be used anywhere
- Commonly used with `enumerate()` and `DICT.items()`
- Extending unpacking uses `*`

Basic unpacking

It is convenient to unpack an iterable into a list of variables. It can be done with any iterable, and is frequently done with the loop variables of a `for` loop, rather than unpacking the value of each iteration separately.

```
var1, ... = iterable
```

Extended unpacking

When unpacking iterables, sometimes you want to grab parts of the iterable as a group. This is provided by extended iterable unpacking. This can also be useful for unpacking nested iterables, where each element of a sequence is an iterable, but the nested iterables are not all the same length.

One (and only one) variable in the result of unpacking can have a star prepended. This variable will receive all values from the iterable that do not go to other variables.

```
a, *b, c = 1, 2, 3, 4, 5, 6
```



Underscore (`_`) can be used as a variable name for values you don't care about.

Example

iterable_unpacking.py

```
values = ['a', 'b', 'c']

x, y, z = values # unpack values (which is an iterable) into individual variables

print(x, y, z)
print()

people = [
    ('Bill', 'Gates', 'Microsoft'),
    ('Steve', 'Jobs', 'Apple'),
    ('Paul', 'Allen', 'Microsoft'),
    ('Larry', 'Ellison', 'Oracle'),
    ('Mark', 'Zuckerberg', 'Facebook'),
    ('Sergey', 'Brin', 'Google'),
    ('Larry', 'Page', 'Google'),
    ('Linux', 'Torvalds', 'Linux'),
]

for row in people:
    first_name, last_name, _ = row # unpack row into variables
    print(first_name, last_name)
print()

for first_name, last_name, _ in people: # a for loop unpacks if there is more than one
    variable
    print(first_name, last_name)
print()
```

iterable_unpacking.py

```
a b c
```

```
Bill Gates  
Steve Jobs  
Paul Allen  
Larry Ellison  
Mark Zuckerberg  
Sergey Brin  
Larry Page  
Linux Torvalds
```

```
Bill Gates  
Steve Jobs  
Paul Allen  
Larry Ellison  
Mark Zuckerberg  
Sergey Brin  
Larry Page  
Linux Torvalds
```

Example

extended_iterable_unpacking.py

```
values = ['a', 'b', 'c', 'd', 'e'] # values has 6 elements

x, y, *z = values # * takes all extra elements from iterable
print(f"x: {x}    y: {y}    z: {z}\n")

x, *y, z = values # * takes all extra elements from iterable
print(f"x: {x}    y: {y}    z: {z}\n")

*x, y, z = values # * takes all extra elements from iterable
print(f"x: {x}    y: {y}    z: {z}\n")

people = [
    ('Bill', 'Gates', 'Microsoft'),
    ('Steve', 'Jobs', 'Apple'),
    ('Paul', 'Allen', 'Microsoft'),
    ('Larry', 'Ellison', 'Oracle'),
    ('Mark', 'Zuckerberg', 'Facebook'),
    ('Sergey', 'Brin', 'Google'),
    ('Larry', 'Page', 'Google'),
    ('Linux', 'Torvalds', 'Linux'),
]

for *name, _ in people: # name gets all but the last field
    print(name)
print()
```

extended_iterable_unpacking.py

```
x: a    y: b    z: ['c', 'd', 'e']  
  
x: a    y: ['b', 'c', 'd']    z: e  
  
x: ['a', 'b', 'c']    y: d    z: e  
  
['Bill', 'Gates']  
['Steve', 'Jobs']  
['Paul', 'Allen']  
['Larry', 'Ellison']  
['Mark', 'Zuckerberg']  
['Sergey', 'Brin']  
['Larry', 'Page']  
['Linux', 'Torvalds']
```


Unpacking function arguments

- Convert from iterable to list of items
- Use * or **

What do you do if you have a list (or other iterable) of three values, and you want to pass them to a method that expects three positional arguments?

You could say `value[0]`, `value[1]`, `value[2]`, etc., but there's a more Pythonic way:

Use `*` to unpack any iterable into individual items. You can combine individual arguments with unpacking:

```
values = [1, 2, 3]
names = ['Rodrigo', 'Srini']
colors = ['pink', 'orange']
foo(5, 10, *values)    # same as foo(5, 10, 1, 2, 3)
bar('blue', *names, 55, *colors) # same as bar('blue', 'Rodrigo', 'Srini', 55, 'pink', 'orange')
```

In a similar way, use two asterisks to unpack a dictionary.

```
config_opts = {'filename': 'foo.txt', 'folder': 'ProjectAlpha'}
config(**config_opts) # same as config(filename='foo.txt', folder='ProjectAlpha')
```

Example

argument_unpacking.py

```
from datetime import date

dates = [
    (1968, 10, 11),
    (1968, 12, 21),
    (1969, 3, 3),
    (1969, 5, 18),
    (1969, 7, 16),
    (1969, 11, 14),
    (1970, 4, 11),
    (1971, 1, 31),
    (1971, 7, 26),
    (1972, 4, 16),
    (1927, 12, 7),
] # tuple of dates

for dt in dates:
    d = date(*dt) # instead of date(dt[0], dt[1], dt[2])
    print(d)

print()

fruits = ["pomegranate", "cherry", "apricot", "date", "Apple", "lemon", "Kiwi",
          "ORANGE", "lime", "Watermelon", "guava", "papaya", "FIG", "pear",
          "banana", "Tamarind", "persimmon", "elderberry", "peach", "BLUEberry",
          "lychee", "grape"]

sort_opts = {
    'key': lambda e: e.lower(),
    'reverse': True,
} # config info in dictionary

sorted_fruits = sorted(fruits, **sort_opts) # dictionary converted to named parameters
print(sorted_fruits)
```

argument_unpacking.py

```
1968-10-11
1968-12-21
1969-03-03
1969-05-18
1969-07-16
1969-11-14
1970-04-11
1971-01-31
1971-07-26
1972-04-16
1927-12-07
```

```
['Watermelon', 'Tamarind', 'pomegranate', 'persimmon', 'pear', 'peach', 'papaya',
'ORANGE', 'lychee', 'lime', 'lemon', 'Kiwi', 'guava', 'grape', 'FIG', 'elderberry',
'date', 'cherry', 'BLUEberry', 'banana', 'apricot', 'Apple']
```

Chapter 5 Exercises

Exercise 5-1 (pres_upper_gen.py)

Step 1

Create a generator expression to read all the presidents' first and last names from `presidents.txt` into a iterator of tuples.

HINT: the file object itself is a generator you can loop over inside this generator expression

Step 2

Create a second generator expression that uses the first one to make a new iterator with the names joined by spaces, and in upper case.

Step 3

Loop through the second generator and print out the names one per line.



Since everything in this lab is an iterator, the file must remain open through step 3.

Exercise 5-2 (fauxrange.py)

Write a generator class named `FauxRange` that emulates the builtin `range()` function. Instances should take up to three arguments, and provide a range of integers.

Chapter 6: Consuming RESTful Data

Objectives

- Writing REST clients
 - Opening URLs and downloading data
 - Fetching data from RESTful servers
- Uploading data to a server

The REST API

- Based on HTTP verbs
 - GET
 - get all records
 - get details for one record
 - POST add a new record
 - PUT replace a record (all fields)
 - PATCH update a record (some fields)
 - DELETE delete a record

REST stands for **R**epresentational **S**tate **T**ransfer, first described (and named) by Roy Fielding in 2000. It is not a protocol or structure, but rather an architectural style resulting from a set of guidelines. It provides for loosely-coupled resource management over HTTP. REST does not enforce a particular implementation.

A RESTful site provides *resources*, which contain *records*. The same API typically contains more than one resource; each has a different *endpoint* (URL).

A RESTful API uses HTTP verbs to manipulate records. The same endpoint can be used for all access; what happens depends on a combination of which HTTP verb is used, plus whether there is more information on the URL.

HTTP Verbs

With just the endpoint (e.g., <https://www.wombats.com/api/v1/wombat>)

- GET retrieves links to all records. Query strings can be used to sort or filter the list.
- POST adds a new record

With endpoint plus record ID (e.g., <https://www.wombats.com/api/v1/wombat/123>)

- GET retrieves the details for that record
- PUT replaces the record
- PATCH updates the record
- DELETE removes the record

Example API

The Art Institute of Chicago API provides the following endpoints:

```
https://api.artic.edu/api/v1/artworks
https://api.artic.edu/api/v1/agents
https://api.artic.edu/api/v1/places
https://api.artic.edu/api/v1/galleries
https://api.artic.edu/api/v1/exhibitions
https://api.artic.edu/api/v1/agent-types
https://api.artic.edu/api/v1/agent-roles
https://api.artic.edu/api/v1/artwork-place-qualifiers
https://api.artic.edu/api/v1/artwork-date-qualifiers
https://api.artic.edu/api/v1/artwork-types
https://api.artic.edu/api/v1/category-terms
https://api.artic.edu/api/v1/images
https://api.artic.edu/api/v1/videos
https://api.artic.edu/api/v1/sounds
https://api.artic.edu/api/v1/texts
```



A list of public RESTful APIs is located here: <https://github.com/public-apis/public-apis>

When is REST not REST?

- REST is guidelines, not protocol
- Implementers are not consistent
- YMMV (your mileage may vary)

REST is a set of guidelines, not a specific protocol. Because of this, REST implementations vary widely. For instance, many APIs use more than one endpoint for the same resource. Many APIs do not return a list of links on a GET request to the endpoint, but the details for every resource. Many APIs vary widely on how you send credentials.

It is thus important to read the docs for each individual API to see exactly what they expect, and what they provide.

The good news is that the variations are mostly in the URLs you need to construct — making the requests and parsing out the data are generally about the same for most APIs.

Consuming REST APIs

- Use `requests.method()`
- Specify parameters, headers, etc. as dictionary
- Use `response.content` to get raw data
- Use `response.json()` to convert JSON to Python

The `requests` module is used to get data from RESTful APIs.

To add GET parameters, pass a dictionary of key-value parameters with the `params` keyword argument:

```
get_parameters = { 'key1': 'value1', ... }  
response = requests.get(URL, params=get_parameters)
```

For POST data, pass a dictionary with the `data` (dictionary) or `json` (JSON data) arguments.

```
post_data = { 'key1': 'value1', ... }  
response = requests.post(URL, data=post_data)
```

To use a proxy, pass a dictionary of protocol/url parameters with the `proxies` keyword

```
proxies={'http':'https://proxy.something.com:1234'}  
response = requests.get(URL, proxies=proxies)
```

To convert a JSON response into a Python data structure, use the `.json()` method on the `response` object.



See the quickstart guide for requests at: <https://requests.readthedocs.io/en/latest/user/quickstart/>

Example

rest_consumer_omdb.py

```
import requests
from pprint import pprint

with open('omdbapikey.txt') as api_in:
    OMDB_API_KEY = api_in.read().rstrip()

OMDB_URL = "http://www.omdbapi.com"

def main():
    requests_params = {'t': 'Black Panther', "apikey": OMDB_API_KEY}
    response = requests.get(OMDB_URL, params=requests_params)
    if response.status_code == requests.codes.OK:
        raw_data = response.json()

        print(f"raw_data['Title']: {raw_data['Title']}")
        print(f"raw_data['Director']: {raw_data['Director']}")
        print(f"raw_data['Year']: {raw_data['Year']}")
        print(f"raw_data['Runtime']: {raw_data['Runtime']}")
        print()

        print('-' * 60)

        print("raw DATA:")
        pprint(response.json())
    else:
        print(f"response.status_code: {response.status_code}")

if __name__ == '__main__':
    main()
```

rest_consumer_omdb.py

```

raw_data['Title']: Black Panther
raw_data['Director']: Ryan Coogler
raw_data['Year']: 2018
raw_data['Runtime']: 134 min

-----
raw DATA:
{'Actors': "Chadwick Boseman, Michael B. Jordan, Lupita Nyong'o",
'Awards': 'Won 3 Oscars. 125 wins & 289 nominations total',
'BoxOffice': '$700,426,566',
'Country': 'United States',
'DVD': 'N/A',
'Director': 'Ryan Coogler',
'Genre': 'Action, Adventure, Sci-Fi',
'Language': 'English, Swahili, Nama, Xhosa, Korean',
'Metascore': '88',
'Plot': "T'Challa, heir to the hidden but advanced kingdom of Wakanda, must "
        'step forward to lead his people into a new future and must confront '
        "a challenger from his country's past.",
'Poster': 'https://m.media-
amazon.com/images/M/MV5BMTg1MTY2MjYzNV5BMl5BanBnXkFtZTgwMTc4NTMwNDI@._V1_SX300.jpg',
'Production': 'N/A',
'Rated': 'PG-13',
'Ratings': [{'Source': 'Internet Movie Database', 'Value': '7.3/10'},
             {'Source': 'Rotten Tomatoes', 'Value': '96%'},
             {'Source': 'Metacritic', 'Value': '88/100'}],
'Released': '16 Feb 2018',
'Response': 'True',
'Runtime': '134 min',
'Title': 'Black Panther',
'Type': 'movie',
'Website': 'N/A',
'Writer': 'Ryan Coogler, Joe Robert Cole, Stan Lee',
'Year': '2018',
'imdbID': 'tt1825683',
'imdbRating': '7.3',
'imdbVotes': '900,774'}

```

Table 12. Keyword Parameters for `requests` methods

Option	Data Type	Description
<code>allow_redirects</code>	<code>bool</code>	set to True if PUT/POST/DELETE redirect following is allowed
<code>auth</code>	<code>tuple</code>	authentication pair (user/token,password/key)
<code>cert</code>	<code>str</code> or <code>tuple</code>	path to cert file or ('cert', 'key') tuple
<code>cookies</code>	<code>dict</code> or <code>CookieJar</code>	cookies to send with request
<code>data</code>	<code>dict</code>	parameters for a POST or PUT request
<code>files</code>	<code>dict</code>	files for multipart upload
<code>headers</code>	<code>dict</code>	HTTP headers
<code>json</code>	<code>str</code>	JSON data to send in request body
<code>params</code>	<code>dict</code>	parameters for a GET request
<code>proxies</code>	<code>dict</code>	map protocol to proxy URL
<code>stream</code>	<code>bool</code>	if False, immediately download content
<code>timeout</code>	<code>float</code> or <code>tuple</code>	timeout in seconds or (connect timeout, read timeout) tuple
<code>verify</code>	<code>bool</code>	if True, then verify SSL cert



These can be used with any of the HTTP request types, as appropriate.

Table 13. `requests.Response` methods and attributes

Method/attribute	Definition
<code>apparent_encoding</code>	Returns the apparent encoding
<code>close()</code>	Closes the connection to the server
<code>content</code>	Content of the response, in bytes
<code>cookies</code>	A Cookiejar object with the cookies sent back from the server
<code>elapsed</code>	A timedelta object with the time elapsed from sending the request to the arrival of the response
<code>encoding</code>	The encoding used to decode <code>r.text</code>
<code>headers</code>	A dictionary of response headers
<code>history</code>	A list of response objects holding the history of request (url)
<code>is_permanent_redirect</code>	True if the response is the permanent redirected url, otherwise False
<code>is_redirect</code>	True if the response was redirected, otherwise False
<code>iter_content()</code>	Iterates over the response
<code>iter_lines()</code>	Iterates over the lines of the response
<code>json()</code>	Converts JSON content to Python data structure (if the result was written in JSON format, if not it raises an error)
<code>links</code>	The header links
<code>next</code>	A PreparedRequest object for the next request in a redirection
<code>ok</code>	True if <code>status_code</code> is less than 400, otherwise False
<code>raise_for_status()</code>	If an error occur, this method a <code>HTTPError</code> object
<code>reason</code>	A text corresponding to the status code
<code>request</code>	The request object that requested this response
<code>status_code</code>	A number that indicates the status (200 is OK, 404 is Not Found)
<code>text</code>	The content of the response, in unicode
<code>url</code>	The URL of the response

Printing JSON

- Default output of JSON is ugly
- `pprint` makes structures human friendly
- Use `pprint.pprint()`

When debugging JSON data, the `print` command is not so helpful. The Python data structure parsed from JSON is just printed out all jammed together, one element after another, and is hard to read.

The `pprint` (pretty print) module will analyze a structure and print it out with indenting, to make it much easier to read.

By default, `pprint()` displays dictionary keys in sorted order. It usually makes sense to view the keys in the same order they were in the JSON file. To do this, add the `sort_dicts=False` parameter to `pprint()`.

You can customize the output even more with other parameters: `indent` (default 1) specifies how many spaces to indent nested structures; `width` (default 80) constrains the width of the output; `depth` (default unlimited) says how many levels to print – levels beyond depth are shown with '...'.

Requests sessions

- Share configuration across requests
- Can be faster
- Instance of `requests.Session`
- Supports context manager (*with* statement)

To make it more convenient to share HTTP information across multiple requests, **requests** provides *sessions*. You can use a session by creating an instance of **requests.Session**. Once the session is created, it contains attributes containing the named arguments to request methods, such as `session.params` or `session.headers`.

A Session object implements the context manager protocol, so it can be used with the **with** statement. This will automatically close the session.

These are normal Python dictionaries, so you can use the *update* method to add information:

```
with requests.Session() as session:
    session.params.update({'token': 'MY_TOKEN'})
    session.headers.update({'accept': 'application/xml'})

    response1 = session.get(...)
    response2 = session.get(...)
```

Then you can call the HTTP methods (**get**, **post**, etc.) from the session object. Any parameters passed to an HTTP method will overwrite those that already exist in the session, but do not update the session.



For more examples of using REST in a real-world situation, see `consume_omdb_main.py` in the **EXAMPLES** folder. This script imports various modules that connect to the OMDB API using various multitasking approaches.

Example

rest_consumer_omdb_sessions.py

```
import requests
from pprint import pprint

with open('omdbapikey.txt') as api_in:
    OMDB_API_KEY = api_in.read().rstrip()

OMDB_URL = "http://www.omdbapi.com"

MOVIE_TITLES = [
    'Black Panther',
    'Frozen',
    'Top Gun: Maverick',
    'Bullet Train',
    'Death on the Nile',
    'Casablanca',
]

def main():
    with requests.Session() as session:
        session.params.update({"apikey": OMDB_API_KEY})
        for movie_title in MOVIE_TITLES:
            params = {'t': movie_title}
            response = session.get(OMDB_URL, params=params)
            if response.status_code == requests.codes.OK:
                raw_data = response.json()
                print(f"raw_data['Title']: {raw_data['Title']}")
                print(f"raw_data['Director']: {raw_data['Director']}")
                print(f"raw_data['Year']: {raw_data['Year']}")
                print(f"raw_data['Runtime']: {raw_data['Runtime']}")
                print()

if __name__ == '__main__':
    main()
```


rest_consumer_omdb_sessions.py

```
raw_data['Title']: Black Panther
raw_data['Director']: Ryan Coogler
raw_data['Year']: 2018
raw_data['Runtime']: 134 min

raw_data['Title']: Frozen
raw_data['Director']: Chris Buck, Jennifer Lee
raw_data['Year']: 2013
raw_data['Runtime']: 102 min

raw_data['Title']: Top Gun: Maverick
raw_data['Director']: Joseph Kosinski
raw_data['Year']: 2022
raw_data['Runtime']: 130 min

raw_data['Title']: Bullet Train
raw_data['Director']: David Leitch
raw_data['Year']: 2022
raw_data['Runtime']: 127 min

raw_data['Title']: Death on the Nile
raw_data['Director']: Kenneth Branagh
raw_data['Year']: 2022
raw_data['Runtime']: 127 min

raw_data['Title']: Casablanca
raw_data['Director']: Michael Curtiz
raw_data['Year']: 1942
raw_data['Runtime']: 102 min
```

Posting data to a RESTful server

- use `requests.post(url, data=dict)`

The **POST** operation adds a new record to a resource using the endpoint.

To post to a RESTful service, use the **data** argument to **request's post** function. It takes a dictionary of parameters, which will be URL-encoded automatically.

If the POST is successful, the server should return response data with a link to the newly created record, with an HTTP response code of 201 ("created") rather than 200 ("OK").

Example

post_to_rest.py

```
from datetime import datetime
import time
import requests

URL = 'http://httpbin.org/post'

for i in range(3):
    response = requests.post( # POST data to server
        URL,
        data={'date': datetime.now(),
              'label': 'test_' + str(i)
            },
        cookies={'python': 'testing'},
        headers={'X-Python': 'Guido van Rossum'},
    )
    if response.status_code in (requests.codes.OK, requests.codes.created):
        print(response.status_code)
        print(response.text)
        print()
        time.sleep(2)
```

post_to_rest.py

```
200
{
  "args": {},
  "data": "",
  "files": {},
  "form": {
    "date": "2026-02-23 20:09:26.533681",
    "label": "test_0"
  },
  "headers": {
    "Accept": "*/*",
    "Accept-Encoding": "gzip, deflate",
    "Content-Length": "48",
    "Content-Type": "application/x-www-form-urlencoded",
    "Cookie": "python=testing",
    "Host": "httpbin.org",
    "User-Agent": "python-requests/2.32.5",
    "X-Amzn-Trace-Id": "Root=1-699cfa46-6335bd884bac5f8272bd6a97",
    "X-Python": "Guido van Rossum"
  },
  "json": null,
  "origin": "136.47.241.207",
  "url": "http://httpbin.org/post"
}
```

```
200
{
  "args": {},
  "data": "",
```

...

Other operations

- Use **requests** functions
 - put
 - delete
 - patch
 - head
 - options

For other HTTP operations, **requests** provides appropriately named functions.

Example

other_web_service_ops.py

```
import requests

print('PUT:')
r = requests.put("http://httpbin.org/put", data={'spam': 'ham'}) # send data via HTTP PUT
request
print(r.status_code, r.text)
print('-' * 60)

print('DELETE:')
r = requests.delete("http://httpbin.org/delete") # send HTTP DELETE request
print(r.status_code, r.text)
print('-' * 60)

print('HEAD:')
r = requests.head("http://httpbin.org/get") # get HTTP headers via HEAD request
print(r.status_code, r.text)
print(r.headers)
print('-' * 60)

print('OPTIONS:')
r = requests.options("http://httpbin.org/get") # get negotiated HTTP options
print(r.status_code, r.text)
print('-' * 60)
```

other_web_service_ops.py

```

PUT:
200 {
  "args": {},
  "data": "",
  "files": {},
  "form": {
    "spam": "ham"
  },
  "headers": {
    "Accept": "*/*",
    "Accept-Encoding": "gzip, deflate",
    "Content-Length": "8",
    "Content-Type": "application/x-www-form-urlencoded",
    "Host": "httpbin.org",
    "User-Agent": "python-requests/2.32.5",
    "X-Amzn-Trace-Id": "Root=1-699cfa4d-62bed03c49545d656e9c1cf6"
  },
  "json": null,
  "origin": "136.47.241.207",
  "url": "http://httpbin.org/put"
}

```

```

-----
DELETE:
200 {
  "args": {},
  "data": "",
  "files": {},
  "form": {},
  "headers": {
    "Accept": "*/*",
    "Accept-Encoding": "gzip, deflate",
    "Content-Length": "0",
    "Host": "httpbin.org",
    "User-Agent": "python-requests/2.32.5",
    "X-Amzn-Trace-Id": "Root=1-699cfa4d-1939e1514d8d33e7490423b5"
  },
  "json": null,
  "origin": "136.47.241.207",
  "url": "http://httpbin.org/delete"
}

```

```

-----
HEAD:
200

```

```
{'Date': 'Tue, 24 Feb 2026 01:09:33 GMT', 'Content-Type': 'application/json', 'Content-  
Length': '307', 'Connection': 'keep-alive', 'Server': 'unicorn/19.9.0', 'Access-Control-  
Allow-Origin': '*', 'Access-Control-Allow-Credentials': 'true'}
```

OPTIONS:

200

Using Thunder Client

- Graphical HTTP Client
- Specify headers and parameters
- Save searches

Thunder Client is an extension to VS Code that provides a GUI-based HTTP client. It lets you specify URLs, plus headers, parameters, data, and any other information that needs to be sent to a web server. You can save searches for replay, and easily make changes to the requests.

This makes it really easy to experiment with REST endpoints without having to constantly re-run your Python scripts.

It also displays the JSON (or other) response in various modes, pretty-printed with syntax highlighting.

Install Thunder Client via the Extensions badge on the activity panel on the far left side of VS Code.

The screenshot shows the Thunder Client interface within a VS Code editor. The top bar displays several open files: `consumer_omdb.py M`, `rest_consumer_omdb_sessions.py 5`, `api_to_api_transfer.txt U`, `post_to_rest.py M`, and `other_web_service`. The left sidebar contains the 'THUNDER CLIENT' panel with tabs for 'New Request', 'Activity', 'Collections', and 'Env'. The 'Collections' tab is active, showing a list of saved requests under the 'ARTIC' collection. One request is visible: a GET request to `api.artic.edu/api/v1/artworks/4` with a status of 'just now'. The main editor area shows the details of the selected request. The 'Query' tab is active, displaying 'Query Parameters' with a table for adding parameters. The right panel shows the 'Response' tab, displaying the JSON response from the API, which includes details about an artwork, such as its ID, title, and image data.

Chapter 6 Exercises

Exercise 6-1 (noaa_precip.py)

NOAA provides an API for climate data. The base page is <https://www.ncdc.noaa.gov/cdo-web/webservices/v2#gettingStarted>.

You will need to request a token via email. Specify the token with the `token` header. Use the `headers` parameter to `requests.get()`.

You can get a list of the web services as described on the page.

For hourly precipitation, the dataset ID is "PRECIP_HLY".

The station ID for Boaz, Alabama is "COOP:010957"

Using your token, the dataset ID, and the station ID, fetch hourly precipitation data for that location from January 1, 1970 through January 31, 1970.

The endpoint is

<https://www.ncdc.noaa.gov/cdo-web/api/v2/data>

and you will need to specify the parameters **datasetid**, **stationid**, **startdate**, and **enddate**.

Remember that only a certain number of values are returned with each call. For purposes of this lab, just get the first page of values.

Try varying the parameters, based on the API docs.

See [ANSWERS/noaa_metadata.py](#) for a script to retrieve values for some of the endpoints.

Chapter 7: Database Access

Objectives

- Understand the Python DB API architecture
- Connect to a database
- Execute simple and parameterized queries
- Fetch single and multiple row results
- Execute non-query statements
- Get metadata about a query
- Start transactions and commit or rollback as needed

The DB API

- Most popular Python DB interface
- Specification, not abstract class
- Many modules for different DB implementations
- Hides actual DBMS implementation

To make database programming simpler and more consistent, Python provides the DB API. This is an API to standardize working with databases. When a package is written to access a database, it is written to conform to the API, and thus programmers do not have to learn a new set of methods and functions for each different database architecture.

DB API objects and methods

```
conn = package.connect(connection-arguments)
cursor = conn.cursor()
num_lines = cursor.execute(query)
num_lines = cursor.execute(query-with-placeholders, iterable)
num_lines = cursor.executemany(query-with-placeholders, iterable)
all_rows = cursor.fetchall()
some_rows = cursor.fetchmany(n)
one_row = cursor.fetchone()
conn.commit()
conn.rollback()
```

Table 14. Available Interfaces (using Python DB API-2.0)

Database	Python package
Firebird (and Interbase)	KInterbasDB
Databricks	databricks-sql-connector
Elasticsearch	elasticsearch-dbapi
Excel	excel-dbapi
Google BigQuery	google-cloud-bigquery
IBM DB2	ibm-db
Ingres	ingmod
Microsoft SQL Server	pymssql
MySQL	pymysql mysql-connector-python (official library) mysqlclient (written in C)
ODBC	pyodbc
Oracle	oracledb cx_oracle (obsolete as of 2022)
PostgreSQL	psycopg (previously psycopg2)
SAP DB (also known as "MaxDB")	sapdbapi
SQLite	sqlite3
Sybase	Sybase



This list is not comprehensive, and there are additional interfaces available for some of the listed DBMSs.

Connecting to a Server

- Import appropriate library
- Use `connect()` to get a connection object
- Specify host, database, username, password, etc.

To connect to a database server, import the package for the specific database. Use the package's `connect()` method to get a connection object. The `connect()` function requires the information needed to access the database, which may include the host, initial database, username, or password.

Argument names for the `connect()` method are not consistent across packages. Most `connect()` methods use individual arguments, such as **host**, **database**, etc., but some use a single string argument.

When finished with the connection, call the `close()` method on the connection object. Many database modules support the context manager `with` statement, and will automatically close the database when the `with` block is exited. Check the documentation to see how this is implemented for a particular database.

Example

```
import pymysql

conn = pymysql.connect (host = "dbserver",
                        user = "adeveloper",
                        passwd = "s3cr3t",
                        db = "samples")

# Interact with database here ...
conn.close()
```

```
import sqlite3

with sqlite3.connect('sample.db') as conn:
    # Interact with database here ...
```

connect() examples

PostgreSQL using `psycopg`

```
conn = psycopg.connect(  
    host="localhost",  
    dbname="postgres",  
    user="adeveloper",  
    password='$3cr3t',  
)
```

Oracle using `oracledb`

```
conn = oracledb.connect(  
    user='adeveloper',  
    password='$3cr3t',  
    dsn="dbhost.example.com/orclpdb"  
)
```

```
conn = oracledb.connect(  
    user='adeveloper',  
    password='$3cr3t',  
    host="localhost",  
    port=1521,  
    servicename="orclpdb",  
)
```

SQLite3 using `sqlite3`

```
conn = sqlite3.connect('testdb') # on-disk database(single file)
```

```
conn = sqlite3.connect(':memory:') # in-memory database
```

Microsoft SQLServer using `pymssql`

```
conn = pymssql.connect(  
    server=r"HOST\INSTANCE",  
    user="adeveloper", # SQLServer user, not Windows user  
    password="s3cr3t", # SQLServer password  
    database="testdb")
```

```
conn = pymssql.connect (
    dsn="DSN",
)
```

MySQL/MariaDB using `pymysql`

```
conn = pymysql.connect (
    host="localhost",
    user="adeveloper",
    passwd="$3cr3t",
    db="testdb",
)
```

Any ODBC-compliant DB using `pyodbc`

```
conn = pyodbc.connect(''
    DRIVER={SQL Server};
    SERVER=localhost;
    DATABASE=testdb;
    UID=adeveloper;
    PWD=$3cr3t
'')
```

```
conn = pyodbc.connect('DSN=testdsn;PWD=$3cr3t')
```



`pyodbc.connect()` has one (string) parameter, not multiple parameters

IBM DB2 using `ibm-db`

```
import ibm_db_dbi as db2
conn = db2.connect(

    "DATABASE=testdb;HOSTNAME=localhost;PORT=50000;PROTOCOL=TCPIP;UID=db2inst1;PWD=scripts;",
    "",
    ""
)
```

Creating a Cursor

- Cursor can execute SQL statements
- Create with `cursor()` method
- Multiple cursors available
 - Standard cursor
 - Returns rows as tuples
 - Other cursors
 - Return dictionary
 - Return hybrid dictionary/list
 - Leave data on server

Once you have a connection object, call `cursor()` to create a cursor object. A cursor is an object that can execute SQL code and fetch results. Each connection may have one or more active cursors.

The default cursor for most packages returns each row as a tuple of values. There are optional cursors that can return data in different formats, or that control whether data is stored on the client or the server.



The examples in this chapter are implemented with SQLite, since the `sqlite3` module is part of the standard library. Most of the examples are also implemented for PostgreSQL and MySQL. See `db_pymysql_*.py`, `db_mysql_connector_*.py`, and `db_psycopg_*.py` in EXAMPLES.

Querying data

- **`cursor.execute(query)`**
 - Gets all data from query
 - Returns # rows in result set
- Cursor is iterator over result set
- Return rows as tuples of values

Once you have a cursor, you can use it to execute queries via the `execute()` method. The first argument to `execute()` is a string containing one SQL statement.

For queries, `execute()` returns the number of rows in the result set. For non-query statements, `execute()` returns the number of rows affected by the operation.

The cursor object is an iterator over the rows in the result set.

For standard cursors, all data is transferred from the database server to your program's memory when `execute()` is called.



For `sqlite3`, `execute()` returns the cursor object, so you can say `execute(QUERY-STATEMENT).fetchall()`.

Example

db_sqlite3_basics.py

```
import sqlite3

# conn = sqlite3.Connection(...)
with sqlite3.connect("../DATA/presidents.db") as conn: # connect to the database

    s3_cursor = conn.cursor() # get a cursor object

    # select specified columns from all presidents
    s3_cursor.execute('''
        select termnum, firstname, lastname, party
        from presidents
    ''') # execute a SQL statement

    for term, firstname, lastname, party in s3_cursor:
        print(f"{term:2d} {firstname:25} {lastname:20} {party}")
    print()
```

db_sqlite3_basics.py

1	George	Washington	no party
2	John	Adams	Federalist
3	Thomas	Jefferson	Democratic - Republican
4	James	Madison	Democratic - Republican
5	James	Monroe	Democratic - Republican
6	John Quincy	Adams	Democratic - Republican
7	Andrew	Jackson	Democratic
8	Martin	Van Buren	Democratic
9	William Henry	Harrison	Whig
10	John	Tyler	Whig

...

39	James Earl 'Jimmy'	Carter	Democratic
40	Ronald Wilson	Reagan	Republican
41	George Herbert Walker	Bush	Republican
42	William Jefferson 'Bill'	Clinton	Democratic
43	George Walker	Bush	Republican
44	Barack Hussein	Obama	Democratic
45	Donald John	Trump	Republican
46	Joseph Robinette	Biden	Democratic
47	Donald John	Trump	Republican

Fetch methods

- Alternate way to get rows
 - `fetchone()` get next row
 - `fetchmany()` get N rows
 - `fetchall()` get all remaining rows

Cursors provide three methods for returning query results. These can be used instead of iterating over the cursor.

`fetchone()` returns the next available row from the query results.

`fetchmany(n)` returns up to n rows. This is useful when the query returns a large number of rows.

`fetchall()` returns a tuple of all rows.

For all three methods, each row is returned as a tuple of values.

Example

db_sqlite3_fetch.py

```
import sqlite3

# conn = sqlite3.Connection(...)
with sqlite3.connect("../DATA/presidents.db") as conn: # connect to the database

    s3_cursor = conn.cursor() # get a cursor object

    # select specified columns from all presidents
    s3_cursor.execute('''
        select termnum, firstname, lastname, party
        from presidents
    ''') # execute a SQL statement

    row = s3_cursor.fetchone()
    print(f"{row = }")
    print('-' * 60)

    for row in s3_cursor.fetchmany(5):
        print(row)
    print('-' * 60)

    for row in s3_cursor.fetchall():
        print(row)
    print()
```

db_sqlite3_fetch.py

```
row = (1, 'George', 'Washington', 'no party')
-----
(2, 'John', 'Adams', 'Federalist')
(3, 'Thomas', 'Jefferson', 'Democratic - Republican')
(4, 'James', 'Madison', 'Democratic - Republican')
(5, 'James', 'Monroe', 'Democratic - Republican')
(6, 'John Quincy', 'Adams', 'Democratic - Republican')
-----
(7, 'Andrew', 'Jackson', 'Democratic')
(8, 'Martin', 'Van Buren', 'Democratic')
```

...

```
(39, "James Earl 'Jimmy'", 'Carter', 'Democratic')
(40, 'Ronald Wilson', 'Reagan', 'Republican')
(41, 'George Herbert Walker', 'Bush', 'Republican')
(42, "William Jefferson 'Bill'", 'Clinton', 'Democratic')
(43, 'George Walker', 'Bush', 'Republican')
(44, 'Barack Hussein', 'Obama', 'Democratic')
(45, 'Donald John', 'Trump', 'Republican')
(46, 'Joseph Robinette', 'Biden', 'Democratic')
(47, 'Donald John', 'Trump', 'Republican')
```

Non-query statements

- Update database
- Returns count of rows affected
- Changes must be committed

The `execute()` method is also used to execute non-query statements, such as **CREATE**, **ALTER**, **UPDATE**, and **DROP**.

As with queries, the first argument is a string containing one SQL statement.

For most DB packages, `execute()` returns the number of rows affected.

To make changes to the database permanent, changes must be committed with `CONNECTION.commit()`.

Example

db_sqlite3_add_row.py

```
from datetime import date
import sqlite3

with sqlite3.connect("../DATA/presidents.db") as s3conn: # connect to database

    sql_insert = """
    insert into presidents
    (lastname, firstname, termstart, termend, birthplace, birthstate, birthdate,
    deathdate, party)
    values ('Ramirez', 'Mary', '2025-01-20', null, 'Topeka',
    'Kansas', '1968-09-22', null, 'Independent')
    """

    cursor = s3conn.cursor()

    try:
        cursor.execute(sql_insert)
    except (sqlite3.OperationalError, sqlite3.DatabaseError, sqlite3.DataError) as err:
        print(err)
        s3conn.rollback()
    else:
        s3conn.commit()
    finally:
        cursor.close()
```

Example

db_sqlite3_delete_row.py

```
from datetime import date
import sqlite3

with sqlite3.connect("../DATA/presidents.db") as conn: # connect to DB

    sql_delete = """
    delete from presidents
    where TERMNUM = 48
    """

    cursor = conn.cursor() # get a cursor

    try:
        cursor.execute(sql_delete)
    except (sqlite3.DatabaseError, sqlite3.OperationalError, sqlite3.DataError) as err:
        print(err)
        conn.rollback()
    else:
        conn.commit()
    finally:
        cursor.close()
```


SQL Injection

- Hijacks SQL code
- Result of string formatting
- Always use parameterized statements

One kind of vulnerability in SQL code is called *SQL injection*. This happens when using string formatting and raw user input to build SQL statements. An attacker can embed malicious SQL commands in input data.

Since the programmer is generating the SQL code as a string, there is no way to check for malicious SQL code. It is best practice to use parameterized statements, which prevents any user input from being *injected* into the SQL statement.



see <http://www.xkcd.com/327> for a well-known web comic on this subject.

Example

db_sql_injection.py

```
#
good_input = 'Google'
malicious_input = "'; drop table customers; -- " # input would come from a web form, for
instance

naive_format = "select * from customers where company_name = '{} ' and company_id != 0"

good_query = naive_format.format(good_input) # string formatting naively adds the user
input to a field, expecting only a customer name
malicious_query = naive_format.format(malicious_input) # string formatting naively adds
the user input to a field, expecting only a customer name

print("Good query:")
print(good_query) # non-malicious input works fine
print()

print("Bad query:")
print(malicious_query) # query now drops a table ('--' is SQL comment)
```

db_sql_injection.py

```
Good query:
select * from customers where company_name = 'Google' and company_id != 0

Bad query:
select * from customers where company_name = "'; drop table customers; -- ' and
company_id != 0
```

Parameterized Statements

- Prevent SQL injection
- More efficient updates
- Use placeholders in query
 - Placeholders vary by DB
- Pass iterable of parameters
- Use `cursor.execute()` or `cursor.executemany()`

When using parameterized statements, you specify a SQL statement as usual, but use placeholders for the user-supplied data. Add an argument to `.execute()` containing the values for the placeholders. Placeholders can be either positional or named. Some DBMSs support both.

Parameterized statements not only protect against SQL injection, but they also are usually more efficient. The DBMS only parses the SQL statement once, then fills in data on each call to `.execute()`.

All SQL statements may be parameterized, including queries.

Different database modules use different placeholders. To see what kind of placeholder a module uses, check `_package_.paramstyle`.

Positional parameters

Positional parameters are filled in from an iterable. Values are filled in from left to right.

psycopg

```
cursor.execute("insert into users (name, id) values (%s, %s)", ["Bob", 123])
```

sqlite3

```
cursor.execute("insert into users (name, id) values (?, ?)", ["Bob", 123])
```

oracledb

```
cursor.execute("insert into users (name, id) values (:1, :2)", ["Bob", 123])
```

Named parameters

Named parameters are filled in from a dictionary. They have the advantage of not needing to be in a particular order.



oracledb supports specifying parameter values as named arguments to `.execute()` as well as a dictionary.

psycopg

```
cursor.execute(
    "insert into users (name, id) values %(name)s, %(id)s",
    {'name': "Bob", 'id': 123}
)
```

sqlite3

```
cursor.execute(
    "insert into users (name, id) values (:name, :id),
    {'name': "Bob", 'id': 123}
)
```

oracledb

```
cursor.execute(
    "insert into users (name, id) values (:name, :id),
    name="Bob", id=123
)

cursor.execute(
    "insert into users (name, id) values (:name, :id),
    {'name': "Bob", 'id': 123}
)
```

executemany()

`executemany()` takes a non-query statement plus an iterable of parameter iterables (such as a list of tuples or a list of dictionaries). It will execute the query once for each item in the outer iterable.

`executemany()` may only be used for DML statements — another name for non-query statements.

```
users = [
    ('Bob', 123),
    ('Raul', 88),
    ('Lakshmi', 17),
]

cursor.executemany(
    "insert into users (name, id) values (%s, %s),",
    users
)
```

```
users = [
    {'name': 'Bob', 'id': 123},
    {'name': 'Raul', 'id': 88},
    {'name': 'Lakshmi', 'id': 17},
]

cursor.executemany(
    "insert into users (name, id) values %(name)s, %(id)s,",
    users
)
```

Table 15. Placeholders for SQL Parameters

Python package	Positional Placeholder	Named Placeholder
pymysql	%s	%(param_name)s
oracledb	:1, :2, etc or :param_name	:param_name
pyodbc	?	None
pymssql	%s or %d	%(param_name)s <i>not</i> %(param_name)d
Psycopg	%s	%(param_name)s
SQLite	?	:param_name

Table 16. Placeholder types

Type	Description	Example
qmark	Question mark	WHERE name=?
numeric	Numeric, positional style	WHERE name=:1
named	Named style	WHERE name=:name
format	ANSI C printf format codes	WHERE name=%s
pyformat	Python extended format codes	WHERE name=%(name)s



`_package_.paramstyle` contains the parameter type for the given package.

Example

db_sqlite3_parameterized.py

```
import sqlite3

# list of one-element tuples -- each tuple provides parameters for
# a SQL statement.
TERMS_TO_UPDATE = [(1,), (5,), (19,), (22,), (36,)]

PARTY_UPDATE = '''
update presidents
set party = "SURPRISE!"
where termnum = ?
''' # ? is SQLite3 placeholder for SQL statement parameter; different DBMSs use
different placeholders

PARTY_QUERY = """
select termnum, firstname, lastname, party
from presidents
where termnum = ?
"""

with sqlite3.connect("../DATA/presidents.db") as s3conn:
    s3cursor = s3conn.cursor()
    # second argument to executemany() is iterable of parameters
    # each parameter is an iterable of values to fill in the placeholders
    s3cursor.executemany(PARTY_UPDATE, TERMS_TO_UPDATE)

    for param in TERMS_TO_UPDATE:
        term = param[0]
        s3cursor.execute(PARTY_QUERY, (term,))
        print(s3cursor.fetchone())
```

db_sqlite3_parameterized.py

```
(1, 'George', 'Washington', 'SURPRISE!')
(5, 'James', 'Monroe', 'SURPRISE!')
(19, 'Rutherford Birchard', 'Hayes', 'SURPRISE!')
(22, 'Grover', 'Cleveland', 'SURPRISE!')
(36, 'Lyndon Baines', 'Johnson', 'SURPRISE!')
```

Example

db_sqlite3_restore_parties.py

```
import sqlite3

# be sure to put values in correct order for placeholders
RESTORE_DATA = [
    ('no party', 1),
    ('Democratic - Republican', 5),
    ('Republican', 19),
    ('Democratic', 22),
    ('Democratic', 36),
]

PARTY_UPDATE = '''
update presidents
set party = ?
where termnum = ?
''' # ? is SQLite3 placeholder; other DBMSs may use other placeholders

PARTY_QUERY = """
select termnum, firstname, lastname, party
from presidents
where termnum = ?
"""

with sqlite3.connect("../DATA/presidents.db") as s3conn:
    s3cursor = s3conn.cursor()
    s3cursor.executemany(PARTY_UPDATE, RESTORE_DATA)
    s3conn.commit()

    # _ is throwaway variable
    for _, termnum in RESTORE_DATA:
        s3cursor.execute(PARTY_QUERY, [termnum])
        print(s3cursor.fetchone())
```

db_sqlite3_restore_parties.py

```
(1, 'George', 'Washington', 'no party')
(5, 'James', 'Monroe', 'Democratic - Republican')
(19, 'Rutherford Birchard', 'Hayes', 'Republican')
(22, 'Grover', 'Cleveland', 'Democratic')
(36, 'Lyndon Baines', 'Johnson', 'Democratic')
```


Example

db_sqlite3_bulk_insert.py

source

```
import sqlite3
import os
import csv

DATA_FILE = '../DATA/fruit_data.csv'

DB_NAME = 'fruits.db'
DB_TABLE = 'fruits'

SQL_CREATE_TABLE = f"""
create table {DB_TABLE} (
    id integer primary key,
    name varchar(30),
    unit varchar(30),
    unitprice decimal(6, 2)
)
""" # SQL statement to create table

SQL_INSERT_ROW = f'''
insert into {DB_TABLE} (name, unit, unitprice) values (?, ?, ?)
''' # parameterized SQL statement to insert one record

SQL_SELECT_ALL = f"""
select name, unit, unitprice from {DB_TABLE}
"""

def main():
    """
    Program entry point.

    :return: None
    """
    conn, cursor = get_connection()
    create_database(cursor)
    populate_database(conn, cursor)
    read_database(cursor)

    cursor.close()
    conn.close()
```

```

def get_connection():
    """
    Get a connection to the PRODUCE database

    :return: SQLite3 connection object.
    """
    if os.path.exists(DB_NAME):
        os.remove(DB_NAME) # remove database if it exists

    conn = sqlite3.connect(DB_NAME) # connect to (new) database
    cursor = conn.cursor()
    return conn, cursor

def create_database(cursor):
    """
    Create the fruit table

    :param conn: The database connection
    :return: None
    """
    cursor.execute(SQL_CREATE_TABLE) # run SQL to create table

def populate_database(conn, cursor):
    """
    Add rows to the fruit table

    :param conn: The database connection
    :return: None
    """
    with open(DATA_FILE) as file_in:
        fruit_data = csv.reader(file_in, quoting=csv.QUOTE_NONNUMERIC)

        for row in fruit_data:
            try:
                # add a row to the table
                cursor.execute(SQL_INSERT_ROW, row)
            except sqlite3.DatabaseError as err:
                print(err)
                conn.rollback()
            else:
                # commit the inserts; without this, no data would be saved
                conn.commit()

def read_database(cursor):
    cursor.execute(SQL_SELECT_ALL)
    for name, unit, unitprice in cursor.fetchall():

```

```
print(f'{name:12s} {unitprice:5.2f}/{unit}')
```

```
if __name__ == '__main__':  
    main()
```

db_sqlite3_bulk_insert.py

pomegranate	0.99/each
cherry	2.25/pound
apricot	3.49/pound
date	1.20/pound
apple	0.55/pound
lemon	0.69/each
kiwi	0.88/each
orange	0.49/each
lime	0.49/each
watermelon	4.50/each
guava	2.88/pound
papaya	1.79/pound
fig	2.29/pound
pear	1.10/pound
banana	0.65/pound

Example

db_sqlite3_execute_many.py

```
import sqlite3
import os
import csv

DATA_FILE = '../DATA/fruit_data.csv'

DB_NAME = 'fruits.db'
DB_TABLE = 'fruits'

SQL_CREATE_TABLE = f"""
create table {DB_TABLE} (
    id integer primary key,
    name varchar(30),
    unit varchar(30),
    unitprice decimal(6, 2)
)
""" # SQL statement to create table

SQL_INSERT_ROW = f'''
insert into {DB_TABLE} (name, unit, unitprice) values (?, ?, ?)
''' # parameterized SQL statement to insert one record

SQL_SELECT_ALL = f"""
select name, unit, unitprice from {DB_TABLE}
"""

def main():
    """
    Program entry point.

    :return: None
    """
    conn, cursor = get_connection()
    create_database(cursor)
    populate_database(conn, cursor)

    # read database to confirm inserts
    read_database(cursor)

    cursor.close()
    conn.close()
```

```

def get_connection():
    """
    Get a connection to the PRODUCE database

    :return: SQLite3 connection object.
    """
    if os.path.exists(DB_NAME):
        os.remove(DB_NAME) # remove existing database if it exists

    conn = sqlite3.connect(DB_NAME) # connect to (new) database
    cursor = conn.cursor()
    return conn, cursor


def create_database(cursor):
    """
    Create the fruit table

    :param conn: The database connection
    :return: None
    """
    cursor.execute(SQL_CREATE_TABLE) # run SQL to create table


def populate_database(conn, cursor):
    """
    Add rows to the fruit table

    :param conn: The database connection
    :return: None
    """
    with open(DATA_FILE) as file_in:
        fruit_data = csv.reader(file_in, quoting=csv.QUOTE_NONNUMERIC)

        try:
            # iterate over rows of input
            # and add each row to database
            # fruit data is iterable of iterables
            # OR, iterable of dictionaries
            cursor.executemany(SQL_INSERT_ROW, fruit_data)
        except sqlite3.DatabaseError as err:
            print(err)
            conn.rollback()
        else:
            # Commit the inserts. Without this, no data
            # would be saved
            conn.commit()

```

```
def read_database(cursor):
    cursor.execute(SQL_SELECT_ALL)
    for name, unit, unitprice in cursor.fetchall():
        print(f'{name:12s} {unitprice:5.2f}/{unit}')

if __name__ == '__main__':
    main()
```

db_sqlite3_execute_many.py

```
pomegranate    0.99/each
cherry         2.25/pound
apricot        3.49/pound
date          1.20/pound
apple          0.55/pound
lemon          0.69/each
kiwi           0.88/each
orange         0.49/each
lime           0.49/each
watermelon     4.50/each
guava          2.88/pound
papaya         1.79/pound
fig            2.29/pound
pear           1.10/pound
banana         0.65/pound
```

Metadata

- **`cursor.description`** returns tuple of tuples
- Fields
 - `name`
 - `type_code`
 - `display_size`
 - `internal_size`
 - `precision`
 - `scale`
 - `null_ok`

Once a query has been executed, the cursor's `description` attribute is a tuple with metadata about the columns in the query. It contains one tuple for each column in the query, containing 7 values describing the column.

For instance, to get the names of the columns, you could say

```
names = [d[0] for d in cursor.description]
```

For non-query statements, `CURSOR.description` returns `None`.

The names are based on the query (with possible aliases), and not necessarily on the names in the table.



Not all of the fields will necessarily be populated. For instance, `sqlite3` only provides column names.

Example

db_sqlite3_metadata.py

```
"""
    Provide metadata (tables and column names) for a Sqlite3 database
"""
from pprint import pprint
import sqlite3

DB_NAME = "../DATA/presidents.db"
TABLE_QUERY = '''select * from presidents where 1 = 2'''

def main():
    cursor = connect_to_db(DB_NAME)
    show_metadata(cursor)

def connect_to_db(database_file):
    with sqlite3.connect(database_file) as s3conn:
        return s3conn.cursor()

def show_metadata(cursor):
    cursor.execute(TABLE_QUERY)
    pprint(cursor.description)
    print()
    column_names = [column_data[0] for column_data in cursor.description]
    print(f"{column_names = }")

if __name__ == '__main__':
    main()
```


db_sqlite3_metadata.py

```
(( 'termnum', None, None, None, None, None, None),
  ( 'lastname', None, None, None, None, None, None),
  ( 'firstname', None, None, None, None, None, None),
  ( 'termstart', None, None, None, None, None, None),
  ( 'termend', None, None, None, None, None, None),
  ( 'birthplace', None, None, None, None, None, None),
  ( 'birthstate', None, None, None, None, None, None),
  ( 'birthdate', None, None, None, None, None, None),
  ( 'deathdate', None, None, None, None, None, None),
  ( 'party', None, None, None, None, None, None))

column_names = ['termnum', 'lastname', 'firstname', 'termstart', 'termend', 'birthplace',
                'birthstate', 'birthdate', 'deathdate', 'party']
```

Dictionary Cursors

- Indexed by column name
- Not standardized in the DB API

Some DB packages provide dictionary cursors, which return a dictionary for each row, instead of a tuple. The keys are the names of the columns, so columns can be accessed by name rather than position.

Each package that provides a dictionary cursor has its own way of creating a dictionary cursor, although they all work the same way.



The `sqlite3` package provides a `Row` cursor, which can be indexed by position or by column name.

Table 17. Builtin Dictionary Cursors

Package	How to get a dictionary cursor
pymssql	<pre>conn = pymssql.connect (... , as_dict=True) dcur = conn.cursor() all cursors will be dict cursors</pre>
psycopg ¹²	<pre>import psycopg.extras conn = psycopg.connect(...) dcur = conn.cursor(cursor_factory=psycopg.extras.DictCursor) only this cursor will be a dict cursor</pre>
sqlite3 ¹	<pre>conn = sqlite3.connect (...) dcur = conn.cursor() dcur.row_factory = sqlite3.Row only this cursor will be a dict cursor conn = sqlite3.connect (...) conn.row_factory = sqlite3.Row dcur = conn.cursor() all cursors will be dict cursors</pre>
pymysql ¹	<pre>import pymysql.cursors conn = pymysql.connect(...) dcur = conn.cursor(pymysql.cursors.DictCursor) only this cursor will be a dict cursor conn = pymysql.connect(... , cursorclass = pymysql.cursors.DictCursor) all cursors will be dict cursors</pre>
oracledb	Not available
pyodbc	Not available
pgdb	Not available

¹ Cursor supports indexing by either key value (dict style) or integer position (list style), as well as iteration. ² Also supports `RealDictCursor` which is an actual dictionary, and `NamedTupleCursor`, which is an actual `namedtuple`

Example

db_sqlite3_dict_cursor.py

```
import sqlite3

s3conn = sqlite3.connect("../DATA/presidents.db")
# uncomment to make _all_ cursors dictionary cursors
# conn.row_factory = sqlite3.Row

NAME_QUERY = '''
    select firstname, lastname
    from presidents
    where termnum < 5
'''

cur = s3conn.cursor()

# select first name, last name from all presidents
cur.execute(NAME_QUERY)

for row in cur.fetchall():
    print(row)
print('-' * 50)

dict_cursor = s3conn.cursor() # get a normal SQLite3 cursor

# make _this_ cursor a dictionary cursor
dict_cursor.row_factory = sqlite3.Row # set the row factory to be a Row object

# Row objects are dict/list hybrids -- row[name] or row[pos]

# select first name, last name from all presidents
dict_cursor.execute(NAME_QUERY)

for row in dict_cursor:
    print(row['firstname'], row['lastname']) # index row by column name

print('-' * 50)
```

db_sqlite3_dict_cursor.py

```
('George', 'Washington')  
('John', 'Adams')  
('Thomas', 'Jefferson')  
('James', 'Madison')
```

```
-----  
George Washington  
John Adams  
Thomas Jefferson  
James Madison  
-----
```

Generic alternate cursors

- Create generator function
 - Get column names from `cursor.description()`
 - For each row
 - Make object from column names and values
 - Dictionary
 - Named tuple
 - Dataclass

For database modules that don't provide a dictionary cursor, the `dict_cursor()` function described below can be used with a cursor from any DB API-compliant package.

The example uses the metadata from the cursor to get the column names, and forms a dictionary by zipping the column names with the column values. `db_alternate_cursors` also provides `namedtuple_cursor()`, which returns each row as a named tuple, and `dataclass_cursor()`, which returns each row as an instance of a custom dataclass.

The functions in `db_alternate_cursors` return generator objects. When you loop over the generator object, each element is a dictionary, named tuple, or instance of a dataclass, depending on which function you called.

Example

db_alterate_cursors.py

```
"""
Generic functions that can be used with any DB API compliant
package.

To use, pass in a cursor after execute()-ing a
SQL query. Then iterate over the generator that is
returned
"""

from collections import namedtuple
from dataclasses import make_dataclass

def get_column_names(cursor):
    return [desc[0] for desc in cursor.description]

def dict_cursor(cursor):
    '''Generate rows as dictionaries'''
    column_names = get_column_names(cursor)
    for row in cursor.fetchall():
        row_dict = dict(zip(column_names, row))
        yield row_dict

def namedtuple_cursor(cursor):
    '''Generate rows as named tuples'''
    column_names = get_column_names(cursor)
    Row = namedtuple('Row', column_names)
    for row in cursor.fetchall():
        yield Row(*row)

def dataclass_cursor(cursor):
    '''Generate rows as dataclass instances'''
    column_names = get_column_names(cursor)
    Row = make_dataclass('row_tuple', column_names)

    for row in cursor.fetchall():
        yield Row(*row)
```

Example

db_sqlite3_alternate_cursors.py

```

"""
Generic functions that can be used with any DB API compliant
package.

To use, pass in a cursor after execute()-ing a
SQL query. Then iterate over the generator that is
returned
"""

import sqlite3
from db_alternate_cursors import *

sql_select = """
SELECT firstname, lastname, party
FROM presidents
WHERE termnum > 39
"""

conn = sqlite3.connect("../DATA/presidents.db")

cursor = conn.cursor()

cursor.execute(sql_select)

for row in dict_cursor(cursor):
    print(row['firstname'], row['lastname'], row['party'])

print('-' * 60)

cursor.execute(sql_select)

for row in namedtuple_cursor(cursor):
    print(row.firstname, row.lastname, row.party)

print('-' * 60)

cursor.execute(sql_select)

for row in dataclass_cursor(cursor):
    print(row.firstname, row.lastname, row.party)

```


db_sqlite3_alternate_cursors.py

```
Ronald Wilson Reagan Republican
George Herbert Walker Bush Republican
William Jefferson 'Bill' Clinton Democratic
George Walker Bush Republican
Barack Hussein Obama Democratic
Donald John Trump Republican
Joseph Robinette Biden Democratic
Donald John Trump Republican
```

```
-----
Ronald Wilson Reagan Republican
George Herbert Walker Bush Republican
William Jefferson 'Bill' Clinton Democratic
George Walker Bush Republican
Barack Hussein Obama Democratic
Donald John Trump Republican
Joseph Robinette Biden Democratic
Donald John Trump Republican
```

```
-----
Ronald Wilson Reagan Republican
George Herbert Walker Bush Republican
William Jefferson 'Bill' Clinton Democratic
George Walker Bush Republican
Barack Hussein Obama Democratic
Donald John Trump Republican
Joseph Robinette Biden Democratic
Donald John Trump Republican
```

Transactions

- Transactions allow safer control of updates
- `commit()` to make database changes permanent
- `rollback()` to discard changes

Sometimes a database task involves more than one change to your database (i.e., more than one SQL statement). You don't want the first SQL statement to succeed and the second to fail; this would leave your database in a corrupt state.

To be certain of data integrity, use **transactions**. This lets you make multiple changes to your database and only commit the changes if all the SQL statements were successful.

For all packages using the Python DB API, a transaction is started when you connect. At any point, you can call `CONNECTION.commit()` to save the changes, or `CONNECTION.rollback()` to discard the changes. If you don't call `commit()` after modifying a table, the data will not be saved.

Example

```
for values in list_of_tuples:
    try:
        cursor.execute(query, values)
    except SQLError:
        dbconn.rollback()
    else:
        dbconn.commit()
```

Autocommit

- Commits after every statement
- Not a best practice

Autocommit mode calls `commit()` after every non-query statement. This is not generally considered a best practice. See the table below for how autocommit is implemented in various DB packages.

Table 18. How to turn on autocommit

Package	Method/Attribute
oracledb	<code>conn.autocommit = True</code>
ibm_db_api	<code>conn.set_autocommit(True)</code>
pymysql	<code>pymysql.connect(..., autocommit=True)</code> or <code>conn.autocommit(True)</code>
psycopg	<code>conn.autocommit = True</code>
sqlite3	<code>sqlite3.connect(dbname, isolation_level=None)</code>



pymysql only supports transaction processing when using the **InnoDB** engine

Object-relational Mappers

- No SQL required
- Maps a class to a table
- All DB work is done by manipulating objects
- Most popular Python ORMs
 - SQLAlchemy
 - Django (which is a complete web framework)

An Object-relational mapper is a module or framework that creates a level of abstraction above the actual database tables and SQL queries. As the name implies, a Python class (object) is mapped to the actual table.

The two most popular Python ORMs are SQLAlchemy which is a standalone ORM, and Django ORM. Django is a comprehensive Web development framework, which provides an ORM as a subpackage. SQLAlchemy is the most fully developed package, and is the ORM used by Flask and some other Web development frameworks.

Instead of querying the database, you call a search method on an object representing a table. To add a row to the table, you create a new instance of the table class, populate it, and call a method like `save()`. You can create a large, complex database system, complete with foreign keys, composite indices, and all the other attributes near and dear to a DBA, without writing the first line of SQL.

You can use Python ORMs in two ways.

One way is to design the database with the ORM. To do this, you create a class for each table in the database, specifying the columns with predefined classes from the ORM. Then you run an ORM command which executes the queries needed to build the database. If you need to make changes, you update the class definitions, and run an ORM command to synchronize the actual DBMS to your classes.

The second way is to map tables to an existing database. You create the classes to match the schemas that have already been defined in the database. Both SQLAlchemy and the Django ORM have tools to automate this process.

NoSQL

- Non-relational database
- Document-oriented
- Can be hierarchical (nested)
- Examples
 - MongoDB
 - Cassandra
 - Redis

A current trend in data storage are called "NoSQL" or non-relational databases. These databases consist of *documents*, which are indexed, and may contain nested data.

NoSQL databases don't contain tables, and do not have relations.

While relational databases are great for tabular data, they are not as good a fit for nested data. Geo-spatial, engineering diagrams, and molecular modeling can have very complex structures. It is possible to shoehorn such data into a relational database, but a NoSQL database might work much better. Another advantage of NoSQL is that it can adapt to changing data structures, without having to rebuild tables if columns are added, deleted, or modified.

Some of the most common NoSQL database systems are MongoDB, Cassandra and Redis.

Chapter 7 Exercises

Exercise 7-1

Part A (president_sqlite.py)

For this exercise, use the SQLite database named `presidents.db` in the DATA folder. It has the following layout

Table 19. Layout of President Table

Field Name	SQLite Data Type	Python Data type	Null	Default
termnum	int(11)	int	YES	NULL
lastname	varchar(32)	str	YES	NULL
firstname	varchar(64)	str	YES	NULL
termstart	date	date	YES	NULL
termend	date	date	YES	NULL
birthplace	varchar(128)	str	YES	NULL
birthstate	varchar(32)	str	YES	NULL
birthdate	date	date	YES	NULL
deathdate	date	date	YES	NULL
party	varchar(32)	str	YES	NULL

Copy the `president.py` module in the top level folder of the student files to `president_sqlite.py` and then modify `president_sqlite.py` to get its data from the Presidents table, as described above, rather than from `presidents.txt`.

Part B (president_main.py)

Copy `president_main.py` to `president_main_sqlite.py` and modify it to import the `President` class from the `president_sqlite` module. It should import the `President` class from this new module that uses the database instead of a text file, but otherwise work the same as before.

Exercise 7-2 (add_pres_sqlite.py)

Add another president to the presidents database. Just make up the data for your new president.

SQL syntax for adding a record is

```
INSERT INTO table ("COL1-NAME",...) VALUES ("VALUE1",...)
```

To do a parameterized insert (the right way!):

```
INSERT INTO table ("COL1-NAME",...) VALUES (?,?,...)
```


Chapter 8: An Overview of Flask

Objectives

- Understand what Flask is
- Understand what Flask isn't
- Review Flask's basic architecture
- Learn some of Flask's features

What is Flask?

- Microframework for web development
- Python package
- Provides
 - URL mapping
 - Templating (Jinja2)
 - RESTful request dispatching

Flask is a Python microframework for web development.

With flask you can develop large or small web applications, as well as provide web services via REST.

Flask provides basic functionality for web development, but does not provide such features as **DBMS**
Authentication ** Form validation

The developer can use any Python packages for these features.



Flask is licensed under the BSD License.

Origins and purpose

- Written by Armin Ronacher
- Pocoo project
- Based on Werkzeug and Jinja2

Flask was created by Armin Ronacher as a Pocoo project. It is based on Werkzeug, a WSGI package, and Jinja2, a template package.

It was originally developed as a tool for creating a bulletin board system.

The creators of Flask also created Werkzeug, Jinja2, Pygments, Sphinx, and other web tools.



"Pocoo" is not a real word – Ronacher and Georg Brandl made it up

Microframework vs Framework

- Framework provides complete structure for app
- Framework makes most decisions
- Microframework provides core facilities
- Microframework does not make most decisions

Flask is a microframework. It provides core facilities for developing web applications.

The developers made a conscious decision to keep Flask small. The "micro" part means that Flask itself is small, compared to frameworks such as Django or Pyramid, not that you can't create large applications.

Django and Pyramid generate the infrastructure for web applications, and the developer fills in details in predefined scripts. Both frameworks provide ORM tools, templates, authentication, session, and other components. Although other tools can be used, it is more work to stray beyond the predefined components.

Flask does not generate any infrastructure. The developer creates scripts that use Flask as needed. Any appropriate components may be used for databases, forms, security, sessions, etc.

Views, controllers, but no models

- Flask provides views via Jinja2 templates
- Flask provides controllers via decorated functions
- Flask does not provide models

The term Model-View-Controller (MVC) is often used to describe the architecture of web applications.

The Model is the raw data that will be used to provide a response to the user. Most web apps that need models implement them as SQL (or possibly no-SQL) databases. Flask does not provide an integrated database package.

The View is the finished resource that is returned to the client (browser or REST client). Views are implemented via Jinja2 templates by default, but any template package can be used.

The Controller is the business logic that turns the Model into the View. In Flask, controllers are implemented as functions. These functions use decorators to map to routes, and return Views (content).

Extensions

- Packages designed to integrate with Flask
- Extend Flask without bloating the core

For convenience, there are many extensions to Flask that integrate useful components into flask. These extensions act as front-ends to many existing packages.

They allow Flask to remain small and flexible, while providing extensive functionality.

Some of the most popular extensions are:

- Flask-Admin
- Flask-Cache
- Flask-Login
- Flask-OAuth
- Flask-SQLAlchemy
- Flask-WTF

To see more extensions, search PyPI (<https://www.pypi.org>) for packages tagged with `Framework::Flask`.

Who's using Flask?

- Pinterest <https://www.pinterest.com/>
- Twilio <https://www.twilio.com/>
- LinkedIn <http://www.linkedin.com>

Some large web sites are implemented with Flask.

Steve Cohen, the API guy at Pinterest, says:

Since early 2012, we've reinvested in the API, putting it on equal footing with the web and since late 2012 every Pinterest client (web, iOS or Android) hits it at some point. Presently, the API does over 12 billion requests per day, all in Flask.

Flask is server-side software

- Provides
 - Routing
 - Business Logic
 - HTTP Responses
- Part of a stack with many layers

It's important to remember that Flask is a framework that provides specific tasks, including routing and HTTP responses (which can be page views or RESTful data). It is not a complete Web solution.

In addition to Flask, you will need expertise in HTML, CSS, JavaScript, as well as Python.

You will also need to be at least somewhat familiar with a WSGI-compatible web server for deployment.

If your application requires permanent data storage, you will need to understand how to use SQL with a relational database, or one of the alternatives, such as NoSQL, or some other data store.

Chapter 9: Jinja Templates

Objectives

- Understand the concept of HTML templates
- Learn the syntax of Jinja templates
- Pass variables into templates
- Use presentation logic

Using Jinja templates

- Move HTML out of view functions
- Contain variables and expressions
- Variables and expressions use { } (mostly)

Creating HTML directly from view functions leads to code that is hard to maintain, and requires changing the application to make any changes in the appearance of web pages.

A better way is to have the view function render a template, which is passed data to be filled into placeholders.

By default, Flask uses Jinja templates. These allow variables to be inserted, as well as providing a few simple control structures (if, for, while).

Templates are normally placed in a subfolder named "templates". They are basically HTML files, plus the placeholders which are specified with `{{ }}` for variables, and `{% %}` for control structures.

To pass data into a template and return the rendered HTML, use the `render_template()` function. The first argument to `render_template()` is the template name, followed by named parameters. The parameter names become the variable names that are used in the template.

Any sort of data may be passed into a template. List and dictionary elements may be accessed via `[]`, as usual. Object attributes may be accessed with `.attribute`, etc.

For convenience, dictionary keys may also be accessed with `.key`, as well as `[key]`.

Templates support inheritance. This allows you to have a base template with the HTML header and body tags, and then content templates that have just the page content. You can put site-wide CSS, JavaScript, etc., in the base template rather than having to copy it to each page-specific template.

Table 20. Jinja Variables and Expressions

Delimiter	Used for
<code>{{ ... }}</code>	Variables
<code>{% ... %}</code>	Control structures (if, while, for, ...)
<code>{# ... #}</code>	Comments
<code># ...</code> <code>## ...</code>	Line statements (only if enabled in app)

Example

flask_examples/templates/flask_hello.html

```
<!DOCTYPE html>
<html>
<head lang="en">
    <meta charset="UTF-8">
    <title></title>
</head>
<body>
<h1>Welcome, {{ username }}!</h1>
<b>Browser:</b> {{ browser }}<br/>
<b>Accept language:</b> {{ accept_language }}<br/>
</body>
</html>
```

flask_examples/flask_hello_template.py

```
from flask import Flask, render_template, request

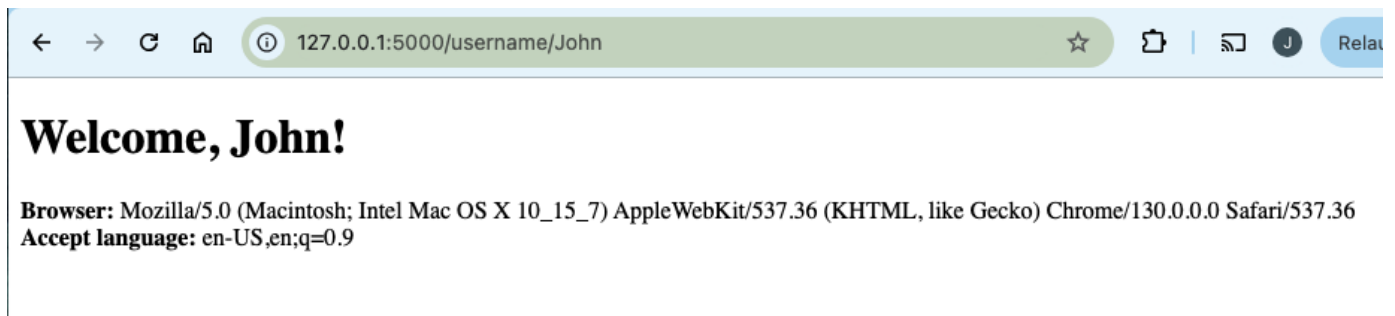
app = Flask(__name__)

@app.route('/')
def index():
    return '<h1>try /username/YOURNAME</h1>', 200

@app.route('/username/<username>')
def user_name(username):
    user_agent = request.headers.get('User-Agent')
    print(f"{request.headers = }")

    accept_language = request.headers.get('Accept-Language')
    return render_template(
        'flask_hello.html',
        browser=user_agent,
        accept_language=accept_language,
        username=username.replace('+', ' '),
    )

if __name__ == '__main__':
    app.run(debug=True)
```



flask_examples/templates/president_results.html

```

<!DOCTYPE html>
<html>
<head lang="en">
    <meta charset="UTF-8">
    <title></title>
</head>
<body>
    {% for president in presidents %}
    <h1>{{ president.first_name }} {{ president.last_name }}</h1>
    <h2>Born in: {{ president.birthplace }}, {{ president.birth_state }}</h2>
    <h2>Lived: {{ president.birth_date }} to {{ president.death_date }}</h2>
    <h2>Party: {{ president.party }}</h2>
    <hr/>
    {% endfor %}
</body>
</html>

```

Example**flask_examples/flask_presidents_template.py**

```

from flask import Flask, render_template

from president import President

app = Flask(__name__)

@app.route('/')
def index():
    """Default page for this site"""
    return '''<h1>try .../president/#</h1>'''

@app.route('/president/<int:termnum>/')
def president_by_term(termnum):
    """Retrieve president information for a specified term number"""
    term = int(termnum)
    if 0 < term < 48:
        presidents_list = [President(term)]
        return render_template('president_results.html', presidents=presidents_list)
    else:
        html_content = '<h2>Sorry, {} is not a valid term number</h2>'.format(term)

```

```
        return html_content

@app.route('/president/<last_name>/')
def president_by_last_name(last_name):
    """Retrieve president information for a specified last name;
       May return info for more than one president
    """
    html_content = ''
    presidents = []
    for i in range(1, 45):
        p = President(i)
        if p.last_name.lower() == last_name.lower():
            presidents.append(p)

    if presidents:
        return render_template('president_results.html', presidents=presidents)
    else:
        return '<h2>Sorry, {} not found</h2>'.format(last_name)

if __name__ == '__main__':
    app.run(debug=True)
```

Jinja variables and filters

- Variables specified with `{{ variable-name }}`
- Filters are functions that are applied to variables
- Filters can select or modify data from variables
- Standard Flask contexts also available

As discussed above, template variables are specified with `{{ variablename }}`. Within the delimiters, variables may be followed by one or more filters. Each filter is preceded by the pipe symbol.

Filters allow the data in a variable to be modified, or allow selection from lists and dictionaries, etc.

Flask global context variables are available without being passed in via the view functions. These will be described later in the course, and include `config`, `request`, `session`, and `g`.

Example

flask_examples/templates/vars_and_filters.html

```
<!DOCTYPE html>
<html>
<head lang="en">
  <meta charset="UTF-8">
  <title></title>
</head>
<body>
<h1>Welcome, {{ username | upper }}!</h1>
<h3>Your browser is {{ browser }}</h3>
<br/>
{{ president.first_name }} {{ president.last_name }}
<br/>
<h3>{{ fruits }}</h3>
<h3>{{ fruits | random }}</h3>
<h3>{{ fruits[0] | first }}</h3>
<h3>{{ fruits | last }}</h3>
<h3>{{ fruits | first | upper }}</h3>
<h3>{{ fruits | join('+') }}</h3>

</body>
</html>
```


flask_examples/flask_vars_and_filters.py

```
from flask import Flask, render_template, request

from president import President

app = Flask(__name__)

FRUITS = ["apple", "banana", "mango", "fig"]

@app.route('/')
def index():
    return '<h1>try /username/YOURNAME</h1>'

@app.route('/username/<username>')
def user_name(username):
    user_agent = request.headers.get('User-Agent')
    return render_template(
        'vars_and_filters.html',
        president=President(26),
        browser=user_agent,
        username=username.replace('+',' '),
        fruits=FRUITS,
    )

if __name__ == '__main__':
    app.run(debug=True)
```

Table 21. Jinja Filters

Filter	Description
<code>abs(number)</code>	Absolute value of argument
<code>attr(attrname)</code>	Get attribute of an object.
<code>batch(value, linecount, fill_with=None)</code>	Batch items. Works like slice but the other way round.
<code>capitalize(s)</code>	Capitalize a value.
<code>center(value, width=80)</code>	Center value in a field of a given width.
<code>default(value, default_value=u'')</code>	value if value is true, else default_value
<code>dictsort(value, case_sensitive=False, by=key)</code>	Sort dict into (key, value) pairs
<code>escape(s)</code>	Convert &, <, >, ', and " to HTML-safe sequences
<code>filesizeformat(value, binary=False)</code>	Format value with 'human-readable' file sizes
<code>first(seq)</code>	Return first item of sequence.
<code>float(value, default=0.0)</code>	Return float conversion or default value
<code>forceescape(value)</code>	Enforce HTML escaping
<code>format(value, *args, **kwargs)</code>	Apply string formatting to object
<code>groupby(value, attribute)</code>	Group sequence of objects by common attribute.
<code>indent(s, width=4, indentfirst=False)</code>	Return indented copy string
<code>int(value, default=0)</code>	Convert value to an integer or return default
<code>join(value, sep=u'', attribute=None)</code>	Concatenate sequence into string
<code>last(seq)</code>	Return last item of sequence.
<code>length(object)</code>	Return number of items in sequence or mapping.
<code>list(value)</code>	Convert value into list.
<code>lower(s)</code>	Convert value to lowercase.
<code>map()</code>	Apply filter to sequence of objects
<code>pprint(value, verbose=False)</code>	Pretty print a variable. Useful for debugging.
<code>random(seq)</code>	Return random item from sequence.
<code>reject(test)</code>	Filters sequence, rejecting items when test succeeds
<code>rejectattr()</code>	Filters sequence, rejecting items then test succeeds on attributes
<code>replace(s, old, new, count=None)</code>	Return copy of value with old replaced with new
<code>reverse(value)</code>	Reverse object or return reverse iterator

Filter	Description
<code>round(value, precision=0, method=common)</code>	Round number to given precision
<code>safe(value)</code>	Mark value as safe (no escaping)
<code>select()</code>	Filters sequence, keeping items when test succeeds
<code>selectattr()</code>	Filters sequence, keeping items when test succeeds on attributes
<code>slice(value, slices, fill_with=None)</code>	Slice iterator;return list of lists containing those items
<code>sort(value, reverse=False, case_sensitive=False, attribute=None)</code>	Sort an iterable
<code>string(object)</code>	Make string unicode if not already
<code>striptags(value)</code>	Strip SGML/XML tags and consolidate whitespace
<code>sum(iterable, attribute=None, start=0)</code>	Return sum of sequence of numbers
<code>title(s)</code>	Return titlecased version of value
<code>trim(value)</code>	Strip leading and trailing whitespace.
<code>truncate(s, length=255, killwords=False, end=…)</code>	Return truncated copy of string
<code>upper(s)</code>	Convert value to uppercase.
<code>urlencode(value)</code>	Escape strings for use in URLs
<code>urlize(value, trim_url_limit=None, nofollow=False, target=None)</code>	Convert URL in plain text to clickable link
<code>wordcount(s)</code>	Count words in string.
<code>wordwrap(s, width=79, break_long_words=True, wrapstring=None)</code>	Return copy of string wrapped to 79 characters

Jinja control structures

- Provide display logic in template
- Enclosed in `{% %}`
- Move some logic out of view function
- Simplified conditionals and loops
- Require end markers

It is convenient to move some logic out of the view function and into the template. For this reason, Jinja provides several control structures. These are enclosed in `{% %}`. They are similar to the same control structures in normal Python, but in Jinja an end-of-block marker is required.

The primary control structures are `for` and `if`, but there are also assignments, macro definitions,

The `for` loop provides some special variables, which as listed in the table following the code example.

Example

flask_examples/templates/control_structures.html

```
<!DOCTYPE html>
<html>
<head lang="en">
    <meta charset="UTF-8">
    <title></title>
</head>
<body>
<h1>Control Structures</h1>
<br/>
<ul>
    {% for fruit in fruits %}
    {% if fruit | length > 5 %}
    <li><i>{{ fruit }}</i></li>
    {% else %}
    <li>{{ fruit }}</li>
    {% endif %}
    {% endfor %}
</ul>

<br/>
{% if number > 25 %}
<h3>SPAM!</h3>
{% else %}
<h3>HAM!</h3>
{% endif %}

</body>
</html>
```

flask_examples/jinja_control_structs.py

```
from random import randint
from flask import Flask, render_template, request

app = Flask(__name__)

FRUITS = ["apple", "banana", "mango", "fig", "muskmelon"]

@app.route('/')
def index():
    return render_template(
```

```
        'control_structures.html',  
        fruits=FRUITS,  
        number=randint(1, 50),  
    )  
  
if __name__ == '__main__':  
    app.run(debug=True)
```

Jinja inheritance

- One template can extend (inherit content from) another
- Useful for consistent looks across an app or site
- Use the `{% block BLOCKNAME %}` control structure

Jinja supports inheritance, which means that one template can use content from another.

Typically one template is a base template, that has the HTML header with common information, and a minimal `<BODY>` tag. Other templates can format data in different ways, but inherit from the base, so that all templates get the same CSS, JavaScript, etc.

Of course, each non-base template can also be customized.

To implement inheritance, first create a base template, and put an empty block in the body, where you would like the content. This block can be named anything, but is typically named "content".

In each non-base template put a Jinja extends block at the top (it must be first). Then, define a block with the same name as the empty block in the base template, but this time containing HTML. The empty block in the base template will be overwritten with the content from the non-base template.

A base template can have more than one block defined, for more complex layouts such as multi-column.

Example

flask_examples/templates/president_base.html

```
<!DOCTYPE html>
<html>
<head lang="en">
    <meta charset="UTF-8">
    <title>{% block title %}{% endblock %}</title>
</head>
<body>
<h1>President Information</h1>
<hr/>
{% block content %}{% endblock %}
</body>
</html>
```

flask_examples/templates/president_detail.html

```
{% extends "president_base.html" %}
{% block title %}
<h2>President Detail</h2>
{% endblock %}
{% block content %}
    {% for president in presidents %}
    <h1>{{ president.first_name }} {{ president.last_name }}</h1>
    <h2>Born in: {{ president.birthplace }}, {{ president.birth_state }}</h2>
    <h2>Lived: {{ president.birth_date }} to {{ president.death_date }}</h2>
    <h2>Party: {{ president.party }}</h2>
    <hr/>
    {% endfor %}
{% endblock %}
```

flask_examples/templates/president_list.html

```
{% extends "president_base.html" %}
{% block content %}
    {% for president in presidents %}
    <h3>{{ president.first_name }} {{ president.last_name }}</h3>
    {% endfor %}
{% endblock %}
```


Example

flask_examples/jinja2_inheritance.py

```
from jinja2 import Environment, FileSystemLoader
from president import President

env = Environment(
    loader = FileSystemLoader(searchpath="templates")
)

list_template = env.get_template("president_list.html")
detail_template = env.get_template("president_detail.html")

presidents = [President(i) for i in range(1, 47)] # get list of presidents

president_list = list_template.render(title="PRESIDENT LIST", presidents=presidents)
print(president_list)
print('-' * 60)

president_details = detail_template.render(title="PRESIDENT DETAILS:",
presidents=presidents)
print(president_details)
```

Adding Bootstrap to templates

- Add to base template for convenience
- Easy to add a nav bar

The Flask-Bootstrap extension makes it easy to incorporate the Bootstrap CSS extensions in your templates.

To use, import Bootstrap() from flask_bootstrap and pass your app to it.

Then you can extend "bootstrap/base.html" in your templates. It's convenient to add Bootstrap and a nav bar to your base template.

flask_examples/templates/president_base_bs.html

```
{% extends "bootstrap/base.html" %}
<!DOCTYPE html>
<html>
<head lang="en">
    <meta charset="UTF-8">
    <title>{% block title %}{% endblock %}</title>
</head>
<body>
{% block navbar %}
<div class="navbar navbar-inverse" role="navigation">
    <div class="container">
        <div class="navbar-header">
            <button type="button" class="navbar-toggle"
            data-toggle="collapse" data-target=".navbar-collapse">
                <span class="sr-only">Toggle navigation</span>
                <span class="icon-bar"></span>
                <span class="icon-bar"></span>
                <span class="icon-bar"></span>
            </button>
            <a class="navbar-brand" href="/">Bootstrap DEMO</a>
        </div>
        <div class="navbar-collapse collapse">
            <ul class="nav navbar-nav">
                <li><a href="/">Home</a></li>
                <li><a href="/presidents">Presidents</a></li>
            </ul>
        </div>
    </div>
</div>
{% endblock %}
```

```
<h1>President Information</h1>
<hr/>
{% block content %}{% endblock %}
</body>
</html>
```

flask_examples/templates/president_list_bs.html

```
{% extends "president_base_bs.html" %}
{% block content %}
    {% for president in presidents %}
        <h3>{{ president.first_name }} {{ president.last_name }}</h3>
    {% endfor %}
    <a href="{{ url_for('index') }}">Return to main page</a>
{% endblock %}
```

Example

flask_examples/templates/bootstrap_hello.html

```
{% extends "president_base_bs.html" %}
<!DOCTYPE html>
<html>
<head lang="en">
    <meta charset="UTF-8">
    <title>Bootstrap Demo</title>
</head>
<body>
{% block content %}
<h1>Control Structures</h1>
<br/>
<ul>
    {% for fruit in fruits %}
    {% if fruit | length > 5 %}
    <li><i>{{ fruit }}</i></li>
    {% else %}
    <li>{{ fruit }}</li>
    {% endif %}
    {% endfor %}
</ul>

<br/>
{% if number > 25 %}
<h3>SPAM!</h3>
{% else %}
<h3>HAM!</h3>
{% endif %}
{% endblock %}
</body>
</html>
```

flask_examples/flask_bootstrap_hello.py

```
from random import randint

from flask_bootstrap import Bootstrap
from flask import Flask, render_template

from president import President

app = Flask(__name__)
Bootstrap(app)
```

```
FRUITS = ["apple", "banana", "mango", "fig", "muskmelon"]

@app.route('/')
def index():
    return render_template(
        'bootstrap_hello.html',
        fruits=FRUITS,
        number=randint(1, 50),
    )

@app.route('/presidents/')
def list_presidents():
    """Return list of all presidents"""
    presidents = []
    for i in range(1, 46):
        presidents.append(President(i))
    return render_template('president_list_bs.html', presidents=presidents)

if __name__ == '__main__':
    app.run(debug=True)
```

Macros

- Like functions
- Executed by Jinja
- Defined in {% %}, but called in {{ }}
- May be imported

Jinja supports macros, which are the Jinja equivalent of Python functions. A macro is defined in a control block.

The control block of a macro defines the macro name and the list of parameters (if any).

The body of the macro can contain any normal HTML page content, as well as Jinja directives.

To use the macro, "call" it inside normal variable directives ({{ }}).

To put your macros in a separate file, use the {% import %} directive.

Macros do not have access to the variables of the current template.

flask_examples/templates/macros.html

```
{% macro page_title(text="Title") %}  
<div class="row">  
    <h2>{{ text }}</h2>  
</div>  
{% endmacro %}
```

flask_examples/templates/using_macros.html

```
{% import 'macros.html' as macros %}  
  
<!DOCTYPE html>  
<html lang="en">  
<head>  
    <meta charset="UTF-8">  
    <title></title>  
</head>  
<body>  
    {{ macros.page_title("This is the page title via the page_title() macro") }}  
</body>  
</html>
```

Example

flask_examples/flask_using_macros.py

```
from flask import Flask, render_template

app = Flask(__name__)

@app.route('/')
def index():
    return render_template(
        'using_macros.html',
    )

if __name__ == '__main__':
    app.run(debug=True)
```

Chapter 9 Exercises

Exercise 9-1 (flask_powers_of_two_bs.py)

Redo the `flask_powers_of_two.py` exercise, using a template and Bootstrap to display the data.

HINT: Use the `<sup></sup>` tags for superscript.

Extra topics

The following topics will be covered at the discretion of the instructor if there is enough time.

Chapter 10: Flask REST Overview

Objectives

- Learn what REST means
- Implement a REST API from scratch (IOW, the "hard way")
- Write view functions to respond to REST requests

What is REST?

- **REpresentational State Transfer**
- Architectural style, not protocol
- Programmatic access to data via web server
- Replaces older protocols (e.g. SOAP)

REST is an architectural style for providing web-based services. It is not a protocol. It replaces older styles, such as RPC and SOAP.

REST specifies six constraints which must be met in order to be called RESTful. Since it is a guideline and not a protocol, all requirements do not necessarily have to be met.

The commonly recognizable part of REST is that typical REST servers are accessed by sending URL path information in the request, rather than URL queries.

In other words, a traditional request might look like <http://yourserver.com/presidents?term=26> The REST equivalent might look like <http://yourserver.com/presidents/term/26> or just <http://yourserver.com/presidents/26>

Your application's RESTful API is defined in terms of such URLs.

Base REST constraints

The base constraints of a web-based REST system are:

- Client-server architecture via HTTP
- Stateless communication (each request is independent of the last)
- Cacheable
- Layered system
- Code on demand (optional)
- Uniform interface
 - Hypertext (HTML) links to reference state (i.e., retrieve data)
 - Hypertext links to get related resource
 - Standard media types (typically JSON, but sometimes XML or others)

Designing a REST API

- It's all about the URL
- Include version in paths
- Human-readable

When designing a RESTful API, it's all about the URL. All communication from the client to the server is done in terms of URL requests. This includes queries (GET) as well as updates (POST, PUT) and deletes (DELETE).

```
http://myserver.com/president/26
```

URLs in your API should be as short as possible while remaining descriptive and human-readable.

Example

```
http://myserver.com/president  
http://myserver.com/president/26  
http://myserver.com/president/roosevelt
```

If the service allows data to be modified, rather than reflect this in the URL, it is controlled by HTTP verbs. Instead of providing deletion with

```
http://myserver.com/president/delete/26
```

provide it with the same URL as the query, but with the client providing a DELETE request, rather than GET:

```
http://myserver.com/president/26
```

Here is a good presentation on the design of APIs:

<https://speakerdeck.com/dzuelke/designing-http-interfaces-and-restful-web-services-sfliveparis2012-2012-06-08>

Using jsonify

- jsonify() creates JSON suitable for the response
 - Pass keyword arguments which becomes keys in JSON dictionary
 - Pass dictionary

flask.jsonify() returns a JSON string created from named parameters or a dictionary.

To add an HTTP status code to your JSON response you can use make_response:

Example

```
from flask import jsonify, make_response

@app.route('/api/wombat')
def post(request):
    # do work ...
    d = make_new_record() # user-defined
    return make_response(jsonify(d), 201)
```

Setting up RESTful routes

- Use view functions
- Parameter types specified with converters
- Path information passed into view functions

To set up a RESTful URL, create a view function as usual. In the route string, enclose one or more parameters in angle brackets, as necessary.

```
/president/<int:term>
```

The word before the colon is a converter, and is used to parse a data type from the path information.

The view function's parameter names must match the names in the route:

```
def president_by_term(term):  
    pass
```

There can be any number of path parameters in a URL.

Typically, the URL just has the name of the resource in the singular. All of the operations depend on which HTTP request type was issued, and whether an ID was specified.



The **Thunder Client** extension for VS Code makes it easy to send HTTP requests with any of the HTTP request types and view the responses.

Example

flask_examples/flask_rest_simple.py

```
from datetime import date
from flask import Flask, request, jsonify
from flask_bootstrap import Bootstrap

from president import President

app = Flask(__name__)
Bootstrap(app)

@app.route('/president/')
def president_list():
    """Return list of all presidents"""
    presidents = get_presidents()
    presidents_as_dicts = [pres_to_dict(p) for p in presidents]
    response = jsonify(presidents=presidents_as_dicts)
    return response

@app.route('/president/<int:term>/')
def president(term):
    return jsonify(pres_to_dict(President(term)))

def pres_to_dict(president):
    """Convert president object to a dictionary"""
    pres_dict = president.__dict__
    return {field_name.removeprefix('_'): value for field_name, value in
pres_dict.items()}

def get_presidents():
    presidents = []
    for i in range(1, 46):
        presidents.append(President(i))
    return presidents

if __name__ == '__main__':
    app.run(debug=True)
```


Table 22. RESTful requests

Verb	URL	Description
GET	/api/wombat	Get list of all wombats
POST	/api/wombat	Create a new wombat
GET	/api/wombat/{id}	Get details of wombat by id
PUT	/api/wombat/{id}	Replace all fields of wombat by id
PATCH	/api/wombat/{id}	Update some fields of wombat by id
DELETE	/api/wombat/{id}	Delete wombat by id

Choosing the return type

- Return type is typically JSON
- Other types requested via HTTP header
- Put logic in view function, or use decorator

RESTful services typically return data as JSON (JavaScript Object Notation). However, the client may request another data type via the HTTP ACCEPT header.

You can decide to provide alternate data types, or just return an error status if that data type is not available.

Example

flask_examples/flask_rest_response.py

```
#
from datetime import date
import xml.etree.ElementTree as ET
from flask import Flask, request, render_template, jsonify
from flask_bootstrap import Bootstrap

from president import President

app = Flask(__name__)
Bootstrap(app)

def pres_to_dict(pres):
    """Convert one president object to a dictionary"""
    pres_dict = {}
    for prop_name in dir(pres):
        if not prop_name.startswith('_'):
            prop_value = getattr(pres, prop_name)
            if isinstance(prop_value, date):
                prop_value = '{0.year:4d}-{0.month:02d}-{0.day:02d}'.format(prop_value)
            pres_dict[prop_name] = prop_value
    return pres_dict

def pres_list_to_xml(pres_list):
    """Convert list of presidents to XML"""
    root_element = ET.Element('presidents')
    for pres in pres_list:
```

```

        pres_element = ET.Element('president')
        fname = ET.Element('firstname')
        fname.text = pres.first_name
        pres_element.append(fname)
        lname = ET.Element('lastname')
        lname.text = pres.last_name
        pres_element.append(lname)
        root_element.append(pres_element)
    return ET.tostring(root_element)

@app.route('/president')
def index():
    """Main page; returns list of all presidents"""
    presidents = []
    for i in range(1, 46):
        presidents.append(President(i))
    accept_type = request.headers.get('ACCEPT')
    response = ''
    print("Accept type:", accept_type)
    if accept_type.startswith('text/html'):
        response = render_template('president_list_bs.html', presidents=presidents)
    elif accept_type.startswith('application/xml'):
        response = pres_list_to_xml(presidents)
    elif accept_type == 'application/json':
        presidents_as_dicts = [pres_to_dict(p) for p in presidents]
        response = jsonify(presidents=presidents_as_dicts)
    # handle error here if non-expected type

    return response

if __name__ == '__main__':
    app.run(debug=True)

```

Handling invalid requests

- Use `app.errorhandler()`

Flask's builtin exceptions typically return human-friendly HTML. This is great for humans, but not so great for API clients.

The solution is to define a custom exception type and install an error handler for it.

Example

```
class InvalidAPIRequest(Exception):
    status_code = 400

    def __init__(self, message, status_code=None, data=None):
        Exception.__init__(self)
        self.message = message
        if status_code is not None:
            self.status_code = status_code
        self._data = data

    def to_dict(self):
        return_value = dict(self.data or ())
        return_value['message'] = self.message
        return return_value

@app.errorhandler(InvalidAPIRequest)
def handle_invalid_usage(error):
    response = jsonify(error.to_dict())
    response.status_code = error.status_code
    return response

@app.route('/president/<int:term>')
def get_president(term):
    if term < 1 or term > 44:
        raise InvalidUsage('Term out of range', status_code=410)
```

Receiving data from POST

- Use the request.form object
- form is a dictionary of fields

To receive data from a form, use a route that specifies a POST request, and use the request.form object to retrieve specific fields.

Example

```
@app.route('/president', methods=['POST'])
def add_president():
    first_name = request.form['firstname']
    last_name = request.form['lastname']
```

Handling a DELETE request

- No special URL
- Use route with DELETE method

To handle a delete request, use a route that specifies the DELETE method.

```
@app.route('/president/<int:term>', methods=['DELETE'])
def delete_president(term):
    pass    # delete president from database here...
```

Helper modules

- REST modules
 - flask-RESTful
 - flask-restplus
 - flask-SQLAlchemy
- Automate tasks from this chapter

At this point, you've learned how to implement a REST API in Flask the hard way.

In the chapter on Flask-RESTful, you'll see how to take advantage of one of the helper modules available that automate resources, routing, and marshalling of data.

The standard REST module for Flask is **flask-RESTful**, but several others are available.

flask-SQLAlchemy is great if you are building a database from scratch. It's a little trickier to use if you're using an existing database.



The fact that Flask is unopinionated is great, usually. When it comes to building an application, you get to make some decisions. Flask-RESTPlus adds Swagger API documentation, which is great. Flask-Marshmallow adds powerful validation of incoming data, which is also great. Unfortunately, those two modules are not specifically designed to work together. Unfortunately, at the time of writing, there is no extension which nicely combines all the parts needed to make a great Flask-based API. That doesn't mean you can't do it — it just means you have to work a little harder to make some tools work together.

Chapter 10 Exercises

Exercise 10-1 (powers_of_two_rest.py)

Rewrite the original flask_powers_of_two.py app to provide the powers of two as JSON, in response to the `/powers` URL.

Use Thunder Client to test. Alternatively, you could use `curl` or any browser.

Chapter 11: Flask RESTful

Objectives

- Create RESTful APIs with Flask-RESTX
 - Serialize data
 - Use Resource objects
 - Return JSON results
 - Handle errors

About Flask extensions

- Install with pip
- Add functionality to flask
- Need to be initialized

Flask advertises itself as a *microframework*. They mean that it is complete, but does not provide a lot of "scaffolding" (infrastructure) out of the box. This lets Flask be productive right out of the box without a lot of extra code getting in your way.

It also means that there's a lot you have to do "by hand" if you just use the flask module by itself.

Flask has many extensions that can be installed and enabled to take care of everyday web app and API needs. (Some extensions come with Flask and do not need to be installed). In fact, **PyPi** lists over 10,000 extensions tagged with "Framework::Flask".



Django is a full framework — it provides a complete scaffolding for a project with one or more apps. You edit the generated scripts and add in your views, etc.

Table 23. Popular Flask Extensions

Extension name	Module name	Description
REST-related		
Flask RESTX	Flask-RESTX	Support for creating RESTful APIs
Flask SQLAlchemy	Flask-SQLAlchemy	Support for accessing databases from REST APIs
Flask Marshmallow	Flask-Marshmallow	Tools for marshalling (serializing) data into JSON or other formats
Flask JWT	Flask-JWT-Extended	Provide API security via JSON Web Tokens
Flask Admin	flask-admin	Add admin interfaces for databases
Other		
Flask Debug Toolbar	flask-debugtoolbar	A Django-like popup toolbar for debugging Web apps
Flask Mail	flask-mail	Provides SMTP interface
Flask WTF	flask-wtf	Support WTF forms
Flask Login	Flask-Login	Implement user-level security
Flask Limiter	Flask=limiter	Control traffic rates
Flask Blueprints	flask-blueprints	Create reusable apps



The module name is what you actually install with **pip**.

Using Flask-RESTX

- Simplifies APIs
- Adds routes
- Provides *Resources*
- Serializes data

Flask-RESTX is a Flask extension that makes it easy to create APIs. The main thing it provides is *resources*. A resource is a class that inherits from the **Resource** base class. It simplifies creation of API views.

Resources are mapped to routes (URLS) within your API.

Flask-RESTX also provides tools for serializing data and for parsing HTTP request parameters and data.

To use Flask-RESTX, import **Api** from the flask_restx module.

Example

```
from flask import Flask
from flask_restx import Api, Resource

app = Flask(__name__) # create the Flask app
api = Api(app) # initialize the Api extension
```



The name used to install the module with pip is **Flask-RESTX**, but the name to import is **flask_restx**.

Creating resources

- Supports HTTP access to API
- Formats HTTP response
- Class returns dict or object
- Return value JSONified

To create a resource, create a class that inherits from **flask_restx.Resource**.

Within the resource class, define methods for the desired HTTP request types (**GET**, **POST**, **PUT**, **DELETE**, or **PATCH**). Whatever those methods return is converted to JSON and used to construct an HTTP response object, with a default status code of 200. You can specify a different status code if needed.

Typically for a given API resource, you will create two resources.

One is for the route with the unique resource id:

```
.../api/presidents/26
```

This is used to retrieve a particular resource with GET, or to update/delete a resource with PUT, PATCH, or DELETE. It should be named like the resource, such as **Presidents**.

The other resource is for the route without the unique resource id:

```
.../api/presidents  
.../api/presidents?party=whig
```

This is used for retrieving a (possibly filtered) list of all resources via GET, or for creating a new resource via POST.

You may want to name resources *ResourceNameResource*, but that is not required. It may help avoid name collisions with other names in your API, such as database objects.

Example

flask_examples/fr_minimal_api.py

```
from flask import Flask
from flask_restful import Api, Resource

app = Flask(__name__) # Create instance of Flask
api = Api(app) # Initialize Flask-RESTful extension

class Student(Resource): # Create resource as class
    def get(self, id): # Define method to handle HTTP GET requests
        return {'id': id, 'name': 'Jane Student'} # Return data (will be converted to
JSON response with status code 200)

# Configure route within API
# GET requests to URL /api/student will call Student.get
api.add_resource(Student, '/api/student/<int:id>')

if __name__ == '__main__':
    app.run(debug=True) # Launch development server with Flask app
```

Adding resources

- Use `api.add_resource()`
- Resource class tied to URL

Use `api.add_resource(){empty}` to register resources with routes in your API. The typical route begins with `__hostname__/api/__resourcename__`, but you can use any URLs you like.

For resources that require a parameter, usually a unique integer ID, you can specify the parameter in angle brackets. By default, the parameter can be any type, but you can specify a type, and the parameter will be converted to that type.

Most of the time there will be two resource classes per resource—one with, and one without, parameters.

Advanced usage:

You can pass multiple URLs to `.add_resource()` and they will all be routed to the resource.



You can add the named parameter `endpoint` to a resource, so you can use it to generate a URL back to a record.

```
api.add_resource(Student, '/api/student', '/api/auditor',
                  endpoint="Student")
```

Example routes

```
api.add_resource(PresidentListResource, '/api/president')
api.add_resource(PresidentResource, '/api/president/<int:termnum>')
api.add_resource(StateResource, '/api/state/<string:abbr>')
api.add_resource(CelsiusResource, '/api/celsius/<float:celsius>')
```

Table 24. Route Converter Types

Type	Description
string	Accepts any text (but not slashes). This is the default.
int	Accepts integers.
float	Accepts floats or integers.
path	Like string but accepts slashes.

Example

flask_examples/fr_add_resources.py

```
import json
from flask import Flask
from flask_restful import Api, Resource

app = Flask(__name__)
api = Api(app)

STUDENTS = [
    'Melinda Gates',
    'Steve Jobs',
    'Larry Wall',
    'Paul Allen',
    'Larry Ellison',
    'Bill Gates',
    'Mark Zuckerberg',
    'Sergey Brin',
    'Larry Page',
    'Linus Torvalds',
]

class Teacher(Resource):
    def get(self):
        return {'name': 'Dennis Ritchie'}

class Student(Resource):
    def get(self, id):
        return {'id': id, 'name': STUDENTS[id]}

class StudentList(Resource):
    def get(self):
        return {'students': [i: s for i, s in enumerate(STUDENTS)]}

api.add_resource(Teacher, '/api/teacher')
api.add_resource(Student, '/api/student/<int:id>')
api.add_resource(StudentList, '/api/student')

if __name__ == '__main__':
    app.run(debug=True)
```


Trying out the API

- Use HTTP client
 - Browser
 - Thunderclient
 - curl
 - Python script
- Use URL as specified in resource routes
- Data returned is JSON

There are several ways to access the API. Of course you can use a browser for a simple GET request.

For POST, PUT, PATCH, or DELETE requests, use **Thunderclient**, **curl**, or a custom Python script using the **requests** module.

Using curl

curl is a command line HTTP client, available from <https://curl.haxx.se/download.html>

It can execute any HTTP request, sending parameters, data, and headers as needed.

```
curl 127.0.0.1:5000/api/teacher
```

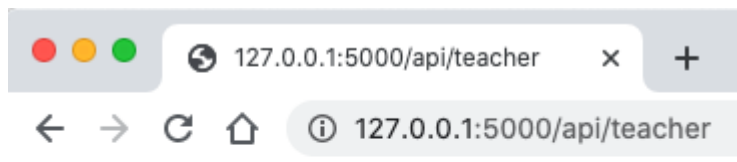
```
{  
  "name": "Dennis Ritchie"  
}
```

```
curl 127.0.0.1:5000/api/students/1
```

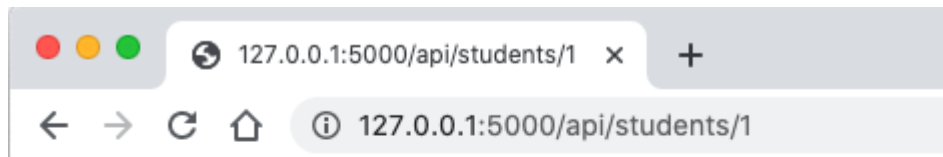
```
{  
  "name": "Steve Jobs"  
}
```

The above **curl** commands (in bold) can be typed in at Windows command prompt or in a Mac/Linux terminal window.

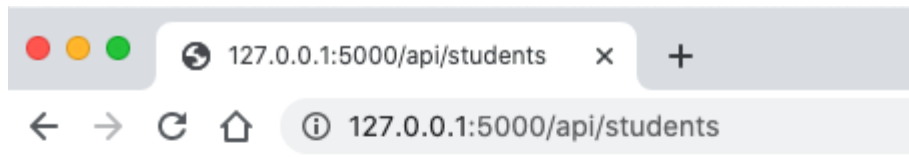
Using a Browser



```
{  "name": "Dennis Ritchie"}
```



```
{  "name": "Steve Jobs"}
```



```
{  "students": {    "0": "Melinda Gates",    "1": "Steve Jobs",    "2": "Larry Wall",    "3": "Paul Allen",    "4": "Larry Ellison",    "5": "Bill Gates",    "6": "Mark Zuckerberg",    "7": "Sergey Brin",    "8": "Larry Page",    "9": "Linus Torvalds"  }}
```

Using Thunderclient

Thunderclient is a VS Code extension for testing APIs.

With Thunderclient, you can execute any kind of HTTP request, and save any number of requests for replay.

Using a Python script

flask_examples/fr_consume_students_api.py

```
import requests

TEACHER_URL = "http://localhost:5000/api/teacher" # Create URLS for API resources
STUDENT_URL = "http://localhost:5000/api/student/"

response = requests.get(TEACHER_URL) # Make the HTTP request to the app
if response.status_code == requests.codes.OK: # Check that the HTTP status code is 200
    (OK)
    print(response.text) # Get the returned JSON as a binary string (bytes object) and
    decode to Python string
    print('=' * 60)

for i in range(10):
    url = f"{STUDENT_URL}{i}" # Construct URL with unique IDs
    response = requests.get(url)
    if response.status_code == requests.codes.OK:
        print(response.text)
    print('-' * 60)
```

Serializing data

- AKA Marshalling
- Specify fields to expose
- Transform fields as needed

To make it simple to provide the data that is rendered to API clients, Flask-RESTX has the **fields** module.

Use this module to specify what fields to expose from a particular object. You can also add fields that were not in the original, or combine values from multiple sources.

Short version: make a list of fields, using `flask_restx.fields`, then apply the **@marshall_with** decorator to resource methods with the field list.

The field list is a dictionary, where the keys are the names you want in the JSON output, and the values are objects defined in the **flask_restx.fields** module.

From any resource method, return a dictionary or an object with attributes. The **@marshall_with** decorator will copy the elements of the dictionary, or the attributes of the object, whose names match the field names, into a JSON string. If you provide the **envelope** argument to **@marshall_with**, the data will be wrapped in a JSON object where the key is the envelope value.

To render a field with a different name than the actual attribute or key name, add the **attribute** argument to specify the actual field name.

Another thing you can provide while marshalling is to add the URL for a given resource to the resource itself. Thus, when the client gets a list of resource, each resource could have the URL for getting that individual resource. To do this, add a **Url** field to the field list, specifying the name of the resource.

Example

flask_examples/fr_serialize_dict.py

```
import json
from flask import Flask
from flask_restful import fields, Resource, marshal_with, Api, reqparse

app = Flask(__name__)
api = Api(app)

president_fields = { # Define fields to be marshalled. Anything can be added here.
    # Select fields to be exposed in API.
    'termnum': fields.Integer,
    'firstname': fields.String,
    'lastname': fields.String,
    'birth_state': fields.String(attribute="birthstate"),
    'party': fields.String,
    'url': fields.Url('presidents'), # Get the URL for this resource
}

with open('presidents.json') as pres_in: # Read presidents data from a JSON file
    PRESIDENTS = json.load(pres_in)

president_count = len(PRESIDENTS['presidents']) # Get the number of presidents for later
validation

error_data = {'error': 'invalid term number'} # Define a default error payload

def invalid_term_number(term_number): # Function to determine whether term number is
valid
    return not (1 <= term_number <= (president_count + 1))

class InvalidAPIRequest(Exception): # New exception type to be raised in case of error

    def __init__(self, message, status_code=400, data=None):
        Exception.__init__(self)
        self.message = message
        self.status_code = status_code
        self._data = data

    def to_dict(self):
        return_value = dict(self.data or ())
        return_value['message'] = self.message
        return return_value

class Presidents(Resource): # Resource which expects unique ID (for GET, PUT, PATCH, and
```

```

DELETE only)
"""
    Handle GET, PUT, PATCH, and DELETE posts.

    GET is only handled if a record number is specified.
"""

    @marshal_with(president_fields, envelope='president') # Use the defined field list,
    president_fields, to control what gets returned; wrap data in a JSON object whose key is
    'president'
    def get(self, termnum): # Term number is passed into GET handler from URL
        """Retrieve one record"""
        if invalid_term_number(termnum): # Check for invalid term number
            raise ValueError(f"Invalid term number: {termnum}")

        p = PRESIDENTS['presidents'][termnum - 1]
        return p # Return dictionary of presidents; it is marshalled (serialized) into a
        dictionary (and later converted to JSON)

    def put(self, termnum): # Implement PUT handler
        """Replace one record"""
        if invalid_term_number(termnum):
            raise ValueError("Invalid term number")
        return {}

    def delete(self, termnum):
        """Delete one record"""
        if invalid_term_number(termnum):
            raise ValueError("Invalid term number")
        return {}

class PresidentsList(Resource): # Resource which does __not__ expect unique ID (for GET
and POST only)
    """
    Handle GET and POST requests.

    GET is only handled if no ID is specified.
    """

    @marshal_with(president_fields, envelope='presidents') # Use field list for each
    resource
    def get(self):
        """List all records"""
        presidents = PRESIDENTS['presidents']
        return presidents

    @marshal_with(president_fields, envelope='president') # POST handler should return
    newly posted data using field list

```

```

def post(self):
    """Add a new record"""
    parser = reqparse.RequestParser() # Create new resource (as a dictionary here --
    IRL would usually create a database record via ORM)
    parser.add_argument('termnum', type=int)
    parser.add_argument('firstname', type=str)
    parser.add_argument('lastname', type=str)
    parser.add_argument('birthstate', type=str)
    parser.add_argument('party', type=str)
    args = parser.parse_args()

    new_president = dict( # Add new resource to dict (again, would add to database)
        termnum=args['termnum'],
        firstname=args['firstname'],
        lastname=args['lastname'],
        birthstate=args['birthstate'],
        party=args['party'],
    )

    PRESIDENTS['presidents'].append(new_president) # Return marshalled data and HTTP
    status code

    return new_president, 201 # HTTP status 201 means "record added"

api.add_resource(PresidentsList, '/api/presidents')
api.add_resource(Presidents, '/api/presidents/<int:termnum>')

if __name__ == '__main__':
    app.run(debug=True)

```

About SQLAlchemy

- ORM for Python
- Maps classes to DB Tables
- Simplifies Web apps and APIs

SQLAlchemy is an ORM (Object-Relational Mapper) for Python. It provides a layer of abstraction between your Python code and a database. Tables are represented as Python objects, and all queries and modifications to the database are made via methods on the objects. Nearly any database can be used. We'll implement SQLite3 in the examples in this course, since it is supported in the standard library.

There are a couple of ways to use SQLAlchemy, depending on whether you're defining the database schema (table layout) in Python or using an existing database.

No matter which way you start, the first thing to do is to create a database *engine*, which is created via a URI-like specification for the database.

To start from scratch, with no existing tables, you define *models*, which are classes that define what you want in the table.

To use an existing database, create an automap base, using the **automap_base()** function from **sqlalchemy.ext.automap**. Initialize this base with the database engine and the **reflect** argument. This will let SQLAlchemy get the details of the table from the database.

Once the automap base is prepared, access models (i.e., tables) by *base.classes.tablename*.

One more thing to do is to create a session. A session is essentially a database connection. You can use the session to make changes to the database, and to commit the changes.

Because of multitasking design of Flask, you need to create a *scoped* session. Otherwise your app will have threading issues.

To retrieve data, use the **query()** method on the session with the object name. The result of **query()** is a query object. From there, you can call **all()** to get all records (like `select * from TABLE`), **filter()** to select records (like `select * from TABLE where CONDITION`), or **first()** to get the first record.

These functions return a list of individual data objects (ie, rows).

```
president_list = session.query(President).all()
president = session.query(President).filter(President.termnum == 3).one()
```


Table 25. SQLAlchemy Query Functions

<code>all()</code>	Returns all the objects in the query
<code>count()</code>	Returns the total number of rows of a query
<code>filter()</code>	Filters the query by applying a criteria
<code>delete()</code>	Removes from the database the rows matched by a query
<code>distinct()</code>	Applies a distinct statement to a query
<code>exists()</code>	Adds an exists operator to a sub-query
<code>first()</code>	Returns the first row in a query
<code>get()</code>	Returns the row referenced by the primary key parameter passed as argument
<code>join()</code>	Creates a SQL join in a query
<code>limit()</code>	Limits the number of rows returned by a query
<code>order_by()</code>	Sets an order in the rows returned by a query



see <https://docs.sqlalchemy.org/en/14/> for complete SQLAlchemy documentation.

Using SQLAlchemy in resources

- Get data from URL/params
- Query DB objects
- Marshall data from objects

You can use the methods in the previous topic to access data, then provide field lists to specify how the data will be rendered. The field lists will use the names of the columns (which are the attributes of the data objects).

A resource method using the **@marshall_with** decorator can just return the data object.

It's not a bad idea to put queries in **try-except** in case something goes wrong.

Example

flask_examples/fr_president_api.py

```
import os
from flask import Flask, jsonify, request
from flask_restful import fields as fr_fields, Resource, marshal_with, Api
from sqlalchemy import create_engine, select
from sqlalchemy.ext.automap import automap_base
from sqlalchemy.orm import scoped_session, sessionmaker
from marshmallow import Schema, fields as ma_fields

app = Flask(__name__)
api = Api(app)

DATABASE_CONNECTION_STRING = 'sqlite:///presidents.db'
# DATABASE_CONNECTION_STRING = 'postgresql+psycopg://postgres:scripts@localhost/postgres'
# DATABASE_CONNECTION_STRING = 'mysql+pymysql://scripts:scripts@localhost/presidents'

DB_ENGINE = create_engine(DATABASE_CONNECTION_STRING, echo=False)
AUTOMAP_BASE = automap_base()
AUTOMAP_BASE.prepare(DB_ENGINE)

PRESIDENT_MODEL = AUTOMAP_BASE.classes.presidents

SESSION_FACTORY = sessionmaker(bind=DB_ENGINE)
SESSION = scoped_session(SESSION_FACTORY)

PRESIDENT_FIELDS = {
    'termnum': fr_fields.Integer,
    'firstname': fr_fields.String,
    'lastname': fr_fields.String,
    'birthstate': fr_fields.String,
    'party': fr_fields.String,
    'relative_uri': fr_fields.Url('presidentid', absolute=False),
    'absolute_uri': fr_fields.Url('presidentid', absolute=True),
}

class PresidentQuerySchema(Schema):
    termnum = ma_fields.Integer()
    firstname = ma_fields.String()
    lastname = ma_fields.String()
    birthstate = ma_fields.String()
    party = ma_fields.String()

PRESIDENT_QUERY_SCHEMA = PresidentQuerySchema()
```

```

PRESIDENT_COUNT = SESSION.query(PRESIDENT_MODEL).count()

def invalid_term_number(term_number):
    return not (1 <= term_number <= PRESIDENT_COUNT)

class InvalidAPIRequest(Exception):
    status_code = 400
    def __init__(self, message, status_code=None, data=None):
        Exception.__init__(self)
        self.message = message
        if status_code is not None:
            self.status_code = status_code
        self._data = data

    def to_dict(self):
        return_value = dict(self._data or {})
        return_value['message'] = self.message
        return return_value

@app.errorhandler(InvalidAPIRequest)
def handle_invalid_request(error):
    response = jsonify(error.to_dict())
    response.status_code = error.status_code
    return response

class PresidentWithoutID(Resource):

    @marshal_with(PRESIDENT_FIELDS, envelope='president')
    def get(self):
        """Retrieve all or filtered records"""
        where_args = []
        args = PRESIDENT_QUERY_SCHEMA.dump(request.args)
        for arg, value in args.items():
            # add query with case-insensitive search
            where_args.append(
                # e.g. PRESIDENT_MODEL.lastname.ilike("roos")
                getattr(PRESIDENT_MODEL, arg).ilike(f"%{value}%")
            )
        sql_stmt = select(PRESIDENT_MODEL).where(*where_args)

        # result is an iterator of "chunks" (tuples)
        result_iterator = SESSION.execute(sql_stmt)

        # first element of each tuple is one model instance
        data = [chunk[0] for chunk in result_iterator if chunk[0].termnum is not None]
        return data

    @marshal_with(PRESIDENT_FIELDS, envelope='president')

```

```

def post(self):
    """Add a new record"""
    args = PRESIDENT_QUERY_SCHEMA.dump(request.json)

    p = PRESIDENT_MODEL(**args)
    SESSION.add(p)
    SESSION.commit()

    return args, 201

class PresidentWithID(Resource):
    @marshal_with(PRESIDENT_FIELDS, envelope='president')
    def get(self, termnum):
        sql_stmt = select(PRESIDENT_MODEL).where(PRESIDENT_MODEL.termnum == termnum)
        try:
            # returns one Row or None
            result_iterator = SESSION.execute(sql_stmt).one_or_none()
        except Exception as err:
            raise InvalidAPIRequest(f"Term number {termnum} not found: {err}")
        else:
            if result_iterator is not None:
                return PRESIDENT_QUERY_SCHEMA.dump(result_iterator[0])
            else:
                raise InvalidAPIRequest(f"No results for term # {termnum}")

    @marshal_with(PRESIDENT_FIELDS, envelope='president')
    def put(self, termnum):
        """Replace one record"""
        if invalid_term_number(termnum):
            raise ValueError("Invalid term number")
        return {}

    @marshal_with(PRESIDENT_FIELDS, envelope='president')
    def patch(self, termnum):
        """Update one record"""
        if invalid_term_number(termnum):
            raise ValueError("Invalid term number")
        return {}

    def delete(self, termnum):
        """Delete one record"""
        p = SESSION.query(PRESIDENT_MODEL).filter(PRESIDENT_MODEL.termnum ==
termnum).one()
        if p:
            SESSION.delete(p)
            SESSION.commit()
            return jsonify(f'Record {termnum} deleted')

```

```
api.add_resource(PresidentWithoutID, '/api/president', endpoint="president")
api.add_resource(PresidentWithID, '/api/president/<int:termnum>', endpoint="presidentid")

if __name__ == '__main__':
    app.run(debug=True)
```

Filtering queries

- Getting GET parameters
- Using `.filter()`
- Stacking filters

What happens if you have a resource with ten million records? The client probably doesn't want to retrieve all ten million rows.

You can allow the client to use GET parameters to **filter** the records.

In a `get()` method, use **`flask_restx.reqparse.RequestParser()`** to access the GET parameters. It is similar to the **`argparse`** module in the standard library. You specify what parameters you want to retrieve, call **`.parse_args()`**, and that returns a dictionary of the parameters.

Then use the `.filter()` method on the object query. This returns a new query, so you can filter more than once, which is convenient for multiple parameters.

See the `get()` method of the **`PresidentsList`** class in **`fr_president_api.py`** for examples of filtering on multiple parameters.

From the Flask docs:

Warning



The whole request parser part of Flask-RESTX is slated for removal and will be replaced by documentation on how to integrate with other packages that do the input/output stuff better (such as marshmallow). This means that it will be maintained until 2.0 but consider it deprecated. Don't worry, if you have code using that now and wish to continue doing so, it's not going to go away any time too soon.

Chapter 11 Exercises

Exercise 11-1 (airport_codes_api.py)

There is a SQLite3 database named **airports.db** in the DATA folder. It contains two columns (other than the ID): **code**, which is the IATA 3-letter code for the airport, and **location**, which is the city, state, and country where the airport is located. There are around 9000 airports in the database.

The schema looks like this:

```
CREATE TABLE airport (  
    id integer primary key,  
    code char(3),  
    location varchar(128)  
);
```

Create an API for the airport data which allows the client to request either a list of all airports, or a particular airport by its code (which is the unique ID).

If you have time, you can try adding query fields to the airport list, so you could list all airports that have "california" in the location text, for instance.

Chapter 12: Container Classes

Objectives

- Recap builtin container types
- Discover extra container types in standard library
- Create custom variations of container types

Container classes

- Contain all elements in memory objects
- Elements are objects or key/object pairs
- Have a length
- Are indexable and iterable

Container classes are objects that can hold multiple objects. All of the objects contained are kept in memory. Containers can be indexed, and can be iterated over. All containers have a length, which is the number of elements.



If `len(_container_)` is 0, the Boolean value of the container is False; otherwise the container is True.

Builtin containers

- `list`, `tuple` *array-like*
- `dict` *dictionary*
- `set`, `frozenset` *set of values*
- `str` *sequence of characters*
- `bytes` *sequence of bytes*

Several container types are builtin. All types have a length (available through the `len()` function).

Array-like

A `list` is a dynamic array, while a `tuple` is more like a struct or record. Both can be indexed or sliced.

Strings (`str`) and byte strings (`bytes`) are also array-like. They have a length, and can be indexed and sliced.

Mapping types

The `dict` type is a dictionary of key/value pairs. In some languages, dictionaries are called hashes, or even more exotic names. Dictionaries are dynamic. Keys must be unique.

The `set` type contains unique values. Elements may be added to and deleted from a set. A `frozenset` is a readonly set.

Strings and bytes

A `str` object is an array of Unicode characters, used for any kind of text.

A `bytes` object is an array of bytes. This can be used for UTF or other encodings, and is also used for "binary" (non-text) data. If an element of a bytes object corresponds to the Unicode value of a printable character, it is displayed as the character when printed to the screen.



Dictionary keys and set elements must be *immutable* (technically *hashable*).

Example

builtin_containers.py

source,python

```
greek_list = ['alpha', 'beta', 'gamma', 'eta', 'alpha', 'omega', 'zeta']
address = ('123 Elm Street', 'Toledo', 'Ohio')
greek_dict = {'alpha': 5, 'beta': 10, 'gamma': 15}
greek_set = {'alpha', 'beta', 'gamma', 'eta', 'zeta'}
greek_str = 'alpha'
greek_bytes = b'omega'

print(f"{greek_list[0] = }")
print(f"{address[0] = }")
print(f"{greek_dict['alpha'] = }")
print(f"{greek_list[-1] = }")
print(f"{greek_list[:3] = }")
print(f"{greek_list[3:] = }")
print(f"{greek_list[2:5] = }")
print(f"{greek_list[-2:] = }")
print(f"{ 'alpha' in greek_list = }")
print(f"{ 'gamma' in greek_list = }")
print(f"{ 'zeta' in greek_set = }")
print(f"{len(greek_list) = }")
print(f"{len(address) = }")
print(f"{len(greek_dict) = }")
print(f"{len(greek_set) = }")
print(f"{greek_list.count('alpha') = }")
print(f"{len(greek_str) = }")
print(f"{greek_str[0] = }")
print(f"{len(greek_bytes) = }")
print(f"{greek_bytes[0] = }")
```

builtin_containers.py

```
greek_list[0] = 'alpha'
address[0] = '123 Elm Street'
greek_dict['alpha'] = 5
greek_list[-1] = 'zeta'
greek_list[:3] = ['alpha', 'beta', 'gamma']
greek_list[3:] = ['eta', 'alpha', 'omega', 'zeta']
greek_list[2:5] = ['gamma', 'eta', 'alpha']
greek_list[-2:] = ['omega', 'zeta']
'alpha' in greek_list = True
'gamma' in greek_list = True
'zeta' in greek_set = True
len(greek_list) = 7
len(address) = 3
len(greek_dict) = 3
len(greek_set) = 5
greek_list.count('alpha') = 2
len(greek_str) = 5
greek_str[0] = 'a'
len(greek_bytes) = 5
greek_bytes[0] = 111
```

Containers in the standard library

- Many containers in the `collections` module
- Variations of lists, tuples, and dicts

The `collections` module provides several additional container types. In general, these are variations on lists, tuples, and dicts.

Counter

A **Counter** is a dict with a default value of 0. There are two ways to use it.

- You can initialize a `Counter` object with any iterable, and it will count those values.
- Whether or not the `Counter` is initialized, you can increment the value for a given key without first checking to see whether the key exists.

In addition, counter provides a method `most_common()`, which returns the n most common elements counted.

Example

`count_with_counter.py`

```
from collections import Counter

with open("../DATA/breakfast.txt") as breakfast_in:
    foods = [line.rstrip() for line in breakfast_in] # create list of foods with EOL
    removed from line

counts = Counter(foods) # initialize Counter object with list of foods

for item, count in counts.items(): # iterate over results
    print(item, count)
print()

print(f"{counts.most_common(4) = }")
```

count_with_counter.py

```
spam 34
Lucky Charms 4
eggs 3
oatmeal 1
sausage 4
upma 3
poha 4
ackee and saltfish 4
bacon 6
pancakes 2
idli 7
dosas 4
waffles 2
crumpets 1

counts.most_common(4) = [('spam', 34), ('idli', 7), ('bacon', 6), ('Lucky Charms', 4)]
```

defaultdict

A **defaultdict** is a dict that provides a default value for missing keys. When the defaultdict is created, you pass in a function that provides that default value. If you need a constant value, such as 0 or "", you can use a lambda expression for the function.

Example

defaultdict_ex.py

```
from collections import defaultdict

dd = defaultdict(lambda: 0) # create default dict with function that returns 0

dd['spam'] = 10 # assign some values to the dict
dd['eggs'] = 22

print(dd['spam']) # print values
print(dd['eggs'])
print(dd['foo']) # missing key 'foo' invokes function and returns 0

print('-' * 60)

fruits = ["pomegranate", "cherry", "apricot", "date", "apple",
"lemon", "kiwi", "orange", "lime", "watermelon", "guava",
"papaya", "fig", "pear", "banana", "tamarind", "persimmon",
"elderberry", "peach", "blueberry", "lychee", "grape" ]

fruit_info = defaultdict(list)

for fruit in fruits:
    first_letter = fruit[0]
    fruit_info[first_letter].append(fruit)

for letter, fruits in sorted(fruit_info.items()):
    print(letter, fruits)
```


defaultdict_ex.py

```
10
22
0
-----
a ['apricot', 'apple']
b ['banana', 'blueberry']
c ['cherry']
d ['date']
e ['elderberry']
f ['fig']
g ['guava', 'grape']
k ['kiwi']
l ['lemon', 'lime', 'lychee']
o ['orange']
p ['pomegranate', 'papaya', 'pear', 'persimmon', 'peach']
t ['tamarind']
w ['watermelon']
```

namedtuple

A **namedtuple** is a tuple with specified field names. In addition to accessing fields as *tuple[0]*, you can access them as *tuple.field_name*. Named tuples map very well to C **structs**.

Example

named_tuples.py

```
from collections import namedtuple
from pprint import pprint

Knight = namedtuple('Knight', 'name title color quest comment') # create named tuple
class with specified fields (could also provide fieldnames as iterable)

k = Knight('Bob', 'Sir', 'green', 'whirled peas', 'Who am i?') # create named tuple
instance (must specify all fields)

print(k.title, k.name) # can access fields by name...
print(k[1], k[0]) # ...or index
print()

knights = [] # initialize list for knight data
with open('../DATA/knights.txt') as knights_in:
    for raw_line in knights_in:
        # strip \n then split line into fields
        name, title, color, quest, comment = raw_line.rstrip().split(':')
        # create instance of Knight namedtuple
        knight = Knight(name, title, color, quest, comment)
        # add tuple to list
        knights.append(knight)

for knight in knights: # iterate over list of knights
    print(f'{knight.title} {knight.name}: {knight.color}')
print()
pprint(knights)
```

named_tuples.py

```
Sir Bob  
Sir Bob
```

```
King Arthur: blue  
Sir Galahad: red  
Sir Lancelot: blue  
Sir Robin: yellow  
Sir Bedevere: red, no blue!  
Sir Gawain: blue
```

```
[Knight(name='Arthur', title='King', color='blue', quest='The Grail', comment='King of  
the Britons'),  
 Knight(name='Galahad', title='Sir', color='red', quest='The Grail', comment="'I could  
handle some more peril'"),  
 Knight(name='Lancelot', title='Sir', color='blue', quest='The Grail', comment="'It\'s  
too perilous!'"),  
 Knight(name='Robin', title='Sir', color='yellow', quest='Not Sure', comment='He boldly  
ran away'),  
 Knight(name='Bedevere', title='Sir', color='red, no blue!', quest='The Grail',  
comment='AARRRRRRRGGGGHH'),  
 Knight(name='Gawain', title='Sir', color='blue', quest='The Grail', comment='none')]
```

deque

A **deque** (pronounced "deck") is a double-ended queue. It is optimized for inserting and removing from the ends, and is much more efficient than a **list** for this purpose. The deque can be initialized with any iterable.

Example

deque_ex.py

```
from collections import deque

d = deque() # Create an empty deque
for c in 'abcdef':
    d.append(c) # Append to the deque
print(f"{d = }")

for c in 'ghijkl':
    d.appendleft(c) # Prepend to the deque
print(f"{d = }")

d.extend('mno') # Extend the deque at the end one letter at a time
print(f"{d = }")

d.extendleft('pqr') # Extend the deque at the beginning one letter at a time
print(f"{d = }")

print(d[9])
print(f"{d[9] = }")
print(f"{d.pop() = }")
print(f"{d.popleft() = }")

print(f"{d = }")
```

deque_ex.py

```
d = deque(['a', 'b', 'c', 'd', 'e', 'f'])
d = deque(['l', 'k', 'j', 'i', 'h', 'g', 'a', 'b', 'c', 'd', 'e', 'f'])
d = deque(['l', 'k', 'j', 'i', 'h', 'g', 'a', 'b', 'c', 'd', 'e', 'f', 'm', 'n', 'o'])
d = deque(['r', 'q', 'p', 'l', 'k', 'j', 'i', 'h', 'g', 'a', 'b', 'c', 'd', 'e', 'f',
'm', 'n', 'o'])
a
d[9] = 'a'
d.pop() = 'o'
d.popleft() = 'r'
d = deque(['q', 'p', 'l', 'k', 'j', 'i', 'h', 'g', 'a', 'b', 'c', 'd', 'e', 'f', 'm',
'n'])
```

OrderedDict

An **OrderedDict** is a dictionary which preserves order. Any iteration over the dictionary's keys or values is guaranteed to be in the same order that the elements were added.

Starting with Python 3.6, all dictionaries preserve order, making **OrderedDict** obsolete. (Of course legacy code may still use them, and some existing modules in the stdlib use or return **OrderedDict**).

Emulating builtin types

- Inherit from builtin type
- Override special methods as needed
- Use `super()` to invoke base class's special methods

To emulate builtin types, you can inherit from builtin classes and override special methods as needed to get the desired behavior; other special methods will work in the normal way.

When overriding the special methods, you usually want to invoke the base class's special methods. Use the `super()` builtin function; the syntax is:

```
super().__specialmethod__(...)
```

Table 26. Special Methods and Variables

Method or Variables	Description
<code>__new__(cls, ...)</code>	Returns new object instance; Called before <code>{dunder}init{dunder}()</code>
<code>__init__(self, ...)</code>	Object initializer (constructor)
<code>__del__(self)</code>	Called when object is about to be destroyed
<code>__repr__(self)</code>	Called by <code>repr()</code> builtin
<code>__str__(self)</code>	Called by <code>str()</code> builtin
<code>__eq__(self, other)</code> <code>__ne__(self, other)</code> <code>__gt__(self, other)</code> <code>__lt__(self, other)</code> <code>__ge__(self, other)</code> <code>__le__(self, other)</code>	Implement comparison operators <code>==</code> , <code>!=</code> , <code>></code> , <code><</code> , <code>>=</code> , and <code><=</code> . <code>self</code> is object on the left.
<code>__cmp__(self, other)</code>	Called by comparison operators if <code>{dunder}eq{dunder}</code> , etc., are not defined
<code>__hash__(self)</code>	Called by <code>hash`()</code> builtin, also used by <code>dict</code> , <code>set</code> , and <code>frozenset</code> operations
<code>__bool__(self)</code>	Called by <code>bool()</code> builtin. Implements truth value (boolean) testing. If not present, <code>bool()</code> uses <code>len()</code>
<code>__unicode__(self)</code>	Called by <code>unicode()</code> builtin
<code>__getattr__(self, name)</code> <code>__setattr__(self, name, value)</code> <code>__delattr__(self, name)</code>	Override normal fetch, store, and deleter
<code>__getattribute__(self, name)</code>	Implement attribute access for new-style classes
<code>__get__(self, instance)</code>	<code>__set__(self, instance, value)</code>
<code>__del__(self, instance)</code>	Implement descriptors
<code>__slots__ = variable-list</code>	Allocate space for a fixed number of attributes.
<code>__metaclass__ = callable</code>	Called instead of <code>type()</code> when class is created.
<code>__instancecheck__(self, instance)</code>	Return <code>True</code> if instance is an instance of class
<code>__subclasscheck__(self, instance)</code>	Return <code>True</code> if instance is a subclass of class
<code>__call__(self, ...)</code>	Called when instance is called as a function.
<code>__len__(self)</code>	Called by <code>len()</code> builtin
<code>__getitem__(self, key)</code>	Implements <code>self[key]</code>
<code>__setitem__(self, key, value)</code>	Implements <code>self[key] = value</code>
<code>__delitem__(self, key)</code>	Implements <code>del self[key]</code>

Method or Variables	Description
<code>__iter__(self)</code>	Called when iterator is applied to container
<code>__reversed__(self)</code>	Called by <code>reversed()</code> builtin
<code>__contains__(self, object)</code>	Implements <code>in</code> operator
<code>__add__(self, other)</code> <code>__sub__(self, other)</code> <code>__mul__(self, other)</code> <code>__floordiv__(self, other)</code> <code>__mod__(self, other)</code> <code>__divmod__(self, other)</code> <code>__pow__(self, other[, modulo])</code> <code>__lshift__(self, other)</code> <code>__rshift__(self, other)</code> <code>__and__(self, other)</code> <code>__xor__(self, other)</code> <code>__or__(self, other)</code>	Implement binary arithmetic operators {plus}, <code>-</code> , {asterisk}, <code>/</code> , <code>//</code> , <code>%</code> , {asterisk}{asterisk}, <code><<</code> , <code>>></code> , <code>&</code> , {caret}, and {vbar}. self is object on left side of expression.
<code>__div__(self, other)</code> <code>__truediv__(self, other)</code>	Implement binary division operator <code>/</code> . <code>{dunder}truediv{dunder}()</code> is called if <code>{dunder}future{dunder}.division</code> is in effect.
<code>__radd__(self, other)</code> <code>__rsub__(self, other)</code> <code>__rmul__(self, other)</code> <code>__rdiv__(self, other)</code> <code>__rtruediv__(self, other)</code> <code>__rfloordiv__(self, other)</code> <code>__rmod__(self, other)</code> <code>__rdivmod__(self, other)</code> <code>__rpow__(self, other)</code> <code>__rlshift__(self, other)</code> <code>__rrshift__(self, other)</code> <code>__rand__(self, other)</code> <code>__rxor__(self, other)</code> <code>__ror__(self, other)</code>	Implement binary arithmetic operators with swapped operands. (Used if left operand does not support the corresponding operation)
<code>__iadd__(self, other)</code> <code>__isub__(self, other)</code> <code>__imul__(self, other)</code> <code>__idiv__(self, other)</code> <code>__itruediv__(self, other)</code> <code>__ifloordiv__(self, other)</code> <code>__imod__(self, other)</code> <code>__ipow__(self, other[, modulo])</code> <code>__ilshift__(self, other)</code> <code>__irshift__(self, other)</code> <code>__iand__(self, other)</code> <code>__ixor__(self, other)</code> <code>__ior__(self, other)</code>	Implement augmented (<code>+=</code> , <code>-=</code> , etc.) arithmetic operators

Method or Variables	Description
<code>__neg__(self)</code> <code>__pos__(self)</code> <code>__abs__(self)</code> <code>__invert__(self)</code>	Implement unary arithmetic operators -, +, abs(), and ~
<code>__oct__(self)</code> <code>__hex__(self)</code>	Implement <code>oct()</code> and <code>hex()</code> builtins
<code>__index__(self)</code>	Implement <code>operator.index()</code>
<code>__coerce__(self, other)</code>	Implement "mixed-mode" numeric arithmetic.

Creating list-like containers

- Inherit from `list`
- Override special methods as needed
- Commonly overridden methods
 - `__getitem__()`
 - `__setitem__()`
 - `__append__()`

To create a list-like container, inherit from `list`. Commonly overridden methods include `{dunder}getitem(){dunder}` and `{dunder}setitem(){dunder}`.

Example

multiindexlist.py

```
class MultiIndexList(list): # Define new class that inherits from list

    def __getitem__(self, item): # Redefine __getitem__ which implements []
        if isinstance(item, tuple): # Check to see if index is tuple
            if len(item) == 0:
                raise ValueError("Tuple must be non-empty")
            else:
                tmp_list = []
                for index in item:
                    tmp_list.append(
                        super().__getitem__(index) # Call list.__getitem__() for each
index in tuple
                    )
                return tmp_list
            else:
                return super().__getitem__(item) # Call the normal __getitem__()

if __name__ == '__main__':
    fruits = 'banana peach nectarine fig kiwi lemon lime'.split()
    m = MultiIndexList(fruits) # Initialize a MultiIndexList with a list
    m.append('apple') # Add an element (works like normal list)
    m.append('mango')
    print(m)

    print(f"m[0]: {m[0]}") # normal indexing
    print(f"m[5, 2, 0]: {m[5, 2, 0]}") # multi-index with tuple
    print(f"m[:4]: {m[:4]}") # normal slice
    print(f"len(m): {len(m)}") # len() works normally
    print(f"m[5]: {m[5]}") # get one item (normal behavior)
    print(f"m[5,]: {m[5,]}") # get list with just one item [m[5]]
    print(f"m[:2,-2:]: {m[:2,-2:]}") # get list with first two, last two items
    print()
    print(f"m: {m}")
    print(m)
    m.extend(['durian', 'kumquat'])
    print(m)
    print()
    for fruit in m:
        print(fruit)
```

multiindexlist.py

```
['banana', 'peach', 'nectarine', 'fig', 'kiwi', 'lemon', 'lime', 'apple', 'mango']
m[0]: banana
m[5, 2, 0]: ['lemon', 'nectarine', 'banana']
m[:4]: ['banana', 'peach', 'nectarine', 'fig']
len(m): 9
m[5]: lemon
m[5,]: ['lemon']
m[:2,-2:]: [['banana', 'peach'], ['apple', 'mango']]

m: ['banana', 'peach', 'nectarine', 'fig', 'kiwi', 'lemon', 'lime', 'apple', 'mango']
['banana', 'peach', 'nectarine', 'fig', 'kiwi', 'lemon', 'lime', 'apple', 'mango']
['banana', 'peach', 'nectarine', 'fig', 'kiwi', 'lemon', 'lime', 'apple', 'mango',
'durian', 'kumquat']

banana
peach
nectarine
fig
kiwi
lemon
lime
apple
mango
durian
kumquat
```

Using collections.abc

- `collections.abc` contains abstract base classes
- Inherit from one or more to create hybrid types

To start from scratch, you can inherit from abstract base classes defined in the `collections.abc` module. These are base classes that define required methods for various container types. In other words, inheriting from `collections.abc.MutableSequence` provides abstract methods required for making a `list`-like class.

Example

container_abc.py

```
from collections.abc import Sized, Iterator # Abstract base classes, used similarly to
interfaces in Java or C#

class BadContainer(Sized): # This class may not be instantiated without defining `len()`
    pass

class GoodContainer(Sized):
    def __len__(self): # This class is fine, since `Sized` requires `len()` to be
        implemented
        return 42

try:
    bad = BadContainer() # Instantiating `BadContainer` raises an error.
except TypeError as err:
    print(err)
else:
    print(bad)

print()

try:
    good = GoodContainer() # Instantiating `GoodContainer` is fine
except TypeError as err:
    print(err)
else:
    print(good)
    print(len(good)) # Builtin function `len()` works with all objects that inherit from
`Sized` (due to implementation of `len()`)

print()

class MyIterator(Iterator): # ABC `Iterator` provides abstract method `next`
    data = 'a', 'b', 'c'
    index = 0

    def __next__(self): # Must be implemented for Iterators
        if self.index >= len(self.data):
            raise StopIteration
        else:
```

```
        return_val = self.data[self.index]
        self.index += 1
        return return_val
```

```
m = MyIterator() # Create instance of `MyIterator`
for i in m: # Iterate over the iterator instance
    print(i)
print()

print(hasattr(m, '__iter__')) # Check to see if `m` is iterable
```

container_abc.py

Can't instantiate abstract class BadContainer with abstract method __len__

```
<__main__.GoodContainer object at 0x1072308d0>
```

```
42
```

```
a
```

```
b
```

```
c
```

```
True
```


Creating dict-like containers

- Inherit from dict
- Implement special methods
- Commonly overridden methods
 - `__getitem__`
 - `__haskey__`
 - `__setitem__`

To create custom dictionaries, inherit from dict and implement special methods as needed. Commonly overridden special methods include `__getitem__()`, `__setitem__()`, and `__haskey__()`.

Example

stringkeydict.py

```
class StringKeyDict(dict): # Create class that inherits from dict
    def __setitem__(self, key, value): # Overwrite how values are stored in the dict via
        _DICT[_KEY_] = _VALUE_
        if isinstance(key, str): # Make sure key is a string
            super().__setitem__(key, value) # Use dict's setitem to set value if it is
not a key
        else:
            # Raise error if non-string key is used
            raise TypeError(f"Keys must be strings not {type(key).__name__}s")

if __name__ == '__main__':
    d = StringKeyDict(a=10, b=20) # Create and initialize StringKeyDict instance
    for k, v in [('c', 30), ('d', 40), (1, 50), (('a', 1), 60), (5.6, 201)]:
        try:
            print(f"Setting {k} to {v}", end=' ')
            d[k] = v # Try to add various key/value pairs
        except TypeError as err:
            print(err) # Error raised on non-string key
        else:
            print('SUCCESS')

    print()
    print(d)
```

stringkeydict.py

```
Setting c to 30 SUCCESS
Setting d to 40 SUCCESS
Setting 1 to 50 Keys must be strings not ints
Setting ('a', 1) to 60 Keys must be strings not tuples
Setting 5.6 to 201 Keys must be strings not floats

{'a': 10, 'b': 20, 'c': 30, 'd': 40}
```

Free-form containers

- Easy to create hybrid containers
- Objects may implement any special methods
- Create list+dict, ordered set
- Be creative

There's really no limit to the types of objects you can create. You can make hybrids that act like lists and dictionaries at the same time. Just implement the special methods required for this behavior.

A well-known example of this is the **Element** class in the `lxml.etree` module. An Element is a list of its children, and at the same time is a dictionary of its XML attributes:

```
e = Element(...)
child_element = e[0]
attr_value = e.get("attribute")
```

Chapter 12 Exercises

Exercise 12-1 (maxlist.py, ringlist.py)

Part A

Create a new type, `MaxList`, that will only grow to a certain size. It should raise an `IndexError` if the user attempts to add an item beyond the limit. `MaxList` should inherit from `list` (or you can start from scratch with `collections.abc.MutableSequence`).

Run the provided script `use_maxlist.py` to try out your `MaxList` class.

HINT: You will need to override the `append()`, `insert()`, and `extend()` methods; to be thorough, you would need to override `{__init__}` as well, so that the initializer doesn't have more than the maximum number of items.

Part B

Copy `maxlist.py` to `ringlist.py` and create a `RingList` class that, once it reaches maximum size, appending to the list also removes the first element, so the list stays constant size.

Run the provided script `use_ringlist.py` to try out your `RingList` class.

Exercise 12-2 (normalstringdict.py)

Create a module which defines the class `NormalStringDict`, that only allows strings as keys *and* values. Values (but not keys) should be normalized by removing surrounding white space and converting to lower case.

Run the provided script `use_nsd.py` to try out your `NormalStringDict` class.

Chapter 13: Beautiful Soup

Objectives

- Use requests to download web pages
- Use Beautiful soup to parse HTML
- Find tags and extract data from HTML

Using requests

- Pythonic front end to urllib, urllib2, httplib, etc
- Makes HTTP requests simple

urllib and friends in the standard library are powerful, but their interfaces are not very Pythonic. You have to do a lot of work for simple tasks, such as adding proxies or authentication.

To avoid all this work, install the **requests** module.

requests implements GET, POST, PUT, and other HTTP verbs, and takes care of all the protocol housekeeping needed to send data on the URL, to send a username/password, and to retrieve data in various formats.

To add url parameters, pass a dictionary of key/value parameters with the **params** keyword:

```
payload = { 'key1': 'value1', ... }  
response = requests.get(URL, params=payload)
```

To use a proxy, pass a dictionary of protocol/url parameters with the **proxies** keyword

```
proxies={'http':'http://proxy.something.com:1234'}  
response = requests.get(URL, proxies=proxies)
```

To fetch the HTML from a site with authentication, pass a username/password tuple with the **auth** keyword

```
auth=('joeuser', 'l0lz')  
response = requests.get(URL, auth=auth)
```



See the quickstart guide for requests at: <https://requests.readthedocs.io/en/latest/user/quickstart/>

Example

http_read_html.py

```
import requests

URL = 'https://www.python.org'

response = requests.get(URL)

if response.ok:
    for header, value in sorted(response.headers.items()): # response.headers is a
        dictionary of the headers
        print(f"{header:20.20s} {value}")
    print()

    print(response.text[:400]) # The text is returned as a bytes object, so it needs to
    be decoded to a string; print the first 200 bytes
    print('...')
    print(response.text[-400:]) # print the last 200 bytes
```


http_read_html.py

```

Accept-Ranges      bytes
Age                1736
Connection         keep-alive
Content-Length     11064
Date              Tue, 24 Feb 2026 01:09:36 GMT
Strict-Transport-Sec max-age=63072000; includeSubDomains; preload
Vary               Cookie
X-Cache            MISS, HIT
X-Cache-Hits       0, 8
X-Served-By        cache-iad-kiad7000081-IAD, cache-iad-kiad7000027-IAD
X-Timer            S1771895376.148194,VS0,VE0
content-encoding   gzip
content-type       text/html; charset=utf-8
server             nginx
via                1.1 varnish, 1.1 varnish
x-frame-options    SAMEORIGIN

<!doctype html>
<html class="no-js" lang="en" dir="ltr">

<head>
  <script defer
    file-types="bz2,chm,dmg,exe,gz,json,msi,msix,pdf,pkg,tgz,xz,zip"
    data-domain="python.org"
    src="https://analytics.python.org/js/script.file-downloads.outbound-
links.js"></script>

    <meta charset="utf-8">

    <link rel="prefetch" href="//ajax.googleapis.com/ajax/libs/jquery/1.8.
...

```

Table 27. Keyword Parameters for `requests` methods

Option	Data Type	Description
<code>allow_redirects</code>	<code>bool</code>	set to True if PUT/POST/DELETE redirect following is allowed
<code>auth</code>	<code>tuple</code>	authentication pair (user/token,password/key)
<code>cert</code>	<code>str</code> or <code>tuple</code>	path to cert file or (<code>cert</code> , <code>key</code>) tuple
<code>cookies</code>	<code>dict</code> or <code>CookieJar</code>	cookies to send with request
<code>data</code>	<code>dict</code>	parameters for a POST or PUT request
<code>files</code>	<code>dict</code>	files for multipart upload
<code>headers</code>	<code>dict</code>	HTTP headers
<code>json</code>	<code>str</code>	JSON data to send in request body
<code>params</code>	<code>dict</code>	parameters for a GET request
<code>proxies</code>	<code>dict</code>	map protocol to proxy URL
<code>stream</code>	<code>bool</code>	if False, immediately download content
<code>timeout</code>	<code>float</code> or <code>tuple</code>	timeout in seconds or (connect timeout, read timeout) tuple
<code>verify</code>	<code>bool</code>	if True, then verify SSL cert



These can be used with any of the HTTP request types, as appropriate.

Table 28. `requests.Response` methods and attributes

Method/attribute	Definition
<code>apparent_encoding</code>	Returns the apparent encoding
<code>close()</code>	Closes the connection to the server
<code>content</code>	Content of the response, in bytes
<code>cookies</code>	A Cookiejar object with the cookies sent back from the server
<code>elapsed</code>	A timedelta object with the time elapsed from sending the request to the arrival of the response
<code>encoding</code>	The encoding used to decode <code>r.text</code>
<code>headers</code>	A dictionary of response headers
<code>history</code>	A list of response objects holding the history of request (url)
<code>is_permanent_redirect</code>	True if the response is the permanent redirected url, otherwise False
<code>is_redirect</code>	True if the response was redirected, otherwise False
<code>iter_content()</code>	Iterates over the response
<code>iter_lines()</code>	Iterates over the lines of the response
<code>json()</code>	Converts JSON content to Python data structure (if the result was written in JSON format, if not it raises an error)
<code>links</code>	The header links
<code>next</code>	A PreparedRequest object for the next request in a redirection
<code>ok</code>	True if <code>status_code</code> is less than 400, otherwise False
<code>raise_for_status()</code>	If an error occur, this method a <code>HTTPError</code> object
<code>reason</code>	A text corresponding to the status code
<code>request</code>	The request object that requested this response
<code>status_code</code>	A number that indicates the status (200 is OK, 404 is Not Found)
<code>text</code>	The content of the response, in unicode
<code>url</code>	The URL of the response

Using Beautiful Soup

- HTML/XML parser
- Pythonic Interface
- Accepts invalid XML or HTML
- Somewhat similar to ElementTree
- Many convenience functions

Beautiful Soup is a module for parsing HTML and XML. Unlike ElementTree, it will parse malformed markup. It has many convenience functions for searching and navigation. It has a Pythonic interface, in some ways similar to ElementTree, treating a tag as a dictionary of attributes; there are many differences as well.



The name "Beautiful Soup" is a reference to a parody song, sung by the Mock Turtle, in Alice's Adventures in Wonderland

Getting Started

- Import bs4
- Create a "soup" object from text or file-like object
- Provides root of XML or HTML

The first thing to do when using BeautifulSoup is to create a "soup" object, by calling the BeautifulSoup function from the module.

This object is the root of the markup tree, and from there you can search or navigate through the document.

The document is primarily made up of Tag objects (the others are text elements, etc.)

Example

bs4_parse_html.py

```
import bs4

with open('../DATA/craigslist.html', 'r') as CL:
    soup = bs4.BeautifulSoup(CL, 'lxml')

for input_tag in soup.findAll('input'):
    tag_name = input_tag.get('name', "*NO NAME*")
    tag_type = input_tag.get('type', '*NO TYPE*')
    print(f"{tag_name} ({tag_type})")
```

bs4_parse_html.py

```
areaID (hidden)
subAreaID (hidden)
query (*NO TYPE*)
*NO NAME* (submit)
```

Retrieving information

- Many find... and fetch... methods
- Most take flexible criteria

The Tag element provides many methods starting with "find" for retrieving information from a document.

These methods take a name filter plus some optional arguments, including matching on attributes.



BeautifulSoup does not support XPath

Example

bs4_parse_people.py

```
import bs4

with open('../DATA/people.xml') as people_in:
    soup = bs4.BeautifulSoup(
        people_in,
        features="xml",
    )

for person in soup.findAll('person'):
    print(f"{person.string}, {person.get('org')}")
```

bs4_parse_people.py

```
Linux Torvalds, Linux
Guido van Rossum, Python
Larry Wall, Perl
Bill Gates, Microsoft
Mark Zuckerberg, Facebook
```

Search Criteria

- Filters
 - Exact string
 - Regular expression
 - List of filters
 - True
 - Function
- Any unknown named parameter matches attributes
- Work with most find... and fetch... methods

The find... methods can take an exact string, but they can be much more flexible than that. The search criteria are known as filters.

The simplest filter is a plain string, which will find exact matches.

For pattern matching, use a regular expression.

You can also supply a list of filters, which will match any filter in the list.

If the filter is True, it will match every tag.

Finally, if the filter is a function object, it will be called with the tag object. If the function returns True, then the tag will be returned. This can be used for complex searches, such as "<p> tags whose class is *definition*".

After the name filter you can specify several named parameters. Any unknown named parameter matches an attribute. Because *class* is a keyword in Python, using *class_* will match *class* attributes.

In general, the second argument to a find method is a filter for any attribute. You can search for text by using the text keyword argument.

There are many more ways to specify search criteria. Check the Beautiful Soup documentation for details.

Example

bs4_filter_people.py

```
from bs4 import BeautifulSoup
import re

rx_digit = re.compile(r'\d')

with open('../DATA/people.xml') as people_in:
    soup = BeautifulSoup(
        markup=people_in,
        features="xml",
    )

for person in soup.findAll('person'):
    print(person.string)
    print('-' * 60)

for tag in soup.findAll(re.compile(r'^s')):
    print(tag)
    print('-' * 60)

for tag in soup.findAll(['street', 'state']):
    print(f"{tag}\n {tag.string}")
    print('-' * 60)

all_tags = soup.findAll(True)
print(f"There were {len(all_tags)} tags")
print('-' * 60)

def has_digit(tag):
    'Return True if specified string contains at least one digit'
    return tag.string != None and rx_digit.search(tag.string)

for tag in soup.findAll(has_digit):
    print(tag.string)
    print('-' * 60)

for tag in soup.findAll(class_='number'):
    print(tag)
    print('-' * 60)

for tag in soup.findAll(re.compile(r'wacker', re.IGNORECASE)):
    print(tag)
```



```
print('-' * 60)
```

bs4_filter_people.py

```
Linux Torvalds  
Guido van Rossum  
Larry Wall  
Bill Gates  
Mark Zuckerberg
```

```
-----  
<street class="streetname">233 S Wacker Dr</street>  
<suite class="number">800</suite>  
<state>IL</state>
```

```
-----  
<street class="streetname">233 S Wacker Dr</street>  
  233 S Wacker Dr  
<state>IL</state>  
  IL
```

```
-----  
There were 13 tags
```

```
-----  
233 S Wacker Dr  
800  
60606
```

```
-----  
<suite class="number">800</suite>  
<zip class="number">60606</zip>  
-----  
-----
```

Table 29. Beautiful Soup Tag methods

Method	Description
<code>clear()</code>	Delete all children
<code>decode()</code>	Return Unicode representation of tag + contents
<code>decode_contents()</code>	Return contents only as unicode string
<code>encode_contents()</code>	Return contents as bytestring (ASCII string)
<code>find()</code>	Return first matching criteria
<code>findAll()</code>	Return all matching criteria
<code>findChild()</code>	Return first child matching criteria
<code>findChildren()</code>	Find all children matching criteria
<code>get()</code>	Returns value of specified attribute, or default
<code>getText()</code>	Return all child strings concatenated
<code>index()</code>	Find index by identity
<code>isSelfClosing()</code>	Attribute set to True if tag is singleton like <tag/>
<code>string</code>	Attribute containing child string
<code>strings</code>	Attribute containing list of all child strings
<code>text</code>	Attribute containing child strings, concatenated

Modifying the Document

- Create new string with `soup.new_string()`
- Create new tag with `soup.new_tag()`
- Add with `append()`, `insert_before()`, `insert_after()`
- Replace with `replace_with()`
- Delete with `extract()` or `decompose()`
- Print with `prettyfy()`

In general, you can modify the document by assignment to tag attributes or strings.

To add elements to your document, create a new tag or string object and add it to the document using `append()` or one of the `insert...()` methods.

To replace an existing element, use the `replace_with()` method.

To delete elements, use either `decompose()` or `extract()`. `decompose()` completely deletes a tag from the document, much like Python's `del` statement. `extract()` removes a tag or string and returns it much like the `pop()` method of a list.

`clear()` removes the contents of a tag.

The `prettyfy()` method outputs the document in nicely indented format.

Example

bs4_modifying.py

```
import bs4

with open('../DATA/people.xml') as people_in: # Open input file
    soup = bs4.BeautifulSoup(
        people_in,
        features='xml',
    ) # Create a BeautifulSoup object

new_tag = soup.new_tag('person', org='Oracle') # Create a new XML 'person' tag with XML
attribute 'org'
new_tag.string = 'Larry Ellison' # Add the string contents for the new tag

people = soup.find('people') # Find the 'people' tag
people.append(new_tag) # Append the new tag as a child of 'people'

suite_tag = soup.find('suite') # Find the 'suite' tag
suite_tag.decompose() # Remove the 'suite' tag

new_xml = soup.prettify() # Create an XML string
print(new_xml)
```

bs4_modifying.py

```
<?xml version="1.0" encoding="utf-8"?>
<cmmap>
  <people>
    <person org="Linux">
      Linux Torvalds
    </person>
    <person org="Python">
      Guido van Rossum
    </person>
    <person org="Perl">
      Larry Wall
    </person>
    <person org="Microsoft">
      Bill Gates
    </person>
    <person org="Facebook">
      Mark Zuckerberg
    </person>
    <person org="Oracle">
      Larry Ellison
    </person>
  </people>
  <address>
    <street class="streetname">
      233 S Wacker Dr
    </street>
    <city>
      Chicago
    </city>
    <state>
      IL
    </state>
    <zip class="number">
      60606
    </zip>
  </address>
</cmmap>
```

Table 30. Beautiful Soup Element methods

Method	Description
<code>append()</code>	Append specified element to this element
<code>extract()</code>	Deletes this element
<code>findAllNext()</code>	Return elements matching criteria, after this element
<code>findAllPrevious()</code>	Return elements matching criteria, before this element
<code>findNext()</code>	Return first item matching criteria, after this element
<code>findNextSibling()</code>	Return siblings matching criteria, after this element
<code>findNextSiblings()</code>	Return siblings matching criteria, after this element
<code>findParent()</code>	Return closest parent matching criteria
<code>FindParents()</code>	Return closest parents matching criteria
<code>insert_after()</code>	Add specified element after this element
<code>insert_before()</code>	Add specified element before this element
<code>select()</code>	Perform CSS selection
<code>setup()</code>	Set up relations between this and other elements

Chapter 13 Exercises

Exercise 13-1 ([scrape_git.py](#))

Use BeautifulSoup to scrape all the links on the git home page (<https://git-scm.com>). Write out the anchor text and the link URLs to a file, in the format of your choice (plain file, JSON, CSV, etc.).

Chapter 14: Advanced Data Handling

Objectives

- Set default dictionary values
- Count items with the Counter object
- Define named tuples
- Prettyprint data structures
- Create and extract from compressed archives
- Save Python structures to the hard drive

Deep vs shallow copying

- Normal assignments create aliases
- New objects are shallow copies
- Use the `copy.deepcopy` module for deep copies *

Consider the following code:

```
colors = ['red', 'blue', 'green']  
c1 = colors
```

The assignment to variable `c1` does not create a new object; `c1` is another name that is *bound* to the same list object as the variable `colors`.

To create a new object, you can either use the list constructor `list()`, or use a slice which contains all elements:

```
c2 = list(colors)  
c3 = colors[::]
```

In both cases, `c2` and `c3` are each distinct objects.

However, the elements of `c2` and `c3` are not copied, but are still bound to the same objects as the elements of `colors`.

For example:

```
data1 = [ [1, 2, 3], [4, 5, 6] ]  
d1 = list(data)  
d1[0].append(99)  
print(data1)
```

This will show that the first element of `data1` contains the value 99, because `data1[0]` and `d1[0]` are both bound to the same object. To do a *deep* (recursive) copy, use the `deepcopy` function in the `copy` module.

Example

deep_copy.py

```
import copy

data = [
    [1, 2, 3],
    [4, 5, 6],
]

d1 = data # Bind d1 to same object as data
d2 = list(data) # Make shallow copy of data and store in d2
d3 = copy.deepcopy(data) # Make deep copy of data and store in d3

d1.append("d1") # Append to d1 (same as appending to data)
d1[0].append(50) # Append to first element of d1 (same first element as data)

d2.append("d2") # Append to d2 (does not affect data)
d2[0].append(99) # Append to first element of d2 (same first element as data and d1)

d3.append("d3") # Append to d3 (does not affect data)
d3[0].append(500) # Append to first element of d3 (does not affect data, d1, or d2)

print("data:", data, id(data))
print("d1:", d1, id(d1))
print("d2:", d2, id(d2))
print("d3:", d3, id(d3))
print()

print("id(d1[0]):", id(d1[0]))
print("id(d2[0]):", id(d2[0]))
print("id(d3[0]):", id(d3[0]))
```

deep_copy.py

```
data: [[1, 2, 3, 50, 99], [4, 5, 6], 'd1'] 4473045568
d1: [[1, 2, 3, 50, 99], [4, 5, 6], 'd1'] 4473045568
d2: [[1, 2, 3, 50, 99], [4, 5, 6], 'd2'] 4473045696
d3: [[1, 2, 3, 500], [4, 5, 6], 'd3'] 4473045632

id(d1[0]): 4472979072
id(d2[0]): 4472979072
id(d3[0]): 4473045440
```

Default dictionary values

- Use `defaultdict` from the `collections` module
- Specify function that provides default value
- Good for counting, datasets

Normally, when you use an invalid key with a dictionary, it raises a `KeyError`. The `defaultdict` class in the `collections` module allows you to provide a default value, so there will never be a `KeyError`. You will provide a function that returns the default value. A lambda function can be used for the function.

Example

`defaultdict_ex.py`

```
from collections import defaultdict

dd = defaultdict(lambda: 0) # create default dict with function that returns 0

dd['spam'] = 10 # assign some values to the dict
dd['eggs'] = 22

print(dd['spam']) # print values
print(dd['eggs'])
print(dd['foo']) # missing key 'foo' invokes function and returns 0

print('-' * 60)

fruits = ["pomegranate", "cherry", "apricot", "date", "apple",
"lemon", "kiwi", "orange", "lime", "watermelon", "guava",
"papaya", "fig", "pear", "banana", "tamarind", "persimmon",
"elderberry", "peach", "blueberry", "lychee", "grape" ]

fruit_info = defaultdict(list)

for fruit in fruits:
    first_letter = fruit[0]
    fruit_info[first_letter].append(fruit)

for letter, fruits in sorted(fruit_info.items()):
    print(letter, fruits)
```

defaultdict_ex.py

```
10
22
0
-----
a ['apricot', 'apple']
b ['banana', 'blueberry']
c ['cherry']
d ['date']
e ['elderberry']
f ['fig']
g ['guava', 'grape']
k ['kiwi']
l ['lemon', 'lime', 'lychee']
o ['orange']
p ['pomegranate', 'papaya', 'pear', 'persimmon', 'peach']
t ['tamarind']
w ['watermelon']
```

Counting with Counter

- Use `collections.Counter`
- Default value is 0
- Initialize or increment elements

For ease in counting, `collections` provides a `Counter` object. This is essentially a `defaultdict` whose default value is zero. Thus, you can just increment the value for any key, whether it's been seen before or no.

In addition, you can initialize a `Counter` object with any iterable.



The `pandas` module will also count data from datasets.

Example

`count_with_counter.py`

```
from collections import Counter

with open("../DATA/breakfast.txt") as breakfast_in:
    foods = [line.rstrip() for line in breakfast_in] # create list of foods with EOL
    removed from line

counts = Counter(foods) # initialize Counter object with list of foods

for item, count in counts.items(): # iterate over results
    print(item, count)
print()

print(f"{counts.most_common(4) = }")
```

count_with_counter.py

```
spam 34
Lucky Charms 4
eggs 3
oatmeal 1
sausage 4
upma 3
poha 4
ackee and saltfish 4
bacon 6
pancakes 2
idli 7
dosas 4
waffles 2
crumpets 1

counts.most_common(4) = [('spam', 34), ('idli', 7), ('bacon', 6), ('Lucky Charms', 4)]
```

Named Tuples

- In `collections` module
- Like `tuple`, but each element has a name
- Use either one
 - `tuple.name`
 - `tuple[n]`

A *named tuple* is a tuple where each element has a name, and can be accessed via the name in addition to the normal access by index. Thus, if `p` were a named tuple representing a point, you could say `p.x` and `p.y`, or you could say `p[0]` and `p[1]`.

A named tuple is created with the `namedtuple` class in the `collections` module. You are essentially creating a new class that inherits from `tuple`.

Pass the name of the new named tuple followed by a string containing the individual field names separated by spaces. `namedtuple` returns a new class containing the specified fields. It must be initialized with values for all fields. Being a tuple, it may not be altered once created.

Convert a named tuple into a dictionary with the `._asdict()` method.



A named tuple is the closest thing Python has to a C struct.

Example

named_tuples.py

```
from collections import namedtuple
from pprint import pprint

Knight = namedtuple('Knight', 'name title color quest comment') # create named tuple
class with specified fields (could also provide fieldnames as iterable)

k = Knight('Bob', 'Sir', 'green', 'whirled peas', 'Who am i?') # create named tuple
instance (must specify all fields)

print(k.title, k.name) # can access fields by name...
print(k[1], k[0]) # ...or index
print()

knights = [] # initialize list for knight data
with open('../DATA/knights.txt') as knights_in:
    for raw_line in knights_in:
        # strip \n then split line into fields
        name, title, color, quest, comment = raw_line.rstrip().split(':')
        # create instance of Knight namedtuple
        knight = Knight(name, title, color, quest, comment)
        # add tuple to list
        knights.append(knight)

for knight in knights: # iterate over list of knights
    print(f'{knight.title} {knight.name}: {knight.color}')
print()
pprint(knights)
```


named_tuples.py

```
Sir Bob  
Sir Bob
```

```
King Arthur: blue  
Sir Galahad: red  
Sir Lancelot: blue  
Sir Robin: yellow  
Sir Bedevere: red, no blue!  
Sir Gawain: blue
```

```
[Knight(name='Arthur', title='King', color='blue', quest='The Grail', comment='King of  
the Britons'),  
 Knight(name='Galahad', title='Sir', color='red', quest='The Grail', comment="'I could  
handle some more peril'"),  
 Knight(name='Lancelot', title='Sir', color='blue', quest='The Grail', comment="'It\'s  
too perilous!'"),  
 Knight(name='Robin', title='Sir', color='yellow', quest='Not Sure', comment='He boldly  
ran away'),  
 Knight(name='Bedevere', title='Sir', color='red, no blue!', quest='The Grail',  
comment='AARRRRRRRGGGGHH'),  
 Knight(name='Gawain', title='Sir', color='blue', quest='The Grail', comment='none')]
```

Alternatives to collections.namedtuple

There are several classes or packages for auto-generating a class or class-like object:

- `typing.NamedTuple`
- `dataclasses.dataclass`
- `pydantic`¹
- `attrs`¹

¹ These packages must be downloaded from **PyPI**, the Python package repository.

Table 31. Comparison of tuples and data class generators

	tuple	namedtuple	typing. NamedTuple	dataclass	pydantic	attrs
Writable	N	N	N	Y	Y	Y
Validation	N	N	N	N ²	Y	N ²
Iterable	Y	Y	Y	N	N	N
Indexable	Y	Y	Y	N	N	N
Methods	N	N	Y ¹	Y	Y	Y
Type hints	N	N	Y	Y	Y	Y

¹ Only when subclassing

² Could be added manually

Printing data structures

- Default representation of data structures is ugly
- `pprint` makes structures human friendly
- Use `pprint.pprint()`
- Useful for debugging

When debugging data structures, the `print()` function is not so helpful. A complex data structure is just printed out all jammed together, one element after another, with only a space between elements. This can be very hard to read.

The `pprint` (pretty print) module provides the `pprint()` function, which will analyze a structure and print it out with indenting and newlines. This makes the data easier to read.

You can customize the output with named parameters. See the following table for details.

Table 32. `pprint()` parameters

Parameter	Description	Default Value
<code>stream</code>	Write data to stream (stdout if None)	<code>None</code>
<code>indent</code>	How many spaces to indent each level	<code>1</code>
<code>width</code>	Only use specified number of columns	<code>80</code>
<code>depth</code>	Only display specified levels of data	<code>None</code>
<code>compact</code>	Try to display data more compactly	<code>False</code>
<code>sort_dicts</code>	Display dictionaries sorted by keys	<code>True</code>
<code>underscore_numbers</code>	Add underscores to numbers for readability	<code>False</code>

Example

pretty_printing.py

```
from pprint import pprint

struct = { # nested data structure
    'epsilon': [
        ['a', 'b', 'c'], ['d', 'e', 'f']
    ],
    'theta': {
        'red': 55,
        'blue': [8, 98, -3],
        'purple': ['Chicago', 'New York', 'L.A.'],
    },
    'alpha': ['g', 'h', 'i', 'j', 'k'],
    'gamma': [39029384, 3827539203, 94838402, 249398063],
}

print('Without pprint (normal output:')
print(struct) # print normally
print()

print('With pprint:')
pprint(struct) # pretty-print
print()

print('With pprint (sort_dicts=False):')
pprint(struct, sort_dicts=False) # Leave dictionary in default order
print()

print('With pprint (depth=2):')
pprint(struct, depth=2) # only print top two levels of structure
print()

print('With pprint (width=40):')
pprint(struct, width=40) # set display width
print()

print('With pprint (underscore_numbers=True):')
pprint(struct, underscore_numbers=True) # Put underscores in large numbers for
readability
```

pretty_printing.py

Without pprint (normal output:

```
{'epsilon': [['a', 'b', 'c'], ['d', 'e', 'f']], 'theta': {'red': 55, 'blue': [8, 98, -3],
'purple': ['Chicago', 'New York', 'L.A.']], 'alpha': ['g', 'h', 'i', 'j', 'k'], 'gamma':
[39029384, 3827539203, 94838402, 249398063]}
```

With pprint:

```
{'alpha': ['g', 'h', 'i', 'j', 'k'],
'epsilon': [['a', 'b', 'c'], ['d', 'e', 'f']],
'gamma': [39029384, 3827539203, 94838402, 249398063],
'theta': {'blue': [8, 98, -3],
          'purple': ['Chicago', 'New York', 'L.A.'],
          'red': 55}}
```

With pprint (sort_dicts=False):

```
{'epsilon': [['a', 'b', 'c'], ['d', 'e', 'f']],
'theta': {'red': 55,
          'blue': [8, 98, -3],
          'purple': ['Chicago', 'New York', 'L.A.']],
'alpha': ['g', 'h', 'i', 'j', 'k'],
'gamma': [39029384, 3827539203, 94838402, 249398063]}
```

With pprint (depth=2):

```
{'alpha': ['g', 'h', 'i', 'j', 'k'],
'epsilon': [...], [...]],
'gamma': [39029384, 3827539203, 94838402, 249398063],
'theta': {'blue': [...], 'purple': [...], 'red': 55}}
```

With pprint (width=40):

```
{'alpha': ['g', 'h', 'i', 'j', 'k'],
'epsilon': [['a', 'b', 'c'],
            ['d', 'e', 'f']],
'gamma': [39029384,
          3827539203,
          94838402,
          249398063],
'theta': {'blue': [8, 98, -3],
          'purple': ['Chicago',
                    'New York',
                    'L.A.'],
          'red': 55}}
```

With pprint (underscore_numbers=True):

```
{'alpha': ['g', 'h', 'i', 'j', 'k'],
'epsilon': [['a', 'b', 'c'], ['d', 'e', 'f']],
'gamma': [39_029_384, 3_827_539_203, 94_838_402, 249_398_063],
```

```
'theta': {'blue': [8, 98, -3],  
          'purple': ['Chicago', 'New York', 'L.A.'],  
          'red': 55}}
```

Zipped archives

- import `zipfile` for zipped files
- Get a list of files
- Extract files

The `zipfile` module provides the `ZipFile` class which allows you to read and write to zipped archives. In either case you first create a `ZipFile` object, specifying the name of the zip file.

Reading zip files

Create an instance of `ZipFile`, specifying the path to the zip file.

To get a list of members (contained files), use `((`ZIPFILE.namelist()`))`.

`((`ZIPFILE.getinfo("member-name")`))` will retrieve metadata about the member as a `ZipInfo` object.

To extract a member, use `ZIPFILE.extract("member-name")`. To read the data from a member without extracting it, use `ZIPFILE.read("member-name")`. When you read member data, it is read in binary mode, so a text file will be read in as a `bytes` object. Use `.decode()` on the bytes object to get a normal Python string.



The `tarfile` module will read and write compressed or uncompressed **tar** files.



The **Zip File Explorer** extension for VS Code will display the contents of zipfiles in the Explorer panel

Example

zipfile_read.py

```
from zipfile import ZipFile

# read & extract from zip file
zip_in = ZipFile("../DATA/textfiles.zip") # Open zip file for reading
print(zip_in.namelist()) # Print list of members in zip file
tyger_text = zip_in.read('tyger.txt').decode() # Read (raw binary) data from member and
convert from bytes to string
print(tyger_text[:100], '\n')
zip_in.extract('parrot.txt') # Extract member
```

zipfile_read.py

```
['fruit.txt', 'parrot.txt', 'tyger.txt', 'spam.txt']
    The Tyger

Tyger! Tyger! burning bright
In the forests of the night,
What immortal hand o
```


Writing to zip files

To create a new zip archive, create an instance of `ZipFile`, specifying mode `w`. Specify `ZIP_DEFLATED` (normal compression) as the compression type.

Add files to the zip archive with the `write()` method on the `ZipFile` object.

Example

`zipfile_write.py`

```
from zipfile import ZipFile, ZIP_DEFLATED
import os.path

file_names = ["parrot.txt", "tyger.txt", "knights.txt", "alice.txt", "poe_sonnet.txt",
"spam.txt"]
file_folder = "../DATA"

zipfile_name = "example.zip"

# creating new, empty, zip file
zip_out = ZipFile(zipfile_name, mode="w", compression=ZIP_DEFLATED) # Create new zip
file

# add files to zip file
for file_name in file_names:
    file_path = os.path.join(file_folder, file_name)
    zip_out.write(file_path, file_name) # Add member to zip file
zip_out.close()

# list files in zip
zip_in = ZipFile(zipfile_name)
print("Files in archive:")
print(zip_in.namelist())
```

Making archives with **shutil**

- Create archive from folder
- Simpler than `zipfile.ZipFile`

The `shutil` module makes it easy to create an archive of an entire folder. It will create archives in the following format: **zip**, **tar**, **gztar**, **bztar**.

To create an archive, call `shutil.make_archive()`. The arguments are:

1. Base name of the output file
2. Format (one of "zip", "tar", "gztar", "bztar", or "xztar").
3. Folder to be archived (current folder if omitted)

Example

`shutil_make_archive.py`

```
import shutil
import os

folder = '../DATA'
archive_name = "datafiles"

for archive_type in 'zip', 'gztar':
    shutil.make_archive(archive_name, archive_type, folder)
```

Serializing Data with **pickle**

- Use the **pickle** module
- Save to file
- Transmit over network

Serializing data means taking a data structure and transforming it so it can be written to a file or other destination, and later read back into the same data structure.

Python uses the **pickle** module for data serialization.

To create pickled data, use either **pickle.dump()** or **pickle.dumps()**. Both functions take a data structure as the first argument. **pickle.dumps()** returns the pickled data as a **bytes** object. **pickle.dump()** writes the data to a file-like object which has been specified as the second argument. The file-like object must be opened for writing.

To read pickled data, use **pickle.load()**, which takes a file-like object that has been open for writing, or **pickle.loads()** which reads from a string. Both functions return the original data structure that had been pickled.



Remember to open pickle files in binary mode.

Example

pickling.py

```
import pickle
from pprint import pprint

# some data structures
airports = {
    'RDU': 'Raleigh-Durham', 'IAD': 'Dulles', 'MGW': 'Morgantown',
    'EWR': 'Newark', 'LAX': 'Los Angeles', 'ORD': 'Chicago'
}

colors = [
    'red', 'blue', 'green', 'yellow', 'black',
    'white', 'orange', 'brown', 'purple'
]

values = [
    3/7, 1/9, 14.5
]

data = [ # list of data structures
    colors,
    airports,
    values,
]

print("BEFORE:")
pprint(data)
print('-' * 60)

with open('../TEMP/pickled_data.pkl', 'wb') as pkl_out: # open pickle file for writing
    in binary mode
    pickle.dump(data, pkl_out) # serialize data structures to pickle file

with open('../TEMP/pickled_data.pkl', 'rb') as pkl_in: # open pickle file for reading in
    in binary mode
    pickled_data = pickle.load(pkl_in) # de-serialize pickle file back into data
    structures

print("AFTER:")
pprint(pickled_data) # view data structures
```

pickling.py

```
BEFORE:
[['red',
  'blue',
  'green',
  'yellow',
  'black',
  'white',
  'orange',
  'brown',
  'purple'],
{'EWR': 'Newark',
 'IAD': 'Dulles',
 'LAX': 'Los Angeles',
 'MGW': 'Morgantown',
 'ORD': 'Chicago',
 'RDU': 'Raleigh-Durham'}],
[0.42857142857142855, 0.1111111111111111, 14.5]]
```

```
-----
AFTER:
[['red',
  'blue',
  'green',
  'yellow',
  'black',
  'white',
  'orange',
  'brown',
  'purple'],
{'EWR': 'Newark',
 'IAD': 'Dulles',
 'LAX': 'Los Angeles',
 'MGW': 'Morgantown',
 'ORD': 'Chicago',
 'RDU': 'Raleigh-Durham'}],
[0.42857142857142855, 0.1111111111111111, 14.5]]
```

Chapter 14 Exercises

Exercise 14-1 (count_ext.py)

Start with the existing script `count_ext.py`.

Add code to count the number of files with each extension in a file tree.

The script will take the starting folder as a command line argument, and then display the total number of files with each distinct file extension that it finds. Files with no extension should be skipped. Use a `Counter` object to do the counting.



Use `os.path.splitext()` to split the filename into a **path,extension** tuple.

Exercise 14-2 (prestuple.py, save_potus_info.py, read_potus_info.py)

Part C

Create a module named `prestuple` which defines a named tuple `President`, with fields **term**, **lastname**, **firstname**, **birthstate**, and **party**.

Part D

Write a script that uses the `csv` module to read the data from `presidents.csv` into a list of `President` named tuples. The script can import the `President` named tuple from the `prestuple` module.

Use the `pickle` module to write the list out to a file named **potus.pkl**.

Part E

Write a script to open **potus.pkl**, and restore the data back into an array.

Then loop through the array and print out each president's first name, last name, and party.



The `prestuple` module is not needed for this script.

Exercise 14-3 (make_zip.py)

Write a script which creates a zip file containing `save_potus_info.py`, `read_potus_info.py`, and **potus.pkl**.



Use `zipfile.ZipFile`

Chapter 15: Concurrency

Objectives

- Understand concurrency concepts
- Differentiate between threads, processes, and async
- Know when threads benefit your program
- Learn the limitations of the GIL
- Create a threaded application
- Use the multiprocessing module
- Develop a multiprocessing application

Concurrency

- Running more than one function concurrently
- Three main ways to achieve it
 - threading
 - multiple processes
 - asynchronous communication
- All supported in standard library

Computer programs spend a lot of their time doing nothing. This occurs when the CPU is waiting for the relatively slow disk subsystem, network stack, or other hardware to fetch data.

Some applications can achieve more throughput by taking advantage of this slack time by seemingly doing more than one thing at a time. With a single-core computer, this doesn't really happen; with a multicore computer, an application really can be executing different instructions at the same time. This is called multiprogramming or *concurrency*.

The three main ways to implement multiprogramming are threading, multiprocessing, and asynchronous communication:

Threading subdivides a single process into multiple subprocesses, or threads, each of which can be performing a different task. Threading in Python is good for IO-bound applications, but does not increase the efficiency of compute-bound applications.

Multiprocessing forks (spawns) new processes to do multiple tasks. Multiprocessing is good for both CPU-bound and IO-bound applications.

Asynchronous communication uses an event loop to poll multiple I/O channels rather than waiting for one to finish. Asynch communication is good for IO-bound applications, and can be faster than the other approaches.

The standard library supports all three.

Threads

- Like processes (but lighter weight)
- Use fewer resources (memory and CPU)
- Process can create one or more additional threads
- Similar to creating new processes with `fork()`

Modern operating systems (OSs) use time-sharing to manage multiple programs which appear to the user to be running simultaneously. Assuming a standard machine with only one CPU, that simultaneity is only an illusion, since only one program can run at a time, but it is a very useful illusion. Each program that is running counts as a process. The OS maintains a process table, listing all current processes. Each process will be shown as currently being in either Run state or Sleep state.

A thread is like a process. A thread might even be a process, depending on the implementation. In fact, threads are sometimes called “lightweight” processes, because threads occupy much less memory, and take less time to create, than do processes.

A process can create any number of threads. This is similar to a process calling the `fork()` function. The process itself is a thread, and could be considered the “main” thread.

Just as processes can be interrupted at any time, so can threads.

The Python Thread Manager

- Python uses underlying OS's threads
- Alas, the GIL – Global Interpreter Lock
- Only one thread runs at a time
- Python interpreter controls end of thread's turn
- Cannot take advantage of multiple processors

Python “piggybacks” on top of the OS's underlying threads system. A Python thread is a real OS thread. If a Python program has three threads, for instance, there will be three entries in the OS's thread list.

However, Python imposes further structure on top of the OS threads. Most importantly, there is a global interpreter lock, the famous (or infamous) GIL. It is set up to ensure that (a) only one thread runs at a time, and (b) that the ending of a thread's turn is controlled by the Python interpreter rather than the external event of the hardware timer interrupt.

The fact that the GIL allows only one thread to execute Python bytecode at a time simplifies the Python implementation by making the object model (including critical built-in types such as dict) implicitly safe against concurrent access. Locking the entire interpreter makes it easier for the interpreter to be multi-threaded, at the expense of much of the parallelism afforded by multi-processor machines. The takeaway is that Python does not currently take advantage of multi-processor hardware.



GIL is pronounced "jill", according to Guido__



For a thorough discussion of the GIL and its implications, see <http://www.dabeaz.com/python/UnderstandingGIL.pdf>.

The `threading` Module

- Provides basic threading services
- Also provides locks, events, and other tools
- Three ways to use threads
 - Instantiate `Thread` with a function
 - Subclass `Thread`
 - Use thread pool from `multiprocessing`

The `threading` module provides basic threading services for Python programs. The usual approach is to subclass `threading.Thread` and define a `run()` method that does the thread's work.

Threads for the impatient

- No class needed (created "behind the scenes")
- For simple applications

For many threading tasks, all you need is a `run()` method and maybe some arguments to pass to it.

For simple tasks, you can just create an instance of `Thread`.

Constructor arguments

`target`

Use the `target` argument to pass in the function for the thread to run.

`args`

Use the `args` argument to pass in a tuple of arguments for the target function.

`name`

Use to set the name of the thread. This is an arbitrary string

`kwargs`

Use `kwargs` to provide a dictionary of named arguments to the target function.



Always pass arguments to `Thread()` by name.

Example

thr_simple.py

```
from threading import Thread, Lock
import random
import time

STDOUT_LOCK = Lock()

def my_task(num): # function to run in each thread
    time.sleep(random.randint(1, 3))
    with STDOUT_LOCK:
        print(f"Hello from thread {num}")

for i in range(16):
    t = Thread(target=my_task, args=(i,)) # create thread
    t.start() # launch thread

print("Done.") # "Done" is printed immediately -- the threads are "in the background"
```

thr_simple.py

```
Done.
Hello from thread 2
Hello from thread 3
Hello from thread 13
Hello from thread 14
Hello from thread 4
Hello from thread 11
Hello from thread 6
Hello from thread 5
Hello from thread 10
Hello from thread 15
Hello from thread 0
Hello from thread 1
Hello from thread 7
Hello from thread 8
Hello from thread 9
Hello from thread 12
```

Subclassing Thread

- Must call base class constructor
- Must define `run()`
- Can implement helper methods

A thread class is a class that starts a thread, and performs some task. Such a class can be repeatedly instantiated, with different parameters, and then started as needed.

The class can be as elaborate as the business logic requires. There are only two rules: the class must call the base class's constructor, and it must define a `run()` method. Other than that, the `run()` method can do pretty much anything it wants to.

The best way to invoke the base class constructor is to use `super().__init__()`.

The `run()` method is invoked when you call the `start()` method on the thread object. The `start()` method does not take any parameters; `run()` has no parameters as well.

Any per-thread arguments can be passed into the constructor when the thread object is created.

Example

thr_subclass.py

```
from threading import Thread, Lock
import random
import time

STDOUT_LOCK = Lock()

class SimpleThread(Thread):
    def __init__(self, num):
        super().__init__() # call base class constructor -- REQUIRED
        self._threadnum = num

    def run(self): # the function that does the work in the thread
        time.sleep(random.randint(1, 3))
        with STDOUT_LOCK:
            print(f"Hello from thread {self._threadnum}")

for i in range(16):
    t = SimpleThread(i) # create the thread
    t.start() # launch the thread

print("Done.")
```

thr_subclass.py

```
Done.
Hello from thread 6
Hello from thread 7
Hello from thread 12
Hello from thread 10
Hello from thread 15
Hello from thread 1
Hello from thread 2
Hello from thread 4
Hello from thread 11
Hello from thread 9
Hello from thread 14
Hello from thread 3
Hello from thread 0
Hello from thread 5
Hello from thread 8
```

Hello from thread 13

Variable sharing

- Variables declared before thread starts are shared
- Variables in the thread function are local

A major difference between ordinary processes and threads how variables are shared.

Each thread has can have its own local variables, just as is the case for any function. However, global variables that existed in the program before a thread was spawned are accessible by the thread.

Write access to global variables should be guarded by locks.

Example

thr_locking.py

```
import threading
import random
import time

WORD_LIST = 'apple banana mango peach papaya cherry lemon watermelon'.split()

MAX_SLEEP_TIME = 3
RESULT_LIST = [] # the threads will append words to this list
RESULT_LIST_LOCK = threading.Lock() # generic locks
STDOUT_LOCK = threading.Lock() # generic locks

class SimpleThread(threading.Thread):
    def __init__(self, word): # thread constructor
        super().__init__() # be sure to call parent constructor
        self._word = word # value is passed into thread for processing

    def run(self): # function invoked by each thread
        time.sleep(random.randint(1, MAX_SLEEP_TIME))

        with STDOUT_LOCK: # acquire lock and release when finished
            print(f"Starting thread {self.ident} with value {self._word}")

        with RESULT_LIST_LOCK: # acquire lock and release when finished
            RESULT_LIST.append(self._word.upper())

all_threads = [] # make list ("pool") of threads (but see Pool later in chapter)
for random_word in WORD_LIST: # inefficiently creating one thread per word...
    t = SimpleThread(random_word) # create thread
    all_threads.append(t) # add thread to "pool"
    t.start() # start thread

print("All threads launched...")

for t in all_threads:
    t.join() # wait for thread to finish

print(RESULT_LIST)
```

thr_locking.py

```
All threads launched...
Starting thread 123145615785984 with value watermelon
Starting thread 123145498259456 with value apple
Starting thread 123145515048960 with value banana
Starting thread 123145565417472 with value papaya
Starting thread 123145582206976 with value cherry
Starting thread 123145598996480 with value lemon
Starting thread 123145531838464 with value mango
Starting thread 123145548627968 with value peach
['WATERMELON', 'APPLE', 'BANANA', 'PAPAYA', 'CHERRY', 'LEMON', 'MANGO', 'PEACH']
```

Thread coordination

- Can't assign to immutable variables
- Use `threading.Event()`
 - `__event__.set()`
 - `__event__.wait()`
 - `__event__.clear()`

Sometimes a thread will need to synchronize with or signal another thread. This can get a little messy when using shared variables.

The `threading` module provides an `Event` object to make this simpler.

An Event object can be either set or cleared. Some other thread can wait for the event to be set, or check to see whether it is set.

This can be useful for setting a timer, or for telling a thread to start processing data.

Example

thr_signal.py

```
from threading import Thread, Event
import time

STOP_TASK = Event()

def do_something():
    for i in range(50):
        if STOP_TASK.is_set():
            break
        print(f'{i}-', end='', flush=True)
        time.sleep(.5)

def interrupt():
    time.sleep(10)
    print("STOPPING!")
    STOP_TASK.set()

if __name__ == "__main__":
    t = Thread(target=interrupt)
    t.start() # start thread, which will set the event 10 seconds later
    do_something() # start function, which will detect the event in about 10 seconds
```

thr_signal.py

```
0-1-2-3-4-5-6-7-8-9-10-11-12-13-14-15-16-17-18-19-STOPPING!
```

Example

thr_sync.py

```
from functools import partial
from threading import Thread, Event, Lock
import time

a_ready = Event()
stdout_lock = Lock()

def pr(*args):
    with stdout_lock:
        print(*args, end='', flush=True)

def divisible_by_n(value, n):
    if value == 0:
        return True
    if (value > 0) and ((value % n) == 0):
        return True
    return False

class ThreadA(Thread):

    def run(self):
        for i in range(1, 50):
            pr(f"A{i}")
            if divisible_by_n(i, 10):
                a_ready.set() # notify b
                pr("/setting/")
            elif divisible_by_n(i, 5):
                a_ready.clear() # stop notifying b
                pr("/clearing/")
            time.sleep(.1)
        pr("/setting/")
        a_ready.set() # notify b and let b finish

class ThreadB(Thread):

    def run(self):
        for i in range(1, 50):
            a_ready.wait() # wait until event is set by ThreadA
            pr(f"B{i}")
            time.sleep(.1)
```

```
t_a = ThreadA()
t_a.start()
t_b = ThreadB()
t_b.start()
t_a.join()
t_b.join()
print()
```

thr_sync.py

```
A1A2A3A4A5/clearing/A6A7A8A9A10/setting/B1A11B2A12B3A13B4A14B5A15/clearing/A16A17A18A19A20/setting/B6A21B7A22B8A23B9A24B10A25/clearing/A26A27A28A29A30/setting/B11B12A31B13A32B14A33B15A34B16A35/clearing/A36A37A38A39A40/setting/B17B18A41A42B19B20A43B21A44B22A45/clearing/A46A47A48A49/setting/B23B24B25B26B27B28B29B30B31B32B33B34B35B36B37B38B39B40B41B42B43B44B45B46B47B48B49
```

Using queues

- Queue contains a list of objects
- Sequence is FIFO
- Worker threads can pull items from the queue
- `queue.Queue` structure has builtin locks

Threaded applications often have some sort of work queue data structure. When a thread becomes free, it will pick up work to do from the queue. When a thread creates a task, it will add that task to the queue.

The queue must be guarded with locks. Python provides the Queue module to take care of all the lock creation, locking and unlocking, and so on, so that you don't have to bother with it.

The `queue` module provides a thread-safe `Queue` object that does not need to be guarded with locks. It is a FIFO sequence of values. You can `.put()` values in one end and `.get()` them from the other.

Example

`thr_queue.py`

```
import random
import queue
from threading import Thread, Lock
import time

NUM_ITEMS = 30000
POOL_SIZE = 128

word_queue = queue.Queue(0) # initialize empty queue

shared_list = []
shared_list_lock = Lock() # create locks
stdout_lock = Lock() # create locks

class RandomWord(): # define callable class to generate words
    def __init__(self):
        with open('../DATA/words.txt') as words_in:
            self._words = [word.rstrip('\n\r') for word in words_in.readlines()]
            self._num_words = len(self._words)

    def __call__(self):
        return self._words[random.randrange(0, self._num_words)]

class Worker(Thread): # worker thread
```



```
def __init__(self): # thread constructor
    Thread.__init__(self)

def run(self): # function invoked by thread
    while True:
        try:
            s1 = word_queue.get(block=False) # get next item from thread
            s2 = s1.upper()
            with shared_list_lock: # acquire lock, then release when done
                shared_list.append(s2)

        except queue.Empty: # when queue is empty, it raises Empty exception
            break

# fill the queue
random_word = RandomWord()
for i in range(NUM_ITEMS):
    w = random_word()
    word_queue.put(w)

start_time = time.ctime()

# populate the threadpool
pool = []
for i in range(PPOOL_SIZE):
    w = Worker() # add thread to pool
    w.start() # launch the thread
    pool.append(w)

for t in pool:
    t.join() # wait for thread to finish

end_time = time.ctime()

print(shared_list[:20])

print(start_time)
print(end_time)
```

thr_queue.py

```
['BROTHERLY', 'MULTIFARIOUSNESS', 'ZINCIFIES', 'PERSPECTIVAL', 'LANKER',  
'DISCOURTEANCING', 'DISABLE', 'FAUTEUIL', 'OUTFOOTS', 'PROTEANS', 'DISCERNIBLE', 'FEAR',  
'REDEVELOPING', 'POPULISMS', 'NURSING', 'CHOWCHOWS', 'PERSECUTES', 'ANDALUSITE',  
'CATACHRESTICAL', 'SULLYING']
```

```
Mon Feb 23 20:10:05 2026
```

```
Mon Feb 23 20:10:06 2026
```

Debugging threaded Programs

- Trickier than non-threaded programs
- Context changes abruptly
- Use `breakpoint()`

Debugging threads can be tricky. If you're used to debugging normal programs, it can be surprising to suddenly jump into the code of a different thread. Debugging deadlocks can be very hard.

Another problem which sometimes occurs is that if you issue a **next** command in your debugging tool, you may end up inside the internal threads code. In such cases, use a **continue** command to get back to your code.

The basic PDB debugger may not work for debugging like this:

```
pdb.py mythreadedprogram.py
```

This is because threads will not inherit the `pdb` process from the main thread. While you can run the debugger on the main program, you won't be able to set breakpoints in threads.

To get around this issue, use `breakpoint()` in your actual code; this builtin function will invoke PDB when it is called. It is a way of setting breakpoints in your code.

When using `breakpoint()`, execute your program normally, not with `pdb`, and it will stop at the first instance of `breakpoint()` and invoke `pdb`.



Prior to version 3.7, import `pdb` and use `pdb.set_trace()` to get the same behavior as `breakpoint()`

The multiprocessing module

- Drop-in replacement for the threading module
- Doesn't suffer from GIL issues
- Provides interprocess communication
- Provides process (and thread) pooling

The `multiprocessing` module can be used as a replacement for `threading`. It uses processes rather than threads to spread out the work to be done. While the entire module doesn't use the same API as `threading`, `multiprocessing.Process` object is a drop-in replacement for `threading.Thread`. Both use `.run()` as the overridable method that does the work, and both use `.start()` to launch. The syntax is the same to create a process without using a class:

```
def myfunc(filename):  
    pass  
  
p = Process(target=myfunc, args=('/tmp/info.dat', ))
```

This solves the GIL issue, but the trade-off is that it's a little slower, and slightly more complicated for tasks (processes) to communicate. However, the module does the heavy lifting of creating pipes to share data.

The `Manager` class provided by `multiprocessing` allows you to create shared variables, as well as locks for them, which work across processes.



On Windows, processes must be started in the `if {dunder}name{dunder} == {dunder}main{dunder}:` block, or they will not work.

Example

`multi_processing.py`

```
import sys  
import random  
from multiprocessing import Manager, Lock, Process, Queue, freeze_support  
from queue import Empty  
import time  
  
NUM_ITEMS = 25000 # set some constants  
POOL_SIZE = 64
```

```

class RandomWord(): # callable class to provide random words
    def __init__(self):
        with open('../DATA/words.txt') as words_in:
            self._words = [word.rstrip('\n\r') for word in words_in]
            self._num_words = len(self._words)

    def __call__(self): # will be called when you call an instance of the class
        return self._words[random.randrange(0, self._num_words)]

class Worker(Process): # worker class -- inherits from Process

    def __init__(self, name, queue, lock, result): # initialize worker process
        Process.__init__(self)
        self.queue = queue
        self.result = result
        self.lock = lock
        self.name = name

    def run(self): # do some work -- will be called when process starts
        while True:
            try:
                word = self.queue.get(block=False) # get data from the queue
                word = word.upper() # modify data
                with self.lock:
                    self.result.append(word) # add to shared result

            except Empty: # quit when there is no more data in the queue
                break

if __name__ == '__main__':
    if sys.platform == 'win32':
        freeze_support()

    word_queue = Queue() # create empty Queue object

    manager = Manager() # create manager for shared data
    shared_result = manager.list() # create list-like object to be shared across all
    processes
    result_lock = Lock() # create locks

    random_word = RandomWord() # create callable RandomWord instance
    for i in range(NUM_ITEMS):
        w = random_word()
        word_queue.put(w) # fill the queue

```

```

start_time = time.ctime()

pool = [] # create empty list to hold processes
for i in range(PPOOL_SIZE): # populate the process pool
    worker_name = f"Worker {i:03d}"
    w = Worker(worker_name, word_queue, result_lock, shared_result) # create worker
process
    #
    w.start() # actually start the process -- note: in Windows, should only call
X.start() from main(), and may not work inside an IDE
    pool.append(w) # add process to pool

for t in pool:
    t.join() # wait for each queue to finish

end_time = time.ctime()

print((shared_result[-50:])) # print last 50 entries in shared result
print(len(shared_result))
print(start_time)
print(end_time)

```

multi_processing.py

```

['CONSOLIDATORS', 'FILMERS', 'INEQUALITIES', 'JOCOSITY', 'LONGED', 'SEAWAYS',
'COLONIZATIONIST', 'PRODIGALITIES', 'MILITIA', 'PIGGYBACKED', 'TOXIN', 'OPPOSABILITIES',
'CONTRACTIVE', 'CHAULMOOGRA', 'ANGIOCARDIOGRAPHY', 'SEXES', 'OVERTONE', 'ZLOTYCH',
'SUBPARTS', 'GUMMOSES', 'POKEY', 'CRITERION', 'BACKBEND', 'IMPREGNABLENESS',
'INVASIVENESS', 'SUBLATION', 'RAISES', 'INGLES', 'BRASSERIE', 'KREUZER', 'KNEW',
'STOVER', 'MONOSOMIC', 'NIDERINGS', 'SALESGIRL', 'BULGES', 'BHOOOTS', 'PLANCHETS',
'SELLOUT', 'RETEACHING', 'DIAPHONES', 'CREW', 'FURANE', 'INTERPOSER', 'SOROSIS', 'PILUS',
'NONSTEROID', 'BRANNIER', 'SENGI', 'SPALLATIONS']
25000
Mon Feb 23 20:10:06 2026
Mon Feb 23 20:10:11 2026

```

Using pools

- Provided by `multiprocessing` and `multiprocessing.dummy`
- Both thread and process pools
- Simplifies multiprocessing tasks

For many multiprocessing tasks, you want to process a list (or other iterable) of data and do something with the results. This is easily accomplished with the `Pool` class provided by the `multiprocessing` module.

This object creates a pool of n processes. Call the `.map()` method with a function that will do the work, and an iterable of data. `.map()` will return a list of results in the same order as the original data.

For a thread pool, import `Pool` from `multiprocessing.dummy`. It works exactly the same, but creates threads.

Example

proc_pool.py

```
import random
from multiprocessing import Pool

POOL_SIZE = 32 # number of processes

with open('../DATA/words.txt') as words_in:
    WORDS = [w.strip() for w in words_in] # read word file into a list, stripping off

random.shuffle(WORDS) # randomize word list

def my_task(word): # actual task
    return word.upper()

if __name__ == '__main__':
    ppool = Pool(POOL_SIZE) # create pool of POOL_SIZE processes

    WORD_LIST = ppool.map(my_task, WORDS) # pass wordlist to pool and get results; map
    assigns values from input list to processes as needed

    print(WORD_LIST[:20]) # print last 20 words

    print(f"Processed {len(WORD_LIST)} words.")
```

proc_pool.py

```
['ODS', 'PIPAGE', 'RAZORS', 'UNHALLOWS', 'RETAKING', 'BEAMLESS', 'DEWAR', 'TINKLED',
'RELATEDNESS', 'NUCLEAL', 'CYCLOPEDIAS', 'WHEREVE', 'POLYMERIZE', 'CAVICORN', 'CERITE',
'HALAKISTS', 'DERAILED', 'SILESIA', 'ONYXES', 'ACETATED']
Processed 173462 words.
```


Example

thr_pool.py

```
from multiprocessing.dummy import Pool # get the thread pool object

POOL_SIZE = 32 # set # of threads to create

with open('../DATA/words.txt') as words_in:
    WORDS = [w.strip() for w in words_in] # get list of 175K words

def my_task(word): # function to apply to each element
    return word.upper()

thread_pool = Pool(POOL_SIZE) # create pool

word_list = thread_pool.map(my_task, WORDS) # map elements across all threads

print(word_list[:20])

print(f"Processed {len(word_list)} words.")
```

thr_pool.py

```
['AA', 'AAH', 'AAHED', 'AAHING', 'AAHS', 'AAL', 'AALII', 'AALIIS', 'AALS', 'AARDVARK',
'AARDVARKS', 'AARDWOLF', 'AARDWOLVES', 'AARGH', 'AARRGH', 'AARRGHH', 'AAS', 'AASVOGEL',
'AASVOGELS', 'AB']
Processed 173462 words.
```

Example

thr_pool_mw.py

```

from multiprocessing.dummy import Pool # .dummy has Pool for threads
import requests
import time

POOL_SIZE = 8

BASE_URL = 'https://www.dictionaryapi.com/api/v3/references/collegiate/json/' # base url
of site to access

with open('dictionaryapikey.txt') as api_key_in:
    API_KEY = api_key_in.read().rstrip() # get credentials

SEARCH_TERMS = [ # terms to search for; each thread will search some of these terms
    'wombat', 'pine marten', 'python', 'pearl',
    'sea', 'formula', 'translation', 'common',
    'business', 'frog', 'muntin', 'automobile',
    'green', 'connect', 'vial', 'battery', 'computer',
    'sing', 'park', 'ladle', 'ram', 'dog', 'scalpel',
    'emulsion', 'noodle', 'combo', 'battery'
]

def main():
    total_times = {}
    for function in get_data_threaded, get_data_serial:
        start_time = time.perf_counter()
        results = function()
        for search_term, result in zip(SEARCH_TERMS, results): # iterate over results,
            mapping them to search terms
                print(search_term.upper(), end=": ")
                if result:
                    print(result)
                else:
                    print("** no results **")
        total_times[function.__name__] = time.perf_counter() - start_time
        print('-' * 60)

    for function_name, elapsed_time in total_times.items():
        print(f"{function_name} took {elapsed_time:.2f} seconds")

def fetch_data(term): # function invoked by each thread for each item in list passed to
map()
    try:
        response = requests.get(

```

```

        BASE_URL + term,
        params={'key': API_KEY},
    ) # make the request to the site
except requests.HTTPError as err:
    print(err)
    return []
else:
    data = response.json() # convert JSON to Python structure
    parts_of_speech = []
    for entry in data: # loop over entries matching search terms
        if isinstance(entry, dict):
            meta = entry.get("meta")
            if meta:
                part_of_speech = entry.get("fl")
                if part_of_speech:
                    parts_of_speech.append(part_of_speech)
    return sorted(set(parts_of_speech)) # return list of parsed entries matching
search term

def get_data_threaded():
    p = Pool(PPOOL_SIZE) # create pool of PPOOL_SIZE threads
    return p.map(fetch_data, SEARCH_TERMS) # launch threads, collect results

def get_data_serial():
    return [fetch_data(w) for w in SEARCH_TERMS]

if __name__ == '__main__':
    main()

```

...

Alternatives to `PoOL.map()`

- map elements of iterable to multiple task function arguments
- map elements asynchronously
- apply task function to a single value
- apply task function to a single value asynchronously

There are some alternative methods to `__Pool__.map()`. These apply functions to the data in different patterns, and can be run asynchronously as well.

Table 33. Pool methods

method	multiple args	concurrent	blocks until done	results ordered
<code>map()</code>	no	yes	yes	yes
<code>apply()</code>	yes	no	yes	no
<code>map_async()</code>	no	yes	no	yes
<code>apply_async()</code>	yes	yes	no	no

Alternatives to threading and multiprocessing

- `asyncio`
- `Twisted`

Threading and forking are not the only ways to have your program do more than one thing at a time. Another approach is asynchronous programming. This technique putting events (typically I/O events) in a list, or queue, and starting an event loop that processes the events one at a time. If the granularity of the event loop is small, this can be as efficient as multiprocessing.

Async

Asynchronous programming is only useful for improving I/O throughput, such as networking clients and servers, or scouring a file system. Like threading (in Python), it will not help with raw computation speed.

The `asyncio` module in the standard library provides the means to write asynchronous clients and servers.

Twisted

The **Twisted** framework is a large and well-supported third-party module that provides support for many kinds of asynchronous communication. It has prebuilt objects for servers, clients, and protocols, as well as tools for authentication, translation, and many others. Find Twisted at twistedmatrix.com/trac.



See the files named `consume_omdb*.py` and `omdblib.py` in EXAMPLES for examples comparing single-threaded, multi-threaded, multi-processing, and async versions of the same program. There are also examples using `concurrent.futures`, an alternate interface for creating thread or process pools.

Chapter 15 Exercises

For each exercise, ask the questions: Should this be multi-threaded or multi-processed? Distributed or local?

Exercise 15-1 (apod.py, apod_downloads.py)

Background

NASA provides many APIs for downloading astronomical images and information. One of these is the APOD (Astronomy Picture Of the Day).

The `apod` module in the root folder provides a function named `fetch_apod()` to fetch one APOD by date. The format for the date is `YYYY-MM-DD`. It downloads the image and saves it to a local file. The function returns `True` for a successful download, `False` otherwise. Some dates will return `False` if the APOD is not an image (it might be a **Youtube** link) or if the request times out. You can ignore those issues.

The `apod` module uses a demo-only API key which has usage restrictions. If you just want to run the script to see what it does, the demo key is sufficient. For writing your own script, go to <https://api.nasa.gov/index.html#signUp> to get a personal API key. The only personal information it requires is an email address. Replace "DEMO_KEY" with your personal API key in the module.

Exercise

Start with the existing script `apod_downloads.py` in the root folder. This script uses the `apod` module to download the NASA APOD for the each day of each January 2023. Note how long it takes as written.

Update the script to be concurrent using a thread pool. Put all the code in the `main()` function.

Compare the speed of the original to the concurrent version.

Exercise 15-2 (folder_scanner.py)

Write a program that takes in a directory name on the command line, then traverses all the files in that directory tree and prints out a count of:

- how many total files
- how many total lines (count `\n`)
- how many bytes (`len()` of file contents)

HINT: Use either a thread or a process pool in combination with `os.walk()`.

Appendix A: Field Guide to Python Expressions

Table 34. Python Expressions

Expression	Meaning
<code>a, b, c</code> <code>(a, b, c)</code>	tuple
<code>a,</code> <code>(a,)</code>	tuple with one element
<code>()</code>	empty tuple
<code>[a, b, c]</code>	list
<code>[]</code>	empty list
<code>{a, b, c}</code>	set
<code>{a:r, b:s, c:t}</code>	dictionary
<code>{}</code>	empty dictionary
<code>[expr for var in iterable if condition]</code>	list comprehension
<code>(expr for var in iterable if condition)</code>	generator expression
<code>{expr for var in iterable if condition}</code>	set comprehension
<code>{key:value for var in iterable if condition}</code>	dictionary comprehension
<code>a, b, c = iterable</code>	iterable unpacking
<code>a, *b, c = iterable</code>	extended iterable unpacking
<code>a if b else c</code>	conditional expression
<code>lambda VAR ...: VALUE</code>	lambda function

Appendix B: String Formatting

Overview

- Strings have a `format()` method
- Allows values to be inserted in strings
- Values can be formatted
- Add a field as placeholders for variable
- Field syntax: `{SELECTOR:FORMATTING}`
- Selector can be empty, index, or keyword
- Formatting controls alignment, width, padding, etc.

Python provides a powerful and flexible way to format data. The string method `format()` takes one or more parameters, which are inserted into the string via placeholders.

The placeholders, called fields, consist of a pair of braces enclosing parameter selectors and formatting directives.

The selector can be followed by a set of formatting directives, which always start with a colon. The simplest directives specify the type of variable to be formatted. For instance, `{1:d}` says to format the second parameter as an integer; `{0:.2f}` says to format the first parameter as a float, rounded to two decimal points.

The formatting part can consist of the following components, which will be explained in detail in the following pages:

```
: [[fill]align][sign][#][0][width][,][.precision][type]
```


Parameter Selectors

- Null for auto-numbering
- Can be numbers or keywords
- Start at 0 for numbers

Selectors refer to which parameter will be used in a placeholder.

Null (empty) selectors — the most common — will be treated as though they were filled in with numbers from left to right, beginning with 0. Null selectors cannot be mixed with numbered or named selectors — either all of the selectors or none of the selectors must be null.

Non-null selectors can be either numeric indices or keywords (strings). Thus, {0} will be replaced with the first parameter, {4} will be replaced with the fifth parameter, and so on. If using keywords, then {name} will be replaced by the value of keyword *name*, and {age} will be replaced by keyword *age*.

Parameters do not have to be in the same order in which they occur in the string, although they typically are. The same parameter can be used in multiple fields.

If positional and keyword parameters are both used, the keyword parameters must come after all positional parameters.

Example

fmt_params.py

```
person = 'Bob'  
age = 22  
  
print(f"{person} is {age} years old.")  
print()
```

fmt_params.py

```
Bob is 22 years old.
```

f-strings

- **f** in front of literal strings
- More readable
- Same rules as `__string__.format()`

Starting with version 3.6, Python also supports *f-strings*.

The big difference from the `format()` method is that the parameters are inside the `{}` placeholders. Place formatting details after a `:` as usual.

Since the parameters are part of the placeholders, parameter numbers are not used.

All of the following formatting tools work with both `__string__.format()` and f-strings.

Example

fmt_fstrings.py

```
person = 'Bob'
age = 22
result = 39.128935

print(f"{person} is {age} years old.")
print(f"The {age}-year-old is {person}.")
print(f"Result is {result:.2f}")
print()
```

fmt_fstrings.py

```
Bob is 22 years old.
The 22-year-old is Bob.
Result is 39.13
```

Data types

- Fields can specify data type
- Controls formatting
- Raises error for invalid types

The type part of the format directive tells the formatter how to convert the value. Builtin types have default formats – *s* for strings, *d* for integers, *f* for float.

Some data types can be specified as either upper or lower case. This controls the output of letters. E.g, `{:x}` would format the number 48879 as *beef*, but `{:X}` would format it as *BEEF*.

The type must generally match the type of the parameter. An integer cannot be formatted with type *s*. Integers can be formatted as floats, but not the other way around. Only integers may be formatted as binary, octal, or hexadecimal.

Example

fmt_types.py

```
person = 'Bob'
value = 488
result = .5617282027

print(f'{person:s}')    # String
print(f'{value:d}')     # Integer (displayed as decimal)
print(f'{value:b}')     # Integer (displayed as binary)
print(f'{value:o}')     # Integer (displayed as octal)
print(f'{value:x}')     # Integer (displayed as hex)
print(f'{value:X}')     # Integer (displayed as hex with uppercase digits)
print(f'{result:f}')    # Float (defaults to 6 places after the decimal point)
print(f'{result:.2f}')  # Float rounded to 2 decimal places
print(f'{result:.2%}')  # Float converted to percent and rounded to 2 decimal places
```

fmt_types.py

```

Bob
488
111101000
750
1e8
1E8
0.561728
0.56
56.17%

```

Table 35. Formatting Types

Type	Description
b	Binary – converts number to base 2
c	Character – converts to corresponding character, like chr()
d	Decimal – outputs number in base 10
e, E	Exponent notation. <i>e</i> prints the number in scientific notation using the letter <i>e</i> to indicate the exponent. <i>E</i> is the same, except it uses the letter <i>E</i>
f, F	Floating point. <i>F</i> and <i>f</i> are the same.
g	General format. For a given precision <i>p</i> ≥ 1 , rounds the number to <i>p</i> significant digits and then formats the result in fixed-point or scientific notation, depending on magnitude. This is the default for numbers
G	Same as <i>g</i> , but upper-cases <i>e</i> , <i>nan</i> , and 'inf'
n	Same as <i>d</i> , but uses locale setting for number separators
o	Octal – converts number to base 8
s	String format. This is the default type for strings
x, X	Hexadecimal – convert number to base 16; A-F match case of <i>x</i> or <i>X</i>
%	Percentage. Multiplies the number by 100 and displays in fixed (<i>f</i>) format, followed by a percent sign.

Field Widths

- Specified as {0:width.precision}
- Width is really minimum width
- Precision is either maximum width or # decimal points

Fields can specify a minimum width by putting a number before the type. If the parameter is shorted than the field, it will be padded with spaces, on the left for numbers, and on the right for strings.

The precision is specified by a period followed by an integer. For strings, precision means the maximum width. Strings longer than the maximum will be truncated. For floating point numbers, precision means the number of decimal places displayed, which will be padded with zeros as needed.

Width and precision are both optional. The default width for all fields is 0; the default precision for floating point numbers is 6.

It is invalid to specify precision for an integer.

Example

fmt_width.py

```
name = 'Ann Elk'
value = 10000
airspeed = 22.347
# note: [] are used to show blank space, and are not part of the formatting
print(f'[{name:s}]')      # Default format -- no padding
print(f'[{name:10s}]')    # Left justify, 10 characters wide
print(f'[{name:3s}]')     # Left justify, 3 characters wide, displays entire string
print(f'[{name:3.3s}]')   # Left justify, 3 characters wide, truncates string to max width
print()
print(f'[{value:8d}]')     # Right justify, decimal, 8 characters wide (all numbers are
right-justified by default)
print(f'[{value:8f}]')     # Right justify int as float, 8 characters wide
print(f'[{airspeed:8f}]')  # Right justify float as float, 8 characters wide
print(f'[{airspeed:.2f}]') # Right justify, float, 3 decimal places, no maximum width
print(f'[{airspeed:8.3f}]') # Right justify, float, 3 decimal places, maximum width 8
```


fmt_width.py

```
[Ann Elk]  
[Ann Elk  ]  
[Ann Elk]  
[Ann]  
  
[ 10000]  
[10000.000000]  
[22.347000]  
[22.35]  
[ 22.347]
```

Alignment

- Alignment within field can be left, right, or centered
 - < left align
 - > right align
 - ^ center
 - = right align but put padding after sign

You can align the data to be formatted. It can be left-aligned (the default), right-aligned, or centered. If formatting signed numbers, the minus sign can be placed on the left side.

Example

fmt_align.py

```
name = 'Ann'
value = 12345
nvalue = -12345

# note: all of the following print in a field 10 characters wide
print('{0:10s}'.format(name))    # Default (left) alignment
print('{0:<10s}'.format(name))  # Explicit left alignment
print('{0:>10s}'.format(name))  # Right alignment
print('{0:^10s}'.format(name))  # Centered
print()
print('{0:10d} [{1:10d}]'.format(value, nvalue))  # Default (right) alignment
print('{0:>10d} [{1:>10d}]'.format(value, nvalue)) # Explicit right alignment
print('{0:<10d} [{1:<10d}]'.format(value, nvalue)) # Left alignment
print('{0:^10d} [{1:^10d}]'.format(value, nvalue)) # Centered
print('{0:=10d} [{1:=10d}]'.format(value, nvalue)) # Right alignment, but pad _after_
sign
```

fmt_align.py

```
[Ann      ]
[Ann      ]
[      Ann]
[  Ann    ]

[   12345] [  -12345]
[   12345] [  -12345]
[12345    ] [-12345   ]
[ 12345   ] [ -12345  ]
[   12345] [-   12345]
```

Fill characters

- Padding character must precede alignment character
- Default is one space
- Can be any character except }

By default, if a field width is specified and the data does not fill the field, it is padded with spaces. A character preceding the alignment character will be used as the fill character.

Example

fmt_fill.py

```
name = 'Ann'
value = 123

print('{:>10s}'.format(name))    # Right justify string, pad with space (default)
print('{:.>10s}'.format(name))    # Right justify string, pad with '.'
print('{:->10s}'.format(name))    # Right justify string, pad with '-'
print('{:~10s}'.format(name))    # Left justify string, pad with '~'
print()
print('{:10d}'.format(value))    # Right justify number, pad with space (default)
print('{:010d}'.format(value))    # Right justify number, pad with zeroes
print('{:_>10d}'.format(value))    # Right justify, pad with '_' ('>' required)
print('{:~>10d}'.format(value))    # Right justify, pad with '~' ('>' required)
```

fmt_fill.py

```
[      Ann]
[.....Ann]
[-----Ann]
[Ann]

[      123]
[000000123]
[_____123]
[+++++++123]
```

Signed numbers

- Can pad with any character except {}
- Sign can be +, -, or space
- Only appropriate for numeric types

The sign character follows the alignment character, and can be plus, minus, or space.

A plus sign means always display + or – preceding non-zero numbers.

A minus sign means only display a sign for negative numbers.

A space means display a – for negative numbers and a space for positive numbers.

Example

fmt_signed.py

```
values = 123, -321, 14, -2, 0

for value in values:
    print(f"default: |{value:4d}|") # default (pipe symbols just to show white space)
print()

for value in values:
    print(f"  plus: |{value:+4d}|") # plus sign puts '+' on positive numbers (and zero)
    and '-' on negative
print()

for value in values:
    print(f" minus: |{value:-4d}|") # minus sign only puts '-' on negative numbers
print()

for value in values:
    print(f" space: |{value: d}|") # space puts '-' on negative numbers and space on
others
print()
```


fmt_signed.py

```
default: | 123|  
default: |-321|  
default: | 14|  
default: | -2|  
default: | 0|
```

```
plus: |+123|  
plus: |-321|  
plus: | +14|  
plus: | -2|  
plus: | +0|
```

```
minus: | 123|  
minus: |-321|  
minus: | 14|  
minus: | -2|  
minus: | 0|
```

```
space: | 123|  
space: |-321|  
space: | 14|  
space: |-2|  
space: | 0|
```

Parameter Attributes

- Specify elements or properties in template
- No need to repeat parameters
- Works with sequences, mappings, and objects

When specifying container variables as parameters, you can select elements in the format rather than in the parameter list. For sequences or dictionaries, index on the selector with []. For object attributes, access the attribute from the selector with . (period).

Example

fmt_attr.py

```
from datetime import date

fruits = 'apple', 'banana', 'mango'
values = [5, 18, 27, 6]
dday = date(1944, 6, 6)
pythons = {'Idle': 'Eric', 'Cleese': 'John', 'Gilliam': 'Terry',
           'Chapman': 'Graham', 'Palin': 'Michael', 'Jones': 'Terry'}

print('{0[0]} {0[2]}'.format(fruits)) # select from tuple
print('{f[0]} {f[2]}'.format(f=fruits)) # named parameter + select from tuple
print()
print('{0[0]} {0[2]}'.format(values)) # Select from list
print()
print('{0[Palin]} {0[Cleese]}'.format(pythons)) # select from dict
print('{names[Palin]} {names[Cleese]}'.format(names=pythons)) # named parameter + select
from dict
print()
print('{0.month}-{0.day}-{0.year}'.format(dday)) # select attributes from date
```

fmt_attrib.py

```
apple mango  
apple mango
```

```
5 27
```

```
Michael John  
Michael John
```

```
6-6-1944
```

Formatting Dates

- Special formats for dates
- Pull appropriate values from date/time objects

To format dates, use special date formats. These are placed, like all formatting codes, after a colon. For instance, `{0:%B %d, %Y}` will format a parameter (which must be a `datetime.datetime` or `datetime.date`) as "Month DD, YYYY".

Example

fmt_dates.py

```
from datetime import datetime

event = datetime(2016, 1, 2, 3, 4, 5)

print(event) # Default string version of date
print()

print("Date is {0:%m}/{0:%d}/{0:%y}".format(event)) # Use three placeholders for month,
day, year
print("Date is {:%m/%d/%y}".format(event)) # Format month, day, year with a single
placeholder
print("Date is {:%A, %B %d, %Y}".format(event)) # Another single placeholder format
```

fmt_dates.py

```
2016-01-02 03:04:05
```

```
Date is 01/02/16
```

```
Date is 01/02/16
```

```
Date is Saturday, January 02, 2016
```

Table 36. Date Formats

Directive	Meaning	See note
%a	Locale's abbreviated weekday name.	
%A	Locale's full weekday name.	
%b	Locale's abbreviated month name.	
%B	Locale's full month name.	
%c	Locale's appropriate date and time representation.	
%d	Day of the month as a decimal number [01,31].	
%f	Microsecond as a decimal number [0,999999], zero-padded on the left	1
%H	Hour (24-hour clock) as a decimal number [00,23].	
%I	Hour (12-hour clock) as a decimal number [01,12].	
%j	Day of the year as a decimal number [001,366].	
%m	Month as a decimal number [01,12].	
%M	Minute as a decimal number [00,59].	
%p	Locale's equivalent of either AM or PM.	2
%S	Second as a decimal number [00,61].	3
%U	Week number of the year (Sunday as the first day of the week) as a decimal number [00,53]. All days in a new year preceding the first Sunday are considered to be in week 0.	4
%w	Weekday as a decimal number [0(Sunday),6].	
%W	Week number of the year (Monday as the first day of the week) as a decimal number [00,53]. All days in a new year preceding the first Monday are considered to be in week 0.	4
%x	Locale's appropriate date representation.	
%X	Locale's appropriate time representation.	
%y	Year without century as a decimal number [00,99].	
%Y	Year with century as a decimal number.	
%z	UTC offset in the form +HHMM or -HHMM (empty string if the the object is naive).	5
%Z	Time zone name (empty string if the object is naive).	
%%	A literal % character.	

1. When used with the `strptime()` method, the `%f` directive accepts from one to six digits and zero pads on the right. `%f` is an extension to the set of format characters in the C standard (but implemented separately in datetime objects, and therefore always available).
2. When used with the `strptime()` method, the `%p` directive only affects the output hour field if the `%I` directive is used to parse the hour.
3. The range really is 0 to 61; according to the Posix standard this accounts for leap seconds and the (very rare) double leap seconds. The time module may produce and does accept leap seconds since it is based on the Posix standard, but the datetime module does not accept leap seconds in `strptime()` input nor will it produce them in `strftime()` output.
4. When used with the `strptime()` method, `%U` and `%W` are only used in calculations when the day of the week and the year are specified.
5. For example, if `utcoffset()` returns `timedelta(hours=-3, minutes=-30)`, `%z` is replaced with the string `-0330`.

Run-time formatting

- Use parameters to specify alignment, precision, width, and type
- Use {} placeholders for runtime values for the above

To specify formatting values at runtime, use a {} placeholder for the value, and insert the desired value in the parameter list. These placeholders are numbered along with the normal placeholders.

Example

fmt_runtime.py

```
FIRST_NAME = 'Fred'
LAST_NAME = 'Flintstone'
AGE = 35

print(f"{FIRST_NAME} {LAST_NAME}")

WIDTH = 12
print(f"[{FIRST_NAME:{WIDTH}s}] [{LAST_NAME:{WIDTH}s}]") # value of WIDTH used in format spec

PAD = '-'
WIDTH = 16
ALIGNMENTS = ('<', '>', '^')

for ALIGNMENT in ALIGNMENTS:
    print(f"{FIRST_NAME:{PAD}{ALIGNMENT}{WIDTH}s} {LAST_NAME:{PAD}{ALIGNMENT}{WIDTH}s}")
# values of PAD, WIDTH, ALIGNMENTS used in format spec
```

fmt_runtime.py

```
Fred Flintstone
[Fred      ] [Flintstone  ]
Fred----- Flintstone-----
-----Fred -----Flintstone
-----Fred----- ---Flintstone---
```

Miscellaneous tips and tricks

- Adding commas to large numbers {n:,}
- Auto-converting parameters to strings (!s)
- Non-decimal prefixes

- Adding commas to large numbers {n:,}

You can add a comma to the format to add commas to numbers greater than 999.

Using a format type of !s will call str() on the parameter and force it to be a string.

Using a # (pound sign) will cause binary, octal, or hex output to be preceded by 0b, 0o, or 0x. This is only valid with type codes b, o, and x.

Example

fmt_misc.py

```
'''Demonstrate misc formatting'''

big_number = 2303902390239

print(f"Big number: {big_number:,d}") # Add commas for readability
print()

value = 27

print(f"Binary: {value:#010b}") # Binary format with leading 0b
print(f"Octal: {value:#010o}") # Octal format with leading 0o
print(f"Hex: {value:#010x}") # Hexadecimal format with leading 0x
print()
```

fmt_misc.py

Big number: 2,303,902,390,239

Binary: 0b00011011

Octal: 0o00000033

Hex: 0x0000001b

Appendix C: Python Bibliography

Data Science

- ***Building machine learning systems with Python***. William Richert, Luis Pedro Coelho. Packt Publishing
- ***High Performance Python***. Mischa Gorlelick and Ian Ozsvald. O'Reilly Media
- ***Introduction to Machine Learning with Python***. Sarah Guido. O'Reilly & Assoc.
- ***iPython Interactive Computing and Visualization Cookbook***. Cyril Rossant. Packt Publishing
- ***Learning iPython for Interactive Computing and Visualization***. Cyril Rossant. Packt Publishing
- ***Learning Pandas***. Michael Heydt. Packt Publishing
- ***Learning scikit-learn: Machine Learning in Python***. Raúl Garreta, Guillermo Moncecchi. Packt Publishing
- ***Mastering Machine Learning with Scikit-learn***. Gavin Hackeling. Packt Publishing
- ***Matplotlib for Python Developers***. Sandro Tosi. Packt Publishing
- ***Numpy Beginner's Guide***. Ivan Idris. Packt Publishing
- ***Numpy Cookbook***. Ivan Idris. Packt Publishing
- ***Practical Data Science Cookbook***. Tony Ojeda, Sean Patrick Murphy, Benjamin Bengfort, Abhijit Dasgupta. Packt Publishing
- ***Python Text Processing with NLTK 2.0 Cookbook***. Jacob Perkins. Packt Publishing
- ***Scikit-learn cookbook***. Trent Hauck. Packt Publishing
- ***Python Data Visualization Cookbook***. Igor Milovanovic. Packt Publishing
- ***Python for Data Analysis***. Wes McKinney. O'Reilly & Assoc

Design Patterns

- ***Design Patterns: Elements of Reusable Object-Oriented Software***. Erich Gamma, Richard Helm, Ralph Johnson, John Vlissides. Addison-Wesley Professional
- ***Head First Design Patterns***. Eric Freeman, Elisabeth Robson, Bert Bates, Kathy Sierra. O'Reilly Media
- ***Learning Python Design Patterns***. Gennadiy Zlobin. Packt Publishing
- ***Mastering Python Design Patterns***. Sakis Kasampalis. Packt Publishing

General Python development

- ***Expert Python Programming***. Tarek Ziadé. Packt Publishing
- ***Fluent Python***. Luciano Ramalho. O'Reilly & Assoc.

- ***Learning Python, 2nd Ed..Mark Lutz, David Asher.*** O'Reilly & Assoc.
- ***Mastering Object-oriented Python.Stephen F. Lott.*** Packt Publishing
- ***Programming Python, 2nd Ed. .Mark Lutz.*** O'Reilly & Assoc.
- ***Python 3 Object Oriented Programming.Dusty Phillips.*** Packt Publishing
- ***Python Cookbook, 3rd. Ed.. David Beazley, Brian K. Jones.*** O'Reilly & Assoc.
- ***Python Essential Reference, 4th. Ed..David M. Beazley.*** Addison-Wesley Professional
- ***Python in a Nutshell.Alex Martelli.*** O'Reilly & Assoc.
- ***Python Programming on Win32.Mark Hammond, Andy Robinson.*** O'Reilly & Assoc.
- ***The Python Standard Library By Example.Doug Hellmann.*** Addison-Wesley Professional

Misc

- ***Python Geospatial Development.Erik Westra.*** Packt Publishing
- ***Python High Performance Programming.Gabriele Lanaro.*** Packt Publishing

Networking

- ***Python Network Programming Cookbook.Dr. M. O. Faruque Sarker.*** Packt Publishing
- ***Violent Python: A Cookbook for Hackers, Forensic Analysts, Penetration Testers and Security Engineers.TJ O'Connor.*** Syngress
- ***Web Scraping with Python.Ryan Mitchell.*** O'Reilly & Assoc.

Testing

- ***Python Testing Cookbook.Greg L. Turnquist.*** Packt Publishing
- ***Learning Python Testing.Daniel Arbuckle.*** Packt Publishing
- ***Learning Selenium Testing Tools, 3rd Ed. .Raghavendra Prasad MG.*** Packt Publishing

Web Development

- ***Building Web Applications with Flask.Italo Maia.*** Packt Publishing
- ***Django 1.0 Website Development.Ayman Hourieh.*** Packt Publishing
- ***Django 1.1 Testing and Development.Karen M. Tracey.*** Packt Publishing
- ***Django By Example.Antonio Melé.*** Packt Publishing
- ***Django Design Patterns and Best Practices.Arun Ravindran.*** Packt Publishing
- ***Django Essentials.Samuel Dauzon.*** Packt Publishing

- ***Django Project Blueprints***.**Asad Jibran Ahmed**.Packt Publishing
- ***Flask Blueprints***.**Joel Perras**.Packt Publishing
- ***Flask by Example***.**Gareth Dwyer**.Packt Publishing
- ***Flask Framework Cookbook***.**Shalabh Aggarwal**.Packt Publishing
- ***Flask Web Development***.**Miguel Grinberg**. O'Reilly & Assoc.
- ***Full Stack Python (e-book only)***.**Matt Makai**.Gumroad (or free download)
- ***Full Stack Python Guide to Deployments (e-book only)***.**Matt Makai**.Gumroad (or free download)
- ***High Performance Django***.**Peter Baumgartner, Yann Malet**.Lincoln Loop
- ***Instant Flask Web Development***.**Ron DuPlain**.Packt Publishing
- ***Learning Flask Framework***.**Matt Copperwaite, Charles O Leifer**.Packt Publishing
- ***Mastering Flask***.**Jack Stouffer**.Packt Publishing
- ***Two Scoops of Django 3.X: Best Practices for the Django Web Framework***. **Daniel Roy Greenfeld, Audrey Roy Greenfeld**.Two Scoops Press
- ***Web Development with Django Cookbook***.**Aidas Bendoraitis**.Packt Publishing

Index

@

@pytest.mark.mark, 64

`__iter__()`, 173

`__next__()`, 173

`io`, 213, 217, 223, 230, 240, 309, 312, 332, 332, 332, 332, 344, 348, 382, 384, 384, 387, 387, 387, 392, 392, 395, 395, 396, 396, 421, 430

`io` module, 392, 396, 404

`io`, 332, 332, 380, 382, 392, 396, 396, 404, 424, 424

`io`, 430

`io` module, 388

`io`, 382, 384, 388

`io`, 380

A

abstract base classes, 30

Anaconda, 108

API, 211, 307

`assert`, 44

asynchronous communication, 401

attributes, 136

B

builtin types

emulating, 344

`bz2`, 395

C

C struct, 384

callable, 147

Cassandra, 255

class

defining at runtime, 161

class data, 18

class method, 19

classes, 8

constructors, 13

defining, 9

inheritance, 21

collections, 168

commit, 251

concurrency, 401

`conftest.py`, 58

connection object, 216

constructors, 13

Container classes, 331

context manager, 213

`copy.deepcopy`, 378

Counter, 335

CSV, 84

custom, 87

dialects, 87

parameters, 85

reader, 87

writer, 87

csv

DictReader, 88

`csv.reader()`, 84

`csv.writer()`, 90

curl, 314

cursor, 216

cursor object, 216

`cx_oracle`, 212

D

database engine, 321

database programming, 211

database schema, 321

database server, 213

Databricks, 212

DB API, 211

decorator arguments, 158

decorator class, 155

decorator function, 152

decorators, 147

decorators in the standard library, 148

defaultdict, 336

`delattr()`, 136

DELETE, 310

delete, 205

DELETE request, 303

deque, 341

- dictionary
 - custom, [354](#)
- dictionary cursor
 - emulating, [248](#)
- dictionary cursors, [243](#)
- Django, [254](#)
- Django**, [58](#), [307](#)
- Django ORM, [254](#)
- DML, [230](#)
- Douglas Crockford, [93](#)
- E**
- Elasticsearch, [212](#)
- Element, [109](#)
- Element**, [110](#), [357](#)
- ElementTree, [108](#)
 - find(), [114](#)
 - findall(), [114](#)
- Excel, [212](#)
- exception, [47](#)
- executing SQL statements, [220](#)
- F**
- Firebird (and Interbase, [212](#))
- fixtures*, [50](#)
- fixtures, [41](#)
- Flask
 - popular extensions, [308](#)
- flask
 - extensions, [307](#)
 - session, [321](#)
- Flask extension, [309](#)
- Flask-RESTX**, [309](#)
- Flask-RESTX**, [309](#)
- flask_restx.Resource**, [310](#)
- for*, [168](#)
- functools*, [99](#)
- functools.wraps, [152](#)
- G**
- generator expression, [170](#), [175](#)
- generator function, [170](#), [178](#)
- GET**, [310](#)
- getattr(), [136](#)
- getroot(), [113](#)

- GIL, [403](#)
- globals(), [131](#)
- Google BigQuery, [212](#)
- gzip**, [395](#)
- H**
- hasattr(), [136](#)
- head, [205](#)
- hooks, [58](#)
- http options, [205](#)

- I**
- IBM DB2, [212](#)
- ibm-db, [212](#)
- ingmod, [212](#)
- Ingres, [212](#)
- inheritance*, [21](#)
- inheritance
 - multiple, [27](#)
- inspect* module, [140](#)
- instance attributes, [11](#)
- instance methods, [12](#)
- iterable*, [168](#)
- iterator*, [168](#), [170](#)
- iterator class, [170](#), [173](#)

- J**
- Java, [42](#)
- JSON, [93](#)
 - custom encoding, [99](#)
 - types, [93](#)
- json* module, [94](#)
- JSON response, [194](#)
- json.dump()*, [97](#)
- json.loads()*, [94](#)

- K**
- KInterbasDB, [212](#)

- L**
- lambda function, [380](#)
- lists
 - custom, [350](#)
- locals(), [131](#)
- lxml

Element, 110
SubElement, 110

lxml.etree, 107

M

make_dataclass(), 161
markers, 42
metadata, 240
metaprogramming, 130
microframework, 307
Microsoft SQL Server, 212
mock object, 72
mock objects, 72
models, 321
MongoDB, 255
monkey patches, 163
multiprocessing, 401
 Manager, 421
multiprocessing.dummy.Pool, 424
multiprocessing.Pool, 424
multiprogramming, 401
 alternatives to, 430
MySQL, 212
mysql-connector-python, 212
mysqlclient, 212

N

namedtuple, 338
namedtuple cursor, 248
node ID, 66
non-query statement, 230
non-relational, 255
NoSQL, 255

O

object instance, 10
Object-relational mapper, 254
ODBC, 212
Oracle, 212
oracledb, 212
OrderedDict, 343
ORM, 254

P

parameterized SQL statements, 230

parametrizing, 61

PATCH, 310

patch, 205

pip, 307

plugins, 58

POST, 203, 310

POST data, 302

PostgreSQL, 212

properties, 14

psycopg, 212

PUT, 310

put, 205

py.test, 46

pymock, 72

pymssql, 212

pymysql, 212

pyodbc, 212

PyPI, 387

pytest

 builtin fixtures, 54

 configuring fixtures, 58

 output capture, 46

 special assertions, 47

 user-defined fixtures, 51

 verbose, 46

pytest, 42, 44

pytest-django, 81

pytest-mock, 73

pytest-qt, 81

pytest.approx(), 47

pytest.fixture, 51

pytest.raises(), 47

R

Redis, 255

redis, 58

requests, 314

requests

 methods

 keyword parameters, 197, 363

 Response

 attributes, 198, 364

Resource, 309

resource, 310

REST, [292](#)

 converter, [296](#)

rollback, [251](#)

routes, [309](#)

running tests, [67](#)

 by component, [67](#)

 by mark, [67](#)

 by name, [67](#)

S

SAP DB, [212](#)

sapdbapi, [212](#)

scoped session, [321](#)

setattr(), [136](#)

singledispatch, [99](#)

SOAP, [292](#)

special methods, [32](#), [344](#)

SQL code, [216](#)

SQL data integrity, [251](#)

SQL injection, [226](#)

SQL queries, [220](#)

SQLAlchemy, [254](#)

SQLAlchemy, [321](#)

SQLAlchemy documentation, [322](#)

SQLite, [212](#)

sqlite3, [212](#)

static method, [38](#)

StopIteration, [173](#)

string formatting

 alignment, [443](#)

 data types, [437](#)

 dates, [453](#)

 field widths, [440](#)

 fill characters, [446](#)

 misc, [459](#)

 parameter attributes, [451](#)

 run-time, [457](#)

 selectors, [434](#)

 signed numbers, [448](#)

SubElement, [110](#)

super(), [22](#)

super(), [25](#)

Sybase, [212](#), [212](#)

T

tar, [395](#)

test case, [41](#)

test cases, [41](#)

test runner, [42](#), [46](#)

test runners, [41](#)

tests

 messages, [44](#)

thread, [402](#)

thread class

 creating, [407](#)

threading, [401](#)

threading issues, [321](#)

threads

 debugging, [420](#)

 locks, [410](#)

 queue, [417](#)

 simple, [405](#)

 variable sharing, [410](#)

Thunder Client, [208](#)

Thunderclient, [314](#)

tolerance

 pytest.approx, [47](#)

transactions, [251](#)

Twisted, [430](#)

type(), [161](#)

U

unit test, [41](#)

unit test components, [41](#)

unit tests

 failing, [69](#)

 mock objects, [73](#)

 running, [46](#)

 skipping, [69](#)

unittest.mock, [72](#)

unittest.mock, [73](#)

unpacking function arguments, [186](#)

W

web-based services, [292](#)

X

xfail, [69](#)

XML, [107](#)

 root element, [121](#)

xml.etree.ElementTree, [107](#)

xml.etree.ElementTree, [108](#)

XPASS, [69](#)

XPath, [118](#)

xUnit, [42](#)

Y

yield, [178](#)

Z

zip, [395](#)